Vue.js 3 - 2026

💚 Une formation présentée par Ascent et Andromed.

Appuyez sur espace pour la page suivante →

Programme : **4-047**

Lot: **4**

Jimmylan Surquin

Fondateur Andromed

- Lille, France 🇫🇷
- Création de contenu sur jimmylansrq
- Blog & Portfolio jimmylan.fr



DISCLAIMER 💚

Dans cette formation nous allons voir Vue.js 3 en profondeur, la dernière version du framework progressif JavaScript.



SOMMAIRE VUE.JS 3 - FORMATION 📜

🎯 Fil Rouge : TaskFlow (notre Trello-like)

💚 1. Introduction à Vue.js et bases fondamentales

🎰 2. Slots et composants génériques

🚀 3. Vue.js - Fonctionnement avancé

🛤️ 4. Navigation et routage avec Vue Router

🍍 5. State Management avec Pinia

🧪 6. Tests sous Vue.js avec Vitest

✅ QCM de validation finale

🔗 Lien du projet Github

🎁 **MODULES BONUS**

✨ Composition API

🚀 CI/CD avec GitLab

🛟 CheatSheet d'aide

Note : Chaque module contient :

- 📖 Cours théorique
- 🎯 **Fil rouge TaskFlow** (notre Trello-like progressif)

JOUR 1 - FONDAMENTAUX ET BASES



Module 1 - Introduction à Vue.js et bases fondamentales

- Rappels JavaScript ES6+
- Architecture Vue.js (MVVM)
- Outils et environnement (Node.js, Vite)
- Votre premier composant Vue (data, methods)
- Directives et data binding (v-bind, v-if, v-for, v-on, v-model)
- Communication entre composants (props, emits)
- Réactivité avancée (computed, watchers, lifecycle, mixins)

Module 2 - Slots et composants génériques

- Définition et intérêts des slots
- Slots simples et nommés
- Directive v-slot
- Scoped slots (slots à portée)
- Slots dynamiques
- Combinaison props & slots
- Composants génériques réutilisables

JOUR 2 – VUE.JS AVANCÉ ET ROUTAGE 📅

Module 3 - Vue.js - Fonctionnement avancé

- Single File Components approfondis (SFC)
- Scoped CSS et CSS Modules
- Variables CSS réactives
- Directives personnalisées
- Composants fonctionnels (cas avancés)

Module 4 - Navigation et routage avec Vue Router

- Introduction à Vue Router
- Installation et configuration
- Routes et navigation (RouterLink, RouterView)
- Routes dynamiques et paramètres
- Navigation programmatique
- Guards de navigation (beforeEach, meta fields)
- Routes imbriquées
- Lazy loading

JOUR 3 - STATE MANAGEMENT ET TESTS 📅

Module 5 - State Management avec Pinia

- Pattern State Management
- Introduction à Pinia
- Installation et configuration
- Création d'un store Pinia
- State, Getters, Actions
- Options API avec Pinia
- Communication entre stores
- Persistence localStorage
- Intégration dans une application

Module 6 - Tests sous Vue.js avec Vitest

- Intérêt des tests automatisés
- Pourquoi tester ? (pyramide des tests)
- Installation de Vitest
- Configuration (vite.config.js, jsdom)
- Tests unitaires (fonctions, composables)
- Tests de composants Vue
- Tests avec Pinia
- Tests avec Router
- Mocks et stubs
- Coverage de code

Projet Final & QCM

- Application de tous les concepts
- QCM de validation finale

Modules Bonus (optionnels)

- Composition API
- CI/CD avec GitLab

Module 1

Introduction à Vue.js et bases fondamentales

Revenir au sommaire

SOMMAIRE DU MODULE

🎯 1.1 Rappels JavaScript ES6+

🏗️ 1.2 Architecture Vue.js

🛠️ 1.3 Outils et environnement

🧩 1.4 Votre premier composant Vue

⚡ 1.5 Directives et data binding

💬 1.6 Communication entre composants

🔄 1.7 Réactivité avancée

♻️ 1.8 Cycle de vie des composants

💪 Exercices Pratiques du Module 1

Revenir au sommaire

1.1 Rappels JavaScript ES6+

Arrow Functions

Les fonctions fléchées offrent une syntaxe plus concise et un binding lexical de `this`

```
// Fonction traditionnelle
function addition(a, b) {
  return a + b;
}

// Arrow function
const addition = (a, b) => a + b;

// Avec bloc
const calculer = (a, b) => {
  const resultat = a + b;
  return resultat;
};
```

Revenir au sommaire

Arrow Functions - Cas d'usage (1/2)

Méthodes de tableaux avec arrow functions

```
// Dans les méthodes de tableaux
const nombres = [1, 2, 3, 4, 5];

const doubles = nombres.map(n => n * 2);
// [2, 4, 6, 8, 10]

const pairs = nombres.filter(n => n % 2 === 0);
// [2, 4]

const somme = nombres.reduce((acc, n) => acc + n, 0);
// 15
```

Revenir au sommaire

Arrow Functions - Cas d'usage (2/2)

Binding lexical de `this`

```
// Binding lexical de 'this'
class Composant {
  constructor() {
    this.valeur = 42;
  }

  methode = () => {
    console.log(this.valeur); // 42
  }
}
```

Revenir au sommaire

Destructuring - Objets

Le destructuring permet d'extraire des valeurs des objets et tableaux

```
// Destructuring d'objet
const utilisateur = {
  nom: 'Dupont',
  prenom: 'Jean',
  age: 30,
  email: 'jean@example.com'
};

const { nom, prenom } = utilisateur;
console.log(nom); // 'Dupont'

// Avec renommage
const { nom: nomFamille } = utilisateur;
console.log(nomFamille); // 'Dupont'
```

Revenir au sommaire

Destructuring - Tableaux et cas avancés

```
// Destructuring de tableau
const nombres = [1, 2, 3, 4, 5];
const [premier, deuxième, ...reste] = nombres;
console.log(premier); // 1
console.log(reste); // [3, 4, 5]

// Dans les paramètres de fonction
const afficherUtilisateur = ({ nom, prenom }) => {
  console.log(`${prenom} ${nom}`);
};

// Valeurs par défaut
const { age = 18 } = utilisateur;
```

Revenir au sommaire

Modules ES6

Les modules permettent d'organiser et de réutiliser le code

```
// exports nommés (math.js)
export const PI = 3.14159;
export const addition = (a, b) => a + b;
export const multiplication = (a, b) => a * b;

// export par défaut (utilisateur.js)
export default class Utilisateur {
  constructor(nom) {
    this.nom = nom;
  }
}
```

Revenir au sommaire

Imports de modules

```
// Import nommé
import { PI, addition } from './math.js';
console.log(addition(2, 3)); // 5

// Import par défaut
import Utilisateur from './utilisateur.js';
const user = new Utilisateur('Jean');

// Import avec alias
import { addition as add } from './math.js';

// Import tout
import * as Math from './math.js';
console.log(Math.PI);
```

Revenir au sommaire

Promises - Gestion asynchrone

Les Promises représentent une valeur qui peut être disponible maintenant, plus tard, ou jamais

```
// Création d'une Promise
const fetchData = () => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const data = { id: 1, nom: 'Produit' };
      resolve(data);
      // ou : reject(new Error('Erreur'));
    }, 1000);
  });
};

// Utilisation
fetchData()
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log('Terminé'));
```

Revenir au sommaire

Async / Await

Syntaxe plus claire pour gérer les opérations asynchrones

```
// Fonction asynchrone
const chargerUtilisateur = async (id) => {
  try {
    const response = await fetch(`/api/users/${id}`);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Erreur:', error);
    throw error;
  }
};

// Utilisation
const utilisateur = await chargerUtilisateur(1);
console.log(utilisateur);
```

Revenir au sommaire

Async / Await - Opérations parallèles

```
// Opérations séquentielles (lent)
const user = await fetchUser();
const posts = await fetchPosts();
const comments = await fetchComments();

// Opérations parallèles (rapide)
const [user, posts, comments] = await Promise.all([
  fetchUser(),
  fetchPosts(),
  fetchComments()
]);

// Promise.race - première qui se termine
const resultat = await Promise.race([
  fetchFromAPI1(),
  fetchFromAPI2()
]);
```

Revenir au sommaire

1.2 Architecture Vue.js

Revenir au sommaire

Présentation de Vue.js

Vue.js est un framework JavaScript progressif pour construire des interfaces utilisateur

Caractéristiques principales

- 🎯 **Progressif** : peut être adopté progressivement (un composant, puis toute l'app)
- ⚡ **Performant** : Virtual DOM et réactivité optimisée
- 🔧 **Flexible** : s'adapte à tous types de projets (widget, SPA, SSR...)
- 📦 **Écosystème riche** : Vue Router, Pinia, Vite... tout est intégré harmonieusement
- 💚 **Communauté active** : support et ressources

Revenir au sommaire

Architecture MVVM

Vue.js suit le pattern MVVM (Model-View-ViewModel)

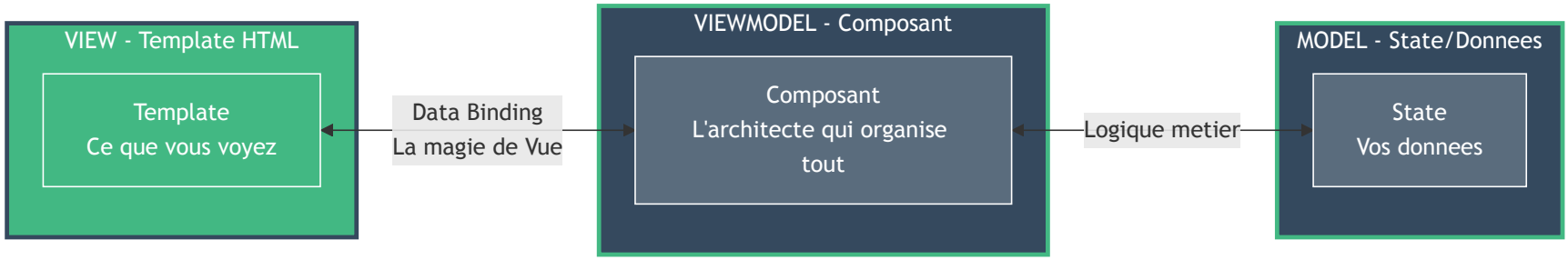
🏗️ Pensez à une maison (vue, composant, state)

Nous allons voir un schéma à la slide suivante, mais pour résumer :

- VIEW : Template HTML (html)
- VIEWMODEL : Composant (script et template)
- MODEL : State / Données (data)

Revenir au sommaire

Schéma : Architecture MVVM



Revenir au sommaire

MVVM dans Vue.js - Exemple (1/2)

MODEL + VIEWMODEL (Script)

```
<script>
export default {
  // MODEL - Données
  data() {
    return {
      compteur: 0,
      utilisateur: {
        nom: 'Jean',
        email: 'jean@example.com'
      }
    }
  },
  // VIEWMODEL - Logique
  methods: {
    incrementer() {
      this.compteur++;
    }
  }
}
</script>
```

Revenir au sommaire

MVVM dans Vue.js - Exemple (2/2)

VIEW (Template)

```
<template>
  <!-- VIEW - Interface -->
  <div>
    <h1>{{ utilisateur.nom }}</h1>
    <p>Compteur: {{ compteur }}</p>
    <button @click="incrementer">+1</button>
  </div>
</template>
```

Revenir au sommaire

Organisation d'une application Vue

Structure typique d'un projet Vue.js moderne

```
mon-projet/
├── src/
│   ├── components/      # Composants réutilisables
│   ├── views/           # Pages/Vues
│   ├── composables/     # Logique réutilisable
│   ├── stores/          # State management (Pinia)
│   ├── router/          # Configuration des routes
│   ├── assets/          # Ressources (images, CSS)
│   ├── App.vue          # Composant racine
│   └── main.js          # Point d'entrée
├── public/              # Fichiers statiques
└── package.json         # Dépendances
```

Revenir au sommaire

1.3 Outils et environnement

Revenir au sommaire

Node.js et npm

Node.js : environnement d'exécution JavaScript côté serveur

npm (Node Package Manager) : gestionnaire de paquets

```
# Vérifier les versions
node --version  # v20.x ou supérieur
npm --version   # v10.x ou supérieur
```

Vous pouvez utiliser nvm (Node Version Manager) pour gérer les versions de Node.js.

Je vous expliquerais en live pour l'installation de nvm et de Node.js.

Concernant les autres gestionnaires de paquets, vous pouvez utiliser pnpm, yarn, bun, etc.

- Pnpm : gestionnaire de paquets rapide et efficace, vous permez de centraliser les dépendances de vos projets.
- Yarn : gestionnaire de paquets rapide , créer par Facebook (meta)
- Bun : gestionnaire de paquets ultra rapide, mais moins mature que pnpm et yarn.

Revenir au sommaire

Vite - Build tool moderne ⚡

Vite est l'outil de build recommandé pour Vue.js 3 en 2025

🏎️ Analogie : Vite vs Webpack

- **Webpack** = Préparer TOUS les ingrédients avant de cuisiner (lent au démarrage)
- **Vite** = Cuisiner à la demande, seulement ce dont vous avez besoin (instantané !)

> Je vous le dis de suite, mais nous n'utiliserons plus webpack en 2026, sur aucun projets.

Avantages de Vite

- ⚡ **Démarrage instantané** : < 1 seconde vs 30-60 sec avec Webpack
- 🔥 **Hot Module Replacement (HMR) ultra-rapide** : vos changements apparaissent instantanément
- 📦 **Build optimisé avec Rollup** : bundle de production ultra-performant
- 🎯 **Support TypeScript natif** : pas de configuration nécessaire
- 🔌 **Système de plugins extensible** : écosystème riche

Revenir au sommaire

💡 Fun fact

Vite signifie "rapide" en français ! Et créé par Evan You, le créateur de Vue.js lui-même 🇫🇷

```
# Créer un projet avec Vite (ULTRA simple)
npm create vite@latest mon-projet -- --template vue
```

avec pnpm :

```
pnpm create vite@latest mon-projet -- --template vue
```

On peut l'écrire de différentes façons, avec npx par exemple :

```
npx create-vite@latest mon-projet -- --template vue
```

npx est un outil qui permet d'exécuter des commandes sans installer de package globalement. Il est très pratique pour exécuter des commandes sans installer de package sur son pc.

Revenir au sommaire

Créer un projet avec Vite

```
# Créer le projet
npm create vite@latest mon-app-vue -- --template vue

# Se déplacer dans le dossier
cd mon-app-vue

# Installer les dépendances
npm install

# Lancer le serveur de développement
npm run dev
```

Revenir au sommaire

Structure d'un projet Vue

```
mon-app-vue/
├── node_modules/        # Dépendances
├── public/
│   └── favicon.ico      # Icône du site
├── src/
│   ├── assets/          # Images, CSS, fonts
│   ├── components/      # Composants Vue
│   ├── App.vue          # Composant principal
│   └── main.js          # Point d'entrée
├── index.html           # Page HTML principale
├── package.json         # Configuration npm
├── vite.config.js       # Configuration Vite
└── README.md
```



Revenir au sommaire

Le fichier main.js

Point d'entrée de l'application

```
import { createApp } from 'vue';
import './style.css';
import App from './App.vue';

// Création de l'application
const app = createApp(App);

// Montage de l'application
app.mount('#app');
```

Revenir au sommaire

Le fichier App.vue (1/2)

Composant racine de l'application - **Script et Template**

```
<script>
import HelloWorld from './components/HelloWorld.vue';

export default {
  components: {
    HelloWorld
  },
  data() {
    return {
      count: 0
    }
  }
}
</script>

<template>
  <div>
    <HelloWorld msg="Vue.js 3 + Vite" />
    <button @click="count++">
      Compteur: {{ count }}
    </button>
  </div>
</template>

<style scoped>
<!-- scoped = les styles sont uniquement appliqués à ce composant -->
button {
  padding: 10px 20px;
  font-size: 16px;
}
</style>
```



Revenir au sommaire

Configuration Vite

Fichier `vite.config.js`

```
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';
import path from 'path';

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src')
    }
  },
  server: {
    port: 3000,
    open: true
  }
});
```

Revenir au sommaire

Alias de chemins

Pour le resolve : on peut utiliser un alias pour les chemins de fichiers.

```
alias: {
  '@': path.resolve(__dirname, './src')
}
```

Cela permet de simplifier les chemins de fichiers dans le code.

```
import HelloWorld from '@/components/HelloWorld.vue';
```

Utile, à la place de mettre le chemin relatif : `./components/HelloWorld.vue` , on peut utiliser l'alias `@` pour le chemin `./src/components/HelloWorld.vue` .



Revenir au sommaire

1.4 Votre premier composant Vue

Revenir au sommaire

Qu'est-ce qu'un composant ?

Les composants sont des **briques réutilisables** qui encapsulent :

- 📝 Template (structure HTML)
- 🎨 Styles (CSS)
- ⚙️ Logique (JavaScript)

🧱 Analogie : Les composants sont comme des Lego

- Vous **réutilisez** les mêmes briques partout
- Chaque brique a sa **responsabilité**
- Vous pouvez **assembler** ces briques de différentes façons

Revenir au sommaire

Création d'un composant simple

Fichier `MonCompteur.vue` - **Options API**

```
<script>
export default {
  data() {
    return {
      compteur: 0
    }
  },
  methods: {
    incrementer() {
      this.compteur++;
    },
    decrementer() {
      this.compteur--;
    }
  }
}
</script>

<template>
  <div>
    <h2>Compteur: {{ compteur }}</h2>
    <button @click="incrementer">+</button>
    <button @click="decrementer">-</button>
  </div>
</template>

<style scoped>
button {
  padding: 10px 20px;
  margin: 5px;
  background: #42b883;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
</style>
```



Revenir au sommaire

Utiliser un composant

```
<script>
import MonCompteur from './components/MonCompteur.vue';

export default {
  components: {
    MonCompteur
  }
}
</script>

<template>
  <div>
    <h1>Mon Application</h1>
    <MonCompteur />
    <MonCompteur />
  </div>
</template>
```

Chaque instance est **indépendante** !



Revenir au sommaire

1.5 Directives et data binding

Revenir au sommaire

Directives et data binding

Nous allons parcourir les directives et les data binding de Vue.js.

Nous allons commencer par le data binding et en suite faire un petit tour sur les directives et les conditions.

Revenir au sommaire

Liaison de données (Data Binding)

Vue.js permet de lier les données du composant au template avec l'**Options API**

🔌 C'est comme brancher une multiprise électrique

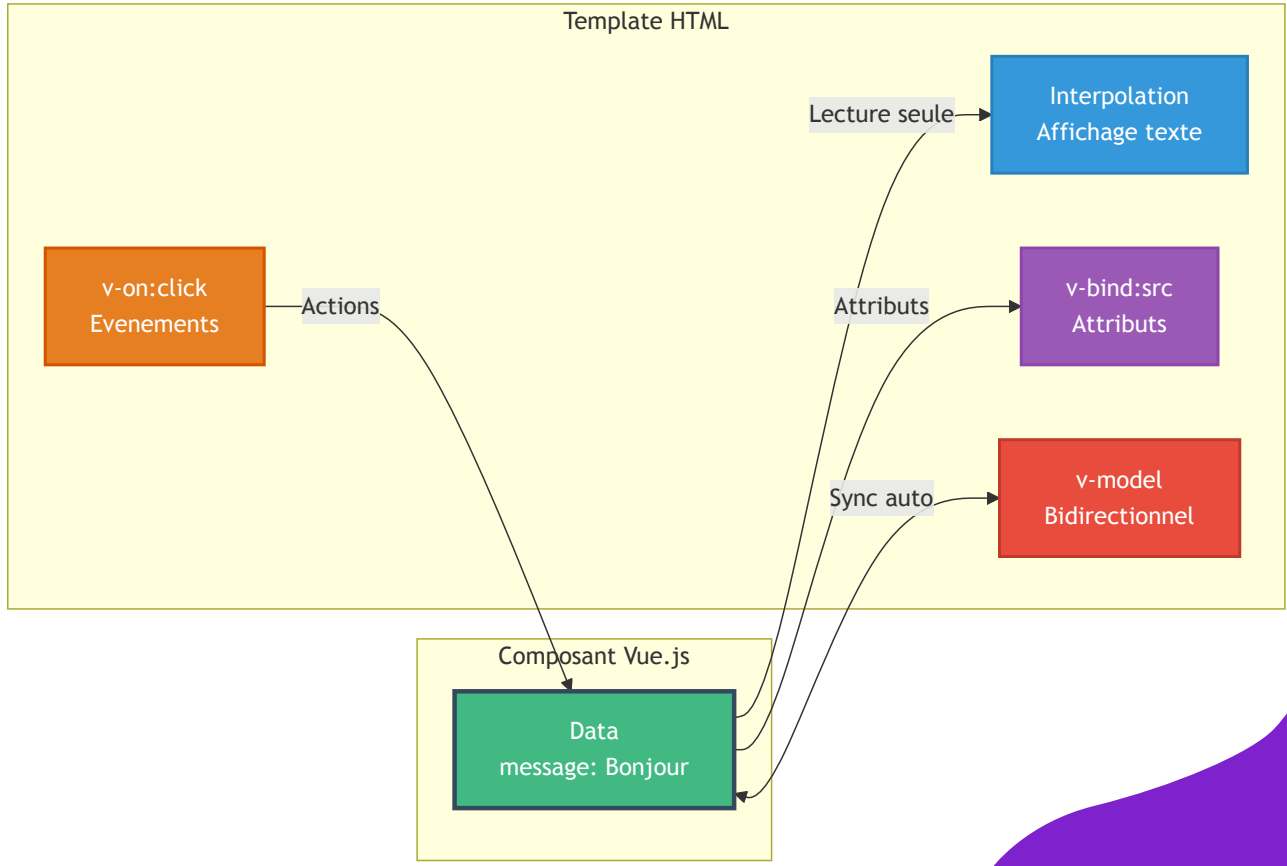
Imaginez que vos données sont l'électricité, et votre template est l'ampoule. **Vue.js est la multiprise** qui les connecte automatiquement !

Types de liaison (les 4 prises)

- **Interpolation :** `{{ variable }}` → Afficher du texte (lecture seule) 📖
- **Attributs :** `v-bind:attr="value"` ou `:attr="value"` → Modifier les attributs HTML 🎨
- **Événements :** `v-on:event="handler"` ou `@event="handler"` → Écouter les actions utilisateur 👂
- **Bidirectionnelle :** `v-model="variable"` → Synchronisation automatique ↔️

Revenir au sommaire

Schéma : Data Binding



Revenir au sommaire

La réactivité de Vue

Avec Vue : changez la variable, l'affichage se met à jour automatiquement ! ✨

C'est ça la **réactivité** : Vue observe vos données et met à jour le DOM automatiquement ! (comme React et Angular)



Revenir au sommaire

Interpolation (Mustache) - 1/2

Affichage de données dans le template avec `{{ }}` - **Options API**

Les données :

```
<script>
export default {
  data() {
    return {
      message: 'Bonjour Vue.js!',
      nombre: 42,
      utilisateur: {
        nom: 'Dupont',
        prenom: 'Jean'
      }
    }
  }
}
</script>
```

Revenir au sommaire

Interpolation (Mustache) - 2/2

Utilisation dans le template :

```
<template>
  <div>
    <h1>{{ message }}</h1>
    <p>Nombre: {{ nombre }}</p>
    <p>Nom complet: {{ utilisateur.prenom }} {{ utilisateur.nom }}</p>
    <p>Calcul: {{ nombre * 2 }}</p>
    <p>Méthode: {{ message.toUpperCase() }}</p>
  </div>
</template>
```

Revenir au sommaire

Directives

Les directives sont des attributs HTML qui permettent de manipuler le DOM de manière dynamique.

Elles sont préfixées par `v-` et sont utilisées pour lier des données au template.

Voici une liste des directives que nous allons voir :

- v-bind
- v-on
- v-if
- v-else
- v-show
- v-for
- v-model

Nous verrons les autres directives dans un autre module plus tard.

Revenir au sommaire

Directive v-bind (1/2)

Lie un attribut HTML à une expression JavaScript - **Options API**

Les données :

```
<script>
export default {
  data() {
    return {
      imageUrl: '/logo.png',
      altText: 'Logo Vue.js',
      isDisabled: false,
      styles: {
        color: 'blue',
        fontSize: '20px'
      }
    }
  }
}
</script>
```



Revenir au sommaire

Directive v-bind (2/2)

Utilisation dans le template :

```
<template>
  <img v-bind:src="imageUrl" v-bind:alt="altText" />
  <!-- Syntaxe courte -->
  <img :src="imageUrl" :alt="altText" />

  <button :disabled="isDisabled">Cliquer</button>
  <div :style="styles">Texte stylé</div>
</template>
```

Revenir au sommaire

Directive v-on (événements) - 1/2

Écoute les événements DOM - **Options API**

Logique :

```
<script>
export default {
  data() {
    return {
      compteur: 0,
      message: ''
    }
  },
  methods: {
    incrementer() {
      this.compteur++;
    },
    afficherMessage(event) {
      console.log('Événement:', event);
      this.message = 'Bouton cliqué!';
    }
  }
}
</script>
```

Revenir au sommaire

Directive v-on (événements) - 2/2

Template :

```
<template>
  <button v-on:click="incrementer">+1</button>
  <!-- Syntaxe courte -->
  <button @click="compteur++">+1</button>
  <button @click="afficherMessage">Message</button>
  <p>{{ compteur }}</p>
</template>
```

Revenir au sommaire

Modificateurs d'événements

Vue.js propose des modificateurs pour faciliter la gestion des événements

```
<template>
  <!-- Empêcher le comportement par défaut -->
  <form @submit.prevent="onSubmit">
    <button type="submit">Envoyer</button>
  </form>

  <!-- Stopper la propagation -->
  <div @click="handleClick">
    <button @click.stop="handleButtonClick">Cliquer</button>
  </div>

  <!-- Une seule fois -->
  <button @click.once="init">Initialiser</button>

  <!-- Combinaisons -->
  <input @keyup.enter="submit" />
  <input @keyup.ctrl.enter="send" />
</template>
```

Revenir au sommaire

Directive v-if / v-else (1/2)

Affichage conditionnel (ajout/suppression du DOM) - **Options API**

Données :

```
<script>
export default {
  data() {
    return {
      isLoggedIn: false,
      userRole: 'admin',
      score: 75
    }
  }
}
</script>
```

Revenir au sommaire

Directive v-if / v-else (2/2)

Template avec conditions :

```
<template>
  <div v-if="isLoggedIn">
    <h1>Bienvenue!</h1>
  </div>
  <div v-else>
    <h1>Veuillez vous connecter</h1>
  </div>

  <div v-if="userRole === 'admin'">Admin</div>
  <div v-else-if="userRole === 'user'">Utilisateur</div>
  <div v-else>Invité</div>
</template>
```



Revenir au sommaire

Directive v-show

Alternative à `v-if` qui utilise CSS (display: none) - **Options API**

```
<script>
export default {
  data() {
    return {
      isVisible: true
    }
  }
}
</script>

<template>
  <div v-show="isVisible">
    Ce contenu est visible
  </div>

  <button @click="isVisible = !isVisible">
    Toggle
  </button>
</template>
```

Revenir au sommaire

v-if vs v-show : La bataille ! ⚔️

🏗️ v-if : Le démolisseur/constructeur

- **Détruit** et **recrée** l'élément dans le DOM à chaque fois
- Comme démolir et reconstruire un mur de Lego à chaque fois
- **Coût élevé** si vous toggle souvent
- ✅ **Utilisez si** : l'élément change rarement

👻 v-show : Le magicien de l'invisibilité

- L'élément reste dans le DOM, juste caché avec `display: none`
- Comme mettre un drap d'invisibilité sur quelque chose
- **Coût faible** pour toggler
- ✅ **Utilisez si** : vous toggle souvent (menu dropdown, modal...)

🎯 Règle d'or

Toggle fréquent (ex: dropdown menu) → `v-show` **Condition rare** (ex: admin panel) → `v-if`

Revenir au sommaire

Directive v-for (1/2)

Rendu de listes - **Options API**

Données :

```
<script>
export default {
  data() {
    return {
      fruits: ['Pomme', 'Banane', 'Orange'],
      utilisateurs: [
        { id: 1, nom: 'Alice', age: 25 },
        { id: 2, nom: 'Bob', age: 30 },
        { id: 3, nom: 'Charlie', age: 35 }
      ]
    }
  }
}
</script>
```



Revenir au sommaire

Directive v-for (2/2)

Utilisation :

```
<template>
  <!-- Tableau simple -->
  <ul>
    <li v-for="(fruit, index) in fruits" :key="index">
      {{ index + 1 }}. {{ fruit }}
    </li>
  </ul>
</template>
```

Revenir au sommaire

v-for avec objets

```vue
<template>
  <!-- Tableau d'objets -->
  <div v-for="user in utilisateurs" :key="user.id">
    <h3>{{ user.nom }}</h3>
    <p>Âge: {{ user.age }}</p>
  </div>

  <!-- Objet -->
  <div v-for="(value, key, index) in objet" :key="key">
    {{ index }}. {{ key }}: {{ value }}
  </div>

  <!-- Nombre -->
  <span v-for="n in 10" :key="n">{{ n }} </span>
</template>
```

Revenir au sommaire

v-for : Point important

⚠️ **Important** : toujours utiliser `:key` avec une valeur unique pour les performances

Le `:key` aide Vue à identifier chaque élément de façon unique et optimiser le rendu.

Revenir au sommaire

v-model - Liaison bidirectionnelle (1/2)

Synchronise automatiquement les données avec les inputs - **Options API**

Données :

```
<script>
export default {
  data() {
    return {
      texte: '',
      isChecked: false,
      selected: '',
      message: ''
    }
  }
}
</script>
```

Revenir au sommaire

v-model - Liaison bidirectionnelle (2/2)

Template :

```
<template>
  <input v-model="texte" type="text" />
  <p>Texte: {{ texte }}</p>

  <input v-model="isChecked" type="checkbox" />
  <p>Coché: {{ isChecked }}</p>

  <select v-model="selected">
    <option>A</option>
    <option>B</option>
  </select>

  <textarea v-model="message"></textarea>
</template>
```

Revenir au sommaire

Présentation de v-model

v-model est un sucre syntaxique pour :

```
<!-- Ces deux lignes sont équivalentes -->
<input v-model="texte" />
<input
  :value="texte"
  @input="texte = $event.target.value"
/>
```

En fait ce principe s'appelle le two-way data binding ou le banana in the box.

two-way c'est bidirectionnel, c'est à dire que la donnée est liée à la vue et à la vue est liée à la donnée.

Elle s'injecte dans le template, et elle est liée à la donnée, donc si on met à un jour via un input par exemple, a donnée sera mise à jour en direct.

Très connu chez Angular , donc en clair , quand vous apprenez une librarie ou un framework js moderne, vous allez pouvoir repetez la même chose. :)

Revenir au sommaire

1.6 Communication entre composants

Revenir au sommaire

Props - Passer des données aux enfants

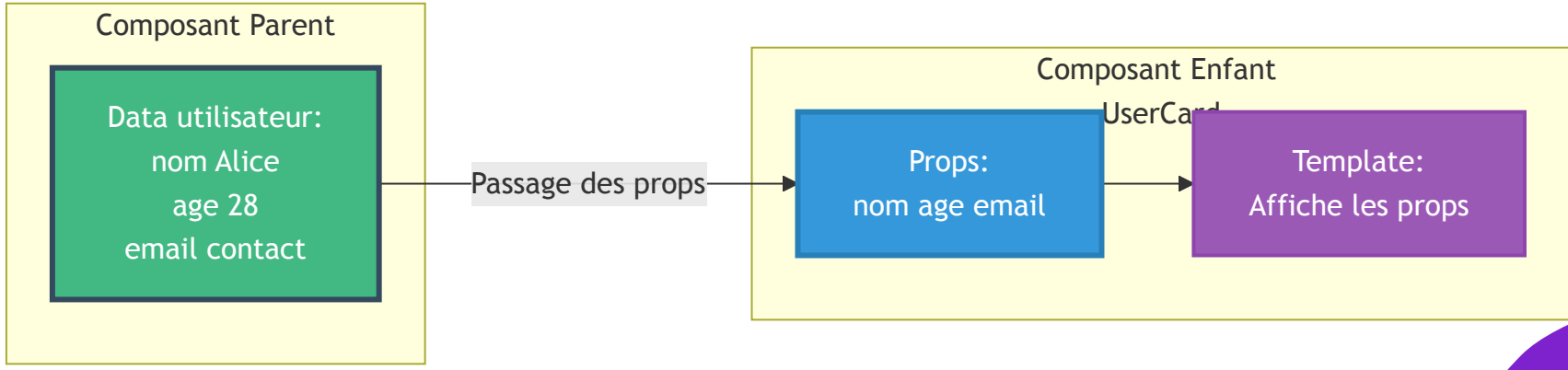
Les props permettent de passer des données du parent à l'enfant - **Options API**

```vue
<!-- UserCard.vue -->
<script>
export default {
  props: {
    nom: String,
    age: Number,
    email: {
      type: String,
      required: true
    }
  }
}
</script>

<template>
  <div class="card">
    <h2>{{ nom }}</h2>
    <p>Âge: {{ age }}</p>
    <p>Email: {{ email }}</p>
  </div>
</template>
```

Revenir au sommaire

Schéma : Communication par Props



Revenir au sommaire

Utilisation des props (1/2)

Script avec les données :

```
<script>
import UserCard from './components/UserCard.vue';

export default {
  components: {
    UserCard
  },
  data() {
    return {
      utilisateur: {
        nom: 'Alice',
        age: 28,
        email: 'alice@example.com'
      }
    }
  }
}
</script>
```

Revenir au sommaire

Utilisation des props (2/2)

Template - passage des props :

```vue
<template>
  <UserCard
    :nom="utilisateur.nom"
    :age="utilisateur.age"
    :email="utilisateur.email"
  />

  <!-- Spread operator -->
  <UserCard v-bind="utilisateur" />
</template>
```

Revenir au sommaire

Validation des props (1/2)

Types et validation de base :

```
<script>
export default {
  props: {
    // Type simple
    titre: String,

    // Type multiple
    id: [String, Number],

    // Obligatoire
    nom: {
      type: String,
      required: true
    }
  }
}
</script>
```

Revenir au sommaire

Validation des props (2/2)

Valeurs par défaut :

```
<script>
export default {
  props: {
    // Valeur par défaut
    age: {
      type: Number,
      default: 18
    },

    // Objet/Tableau par défaut (fonction)
    options: {
      type: Object,
      default: () => ({})
    }
  }
}
</script>
```

Revenir au sommaire

Validation avancée des props (1/2)

Validation personnalisée et enum :

```
<script>
export default {
  props: {
    // Validation personnalisée
    email: {
      type: String,
      validator: (value) => {
        return value.includes('@');
      }
    },

    // Enum
    statut: {
      type: String,
      validator: (value) => {
        return ['actif', 'inactif', 'attente'].includes(value);
      }
    }
  }
}
</script>
```

Revenir au sommaire

Validation avancée des props (2/2)

Type personnalisé complexe :

```
<script>
export default {
  props: {
    // Type personnalisé
    utilisateur: {
      type: Object,
      validator: (value) => {
        return value.hasOwnProperty('nom') &&
               value.hasOwnProperty('email');
      }
    }
  }
}
</script>
```

Revenir au sommaire

Événements personnalisés (emit)

Les enfants peuvent communiquer avec les parents via des événements - **Options API**

🎯 Concept : Communication enfant → parent

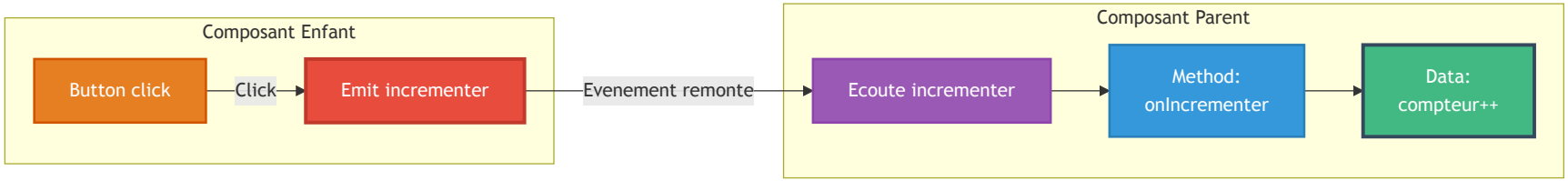
- **Props** : Parent → Enfant (données descendantes ⬇️)
- **Events (emit)** : Enfant → Parent (événements remontants ⬆️)

📞 Analogie : Appel téléphonique

- Le **parent** attend un appel (écoute l'événement)
- L'**enfant** appelle quand il a besoin de quelque chose (émet l'événement)

Revenir au sommaire

Schéma : Communication par Events



Revenir au sommaire

Étape 1 : Le composant Parent (1/2)

Le parent qui écoute les événements

Script :

```
<script>
import CompteurEnfant from './components/CompteurEnfant.vue';

export default {
  components: {
    CompteurEnfant
  },
  data() {
    return {
      compteur: 0
    }
  },
  methods: {
    onIncrementer(valeur) {
      this.compteur += valeur;
    },
    onDecrementer(valeur) {
      this.compteur -= valeur;
    }
  }
}
</script>
```

Revenir au sommaire

Étape 1 : Le composant Parent (2/2)

Template - le parent écoute avec @

```
<template>
  <div>
    <h1>Compteur Parent: {{ compteur }}</h1>

    <!-- Le parent ÉCOUTE les événements de l'enfant -->
    <CompteurEnfant
      @incrementer="onIncrementer"
      @decrementer="onDecrementer"
    />
  </div>
</template>
```

Le parent **attend** que l'enfant émette les événements `incrementer` et `decrementer` .



Revenir au sommaire

Étape 2 : Le composant Enfant (1/2)

L'enfant qui émet les événements

Script :

```vue
<!-- CompteurEnfant.vue -->
<script>
export default {
  // Déclaration des événements que ce composant peut émettre
  emits: ['incrementer', 'decrementer'],

  methods: {
    handleIncrement() {
      // Émet l'événement 'incrementer' avec la valeur 1
      this.$emit('incrementer', 1);
    },
    handleDecrement() {
      // Émet l'événement 'decrementer' avec la valeur 1
      this.$emit('decrementer', 1);
    }
  }
}
</script>
```

Revenir au sommaire

Étape 2 : Le composant Enfant (2/2)

Template :

```
<template>
  <div>
    <button @click="handleIncrement">+</button>
    <button @click="handleDecrement">-</button>
  </div>
</template>
```

Quand on clique sur les boutons, l'enfant **émet** les événements vers le parent.

Revenir au sommaire

Validation des événements (1/2)

Validation des types d'événements :

```
<script>
export default {
  emits: {
    // Validation simple
    incrementer: (value) => {
      return typeof value === 'number' && value > 0;
    },
    decrementer: (value) => {
      return typeof value === 'number' && value > 0;
    }
  }
}
</script>
```

Revenir au sommaire

Validation des événements (2/2)

Validation multi-paramètres :

```
<script>
export default {
  emits: {
    change: (id, value) => {
      return typeof id === 'string' && id.length > 0;
    }
  },
  methods: {
    handleIncrement() {
      this.$emit('incrementer', 1);
    }
  }
}
</script>
```

Revenir au sommaire

Composants dynamiques (1/2)

Changer de composant dynamiquement avec `<component>`

Script :

```
<script>
import Accueil from './components/Accueil.vue';
import Profil from './components/Profil.vue';
import Parametres from './components/Parametres.vue';

export default {
  components: {
    Accueil,
    Profil,
    Parametres
  },
  data() {
    return {
      currentComponent: 'Accueil'
    }
  },
  computed: {
    components() {
      return { Accueil, Profil, Parametres };
    }
  }
}
</script>
```

Revenir au sommaire

Composants dynamiques (2/2)

Template :

```
<template>
  <div>
    <button @click="currentComponent = 'Accueil'">Accueil</button>
    <button @click="currentComponent = 'Profil'">Profil</button>
    <button @click="currentComponent = 'Parametres'">Paramètres</button>

    <component :is="components[currentComponent]" />
  </div>
</template>
```

Revenir au sommaire

Gestion de listes de composants (1/3)

Import et données :

```
<script>
import TodoItem from './components/TodoItem.vue';

export default {
  components: {
    TodoItem
  },
  data() {
    return {
      todos: [
        { id: 1, text: 'Apprendre Vue.js', done: false },
        { id: 2, text: 'Créer un projet', done: false },
        { id: 3, text: 'Déployer', done: false }
      ]
    }
  }
}
</script>
```

Revenir au sommaire

Gestion de listes de composants (2/3)

Méthodes :

```
<script>
export default {
  methods: {
    ajouterTodo(text) {
      this.todos.push({
        id: Date.now(),
        text,
        done: false
      });
    },
    toggleTodo(todo) {
      todo.done = !todo.done;
    }
  }
}
</script>
```

Revenir au sommaire

Gestion de listes de composants (3/3)

Template avec v-for :

```
<template>
  <TodoItem
    v-for="todo in todos"
    :key="todo.id"
    :todo="todo"
    @toggle="toggleTodo(todo)"
  />
</template>
```

Revenir au sommaire

Affichage conditionnel de composants (1/2)

Script :

```
<script>
import LoginForm from './components/LoginForm.vue';
import Dashboard from './components/Dashboard.vue';
import AdminPanel from './components/AdminPanel.vue';
import UserPanel from './components/UserPanel.vue';

export default {
  components: {
    LoginForm,
    Dashboard,
    AdminPanel,
    UserPanel
  },
  data() {
    return {
      isLoggedIn: false,
      userRole: 'user'
    }
  },
  methods: {
    handleLogin() {
      this.isLoggedIn = true;
    }
  }
}
</script>
```

Revenir au sommaire

Affichage conditionnel de composants (2/2)

Template :

```
<template>
  <div>
    <LoginForm v-if="!isLoggedIn" @login="handleLogin" />

    <Dashboard v-else>
      <AdminPanel v-if="userRole === 'admin'" />
      <UserPanel v-else />
    </Dashboard>
  </div>
</template>
```

Revenir au sommaire

1.7 Réactivité avancée

Revenir au sommaire

Propriétés calculées (computed) - 1/2

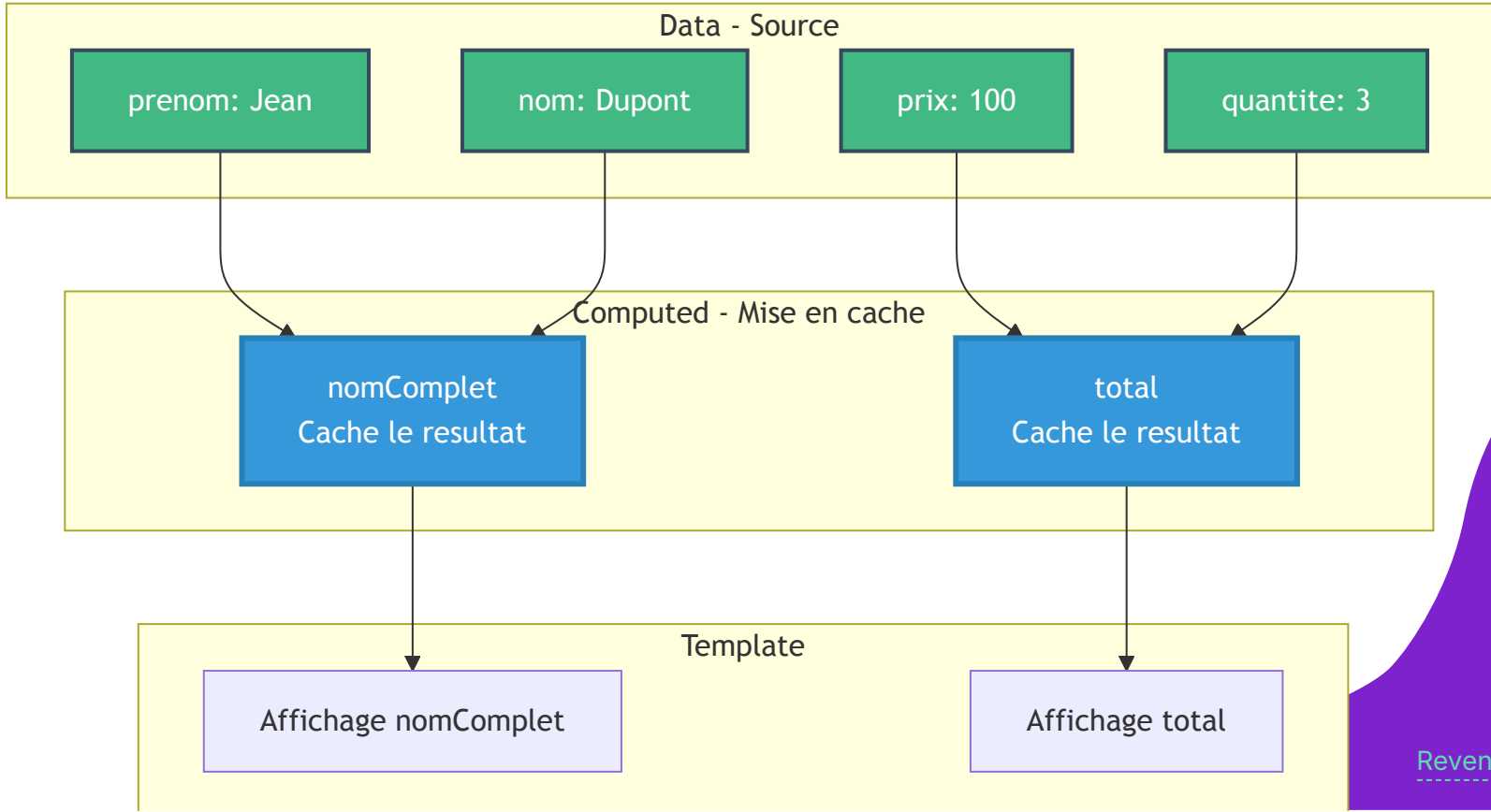
Les propriétés calculées sont des valeurs dérivées qui se mettent à jour automatiquement - **Options API**

Script :

```
<script>
export default {
  data() {
    return {
      prenom: 'Jean',
      nom: 'Dupont',
      prix: 100,
      quantite: 3
    }
  },
  computed: {
    nomComplet() {
      return `${this.prenom} ${this.nom}`;
    },
    total() {
      return this.prix * this.quantite;
    }
  }
}
</script>
```

Revenir au sommaire

Schéma : Computed Properties



Revenir au sommaire

Propriétés calculées (computed) - 2/2

Template :

```
<template>
  <p>{{ nomComplet }}</p>
  <p>Total: {{ total }}€</p>
</template>
```

Revenir au sommaire

Computed vs Methods : Le match ! 🥊 (1/2)

💾 Computed : Le malin qui mémorise

Comme un étudiant qui note sa réponse et ne recalcule que si la question change !

```
<script>
export default {
  data() {
    return {
      nombres: [1, 2, 3, 4, 5]
    }
  },
  computed: {
    // Computed = mise en cache
    sommeComputed() {
      console.log('Calcul computed'); // Affiché 1 seule fois !
      return this.nombres.reduce((a, b) => a + b, 0);
    }
  },
  methods: {
    // Method = recalculé à chaque fois
    sommeMethod() {
      console.log('Calcul method'); // Affiché 2 fois !
      return this.nombres.reduce((a, b) => a + b, 0);
    }
  }
}
</script>
```

Revenir au sommaire

Computed vs Methods : Le match ! 🥊 (2/2)

Template :

```
<template>
  <p>{{ sommeComputed }}</p> <!-- Calculé 1 fois ✨ -->
  <p>{{ sommeComputed }}</p> <!-- Cache utilisé 💨 -->
  <p>{{ sommeMethod() }}</p> <!-- Calculé 🔄 -->
  <p>{{ sommeMethod() }}</p> <!-- Recalculé 🔄 -->
</template>
```

🎯 Règle d'or

Computed : valeur dérivée qui dépend d'autres valeurs (nom complet = prénom + nom) **Methods** : actions/opérations (envoyer email, sauvegarder...)

Revenir au sommaire

Observateurs (watch)

Les **watchers** permettent d'exécuter du code quand une donnée change - **Options API**

🎯 Quand utiliser watch ?

- **Computed** : Calculer une valeur dérivée (prénom + nom = nom complet)
- **Watch** : Effectuer une action en réponse à un changement (sauvegarder, appel API...)

```
<script>
export default {
  data() {
    return {
      recherche: '',
      resultats: []
    }
  },
  watch: {
    // Watcher simple
    recherche(nouvelleValeur, ancienneValeur) {
      console.log(`Recherche: ${ancienneValeur} → ${nouvelleValeur}`);
      this.chercherProduits(nouvelleValeur);
    }
  },
  methods: {
    chercherProduits(terme) {
      // Appel API...
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Options avancées

```
<script>
export default {
  data() {
    return {
      utilisateur: {
        nom: 'Dupont',
        prenom: 'Jean'
      },
      compteur: 0
    }
  },
  watch: {
    // Watch immédiat (exécuté au montage)
    compteur: {
      handler(newVal) {
        console.log('Compteur:', newVal);
      },
      immediate: true // ← Exécuté dès le montage
    },

    // Watch profond (objet imbriqué)
    utilisateur: {
      handler(newVal) {
        console.log('Utilisateur modifié:', newVal);
        this.sauvegarder();
      },
      deep: true // ← Surveille les propriétés imbriquées
    }
  }
}
</script>
```

Revenir au sommaire

Watch vs Computed : Cas pratique

Exemple : Recherche avec debounce

```
<script>
export default {
  data() {
    return {
      searchQuery: '',
      searchResults: [],
      loading: false
    }
  },
  watch: {
    // ✅ WATCH : parfait pour les effets secondaires (API, etc.)
    searchQuery(newQuery) {
      this.loading = true;

      // Debounce pour éviter trop d'appels API
      clearTimeout(this.debounceTimer);
      this.debounceTimer = setTimeout(async () => {
        const response = await fetch(`/api/search?q=${newQuery}`);
        this.searchResults = await response.json();
        this.loading = false;
      }, 300);
    }
  },
  computed: {
    // ✅ COMPUTED : parfait pour les calculs
    hasResults() {
      return this.searchResults.length > 0;
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Cas d'usage : Validation de formulaire

Valider un champ en temps réel

```
<script>
export default {
  data() {
    return {
      email: '',
      emailError: '',
      password: '',
      passwordStrength: ''
    }
  },
  watch: {
    // Valider l'email à chaque changement
    email(newEmail) {
      if (!newEmail) {
        this.emailError = '';
        return;
      }

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailRegex.test(newEmail)) {
        this.emailError = 'Email invalide';
      } else {
        this.emailError = '';
      }
    },

    // Calculer la force du mot de passe
    password(newPassword) {
      if (newPassword.length < 6) {
        this.passwordStrength = 'Faible';
      } else if (newPassword.length < 10) {
        this.passwordStrength = 'Moyen';
      } else {
        this.passwordStrength = 'Fort';
      }
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Cas d'usage : Synchronisation localStorage

Sauvegarder automatiquement dans le localStorage

```
<script>
export default {
  data() {
    return {
      preferences: {
        theme: 'light',
        language: 'fr',
        notifications:'true
      }
    }
  },
  watch: {
    // Sauvegarder les préférences à chaque modification
    preferences: {
      handler(newPreferences) {
        localStorage.setItem('userPreferences', JSON.stringify(newPreferences));
        console.log('Préférences sauvegardées !');
      },
      deep: true, // Important pour surveiller les propriétés imbriquées
      immediate: false // Pas besoin de sauvegarder au montage
    }
  },
  mounted() {
    // Charger les préférences au démarrage
    const saved = localStorage.getItem('userPreferences');
    if (saved) {
      this.preferences = JSON.parse(saved);
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Cas d'usage : Calculateur en temps réel

Calculer automatiquement un résultat quand les inputs changent

```
<script>
export default {
  data() {
    return {
      longueur: 0,
      largeur: 0,
      surface: 0,
      perimetre: 0,
      prixM2: 1500,
      coutTotal: 0
    }
  },
  watch: {
    // Recalculer quand la longueur change
    longueur() {
      this.calculer();
    },
    // Recalculer quand la largeur change
    largeur() {
      this.calculer();
    },
    // Recalculer le coût si le prix au m² change
    prixM2() {
      this.calculer();
    }
  },
  methods: {
    calculer() {
      // Calculs de surface
      this.surface = this.longueur * this.largeur;
      this.perimetre = 2 * (this.longueur + this.largeur);
      this.coutTotal = this.surface * this.prixM2;

      // Log pour le débogage
      console.log(`Surface: ${this.surface}m² - Coût: ${this.coutTotal}€`);
    }
  }
}
</script>

<template>
  <div>
    <input v-model.number="longueur" placeholder="Longueur (m)" />
    <input v-model.number="largeur" placeholder="Largeur (m)" />
    <input v-model.number="prixM2" placeholder="Prix au m²" />

    <p>Surface: {{ surface }}m²</p>
    <p>Périmètre: {{ perimetre }}m</p>
    <p>Coût total: {{ coutTotal }}€</p>
  </div>
</template>
```

Revenir au sommaire

Watch - Cas d'usage : Comparaison ancienne/nouvelle valeur

Détecter des changements spécifiques

```
<script>
export default {
  data() {
    return {
      stockQuantity: 100,
      alertThreshold: 10
    }
  },
  watch: {
    stockQuantity(newQuantity, oldQuantity) {
      // Alerte quand le stock passe sous le seuil
      if (newQuantity <= this.alertThreshold && oldQuantity > this.alertThreshold) {
        this.sendLowStockAlert();
        console.log(`⚠️ ALERTE : Stock faible (${newQuantity} unités)`);
      }

      // Message quand le stock est reconstitué
      if (newQuantity > this.alertThreshold && oldQuantity <= this.alertThreshold) {
        console.log(`✅ Stock reconstitué (${newQuantity} unités)`);
      }

      // Alerte critique si stock à 0
      if (newQuantity === 0 && oldQuantity > 0) {
        console.log('🔴 STOCK EPUISÉ !');
      }
    }
  },
  methods: {
    sendLowStockAlert() {
      // Envoyer une notification, email, etc.
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Cas d'usage : Charger des données liées

Charger des données dépendantes automatiquement

```
<script>
export default {
  data() {
    return {
      selectedCountry: 'FR',
      cities: [],
      loading: false
    }
  },
  watch: {
    // Charger les villes quand le pays change
    async selectedCountry(newCountry) {
      if (!newCountry) {
        this.cities = [];
        return;
      }

      this.loading = true;
      try {
        const response = await fetch(`/api/cities?country=${newCountry}`);
        this.cities = await response.json();
      } catch (error) {
        console.error('Erreur chargement villes:', error);
        this.cities = [];
      } finally {
        this.loading = false;
      }
    }
  }
}
</script>
```

Revenir au sommaire

Watch - Récapitulatif des cas d'usage

Cas d'usage	Quand l'utiliser
Validation	Valider un formulaire en temps réel
localStorage	Sauvegarder automatiquement des données
Filtres	Filtrer une liste selon plusieurs critères
Alertes	Détecter des seuils ou conditions spécifiques
Chargement cascade	Charger des données liées (pays → villes)
Debounce	Limiter les appels API (recherche)
Analytics	Tracker les changements utilisateur

💡 Règle générale

Watch = Effets secondaires (API, localStorage, logs, notifications...)

Revenir au sommaire

Requêtes HTTP avec fetch (1/2)

Données :

```
<script>
export default {
  data() {
    return {
      users: [],
      loading: true,
      error: null
    }
  }
}
</script>
```

Revenir au sommaire

Requêtes HTTP avec fetch (2/2)

Méthode et lifecycle :

```
<script>
export default {
  methods: {
    async fetchUsers() {
      try {
        this.loading = true;
        const response = await fetch('https://api.example.com/users');

        if (!response.ok) {
          throw new Error('Erreur réseau');
        }

        this.users = await response.json();
      } catch (e) {
        this.error = e.message;
      } finally {
        this.loading = false;
      }
    }
  },
  mounted() {
    this.fetchUsers();
  }
}
</script>
```

Revenir au sommaire

Requêtes POST avec fetch (1/2)

Données du formulaire :

```
<script>
export default {
  data() {
    return {
      nom: '',
      email: '',
    }
  }
}
</script>
```

Revenir au sommaire

Requêtes POST avec fetch (2/2)

Méthode POST :

```
<script>
export default {
  methods: {
    async creerUtilisateur() {
      try {
        const response = await fetch('https://api.example.com/users', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
          },
          body: JSON.stringify({
            nom: this.nom,
            email: this.email
          })
        });

        const data = await response.json();
        console.log('Utilisateur crée:', data);
      } catch (error) {
        console.error('Erreur:', error);
      }
    }
  }
}
</script>
```

Revenir au sommaire

Gestion des données avec axios (1/2)

Alternative à fetch (plus de fonctionnalités)

```
npm install axios
```

Import et données :

```
<script>
import axios from 'axios';

export default {
  data() {
    return {
      users: []
    }
  }
}
</script>
```

Revenir au sommaire

Gestion des données avec axios (2/2)

Méthodes GET et POST :

```
<script>
export default {
  async mounted() {
    try {
      const { data } = await axios.get('/api/users');
      this.users = data;
    } catch (error) {
      console.error(error);
    }
  },
  methods: {
    async createUser(userData) {
      await axios.post('/api/users', userData);
    }
  }
}
</script>
```

Revenir au sommaire

Bilan de compétences

🎯 Ce que vous savez maintenant

- ✅ **JavaScript ES6+** : Arrow functions, destructuring, modules, async/await
- ✅ **Architecture Vue.js** : MVVM, organisation d'application
- ✅ **Outils** : Node.js, Vite, structure de projet
- ✅ **Concepts fondamentaux** : Data binding, interpolation, directives
- ✅ **Composants** : Création, props, événements, dynamiques
- ✅ **Réactivité** : Computed, lifecycle, composables, requêtes HTTP

Revenir au sommaire

1.8 Cycle de vie des composants

Revenir au sommaire

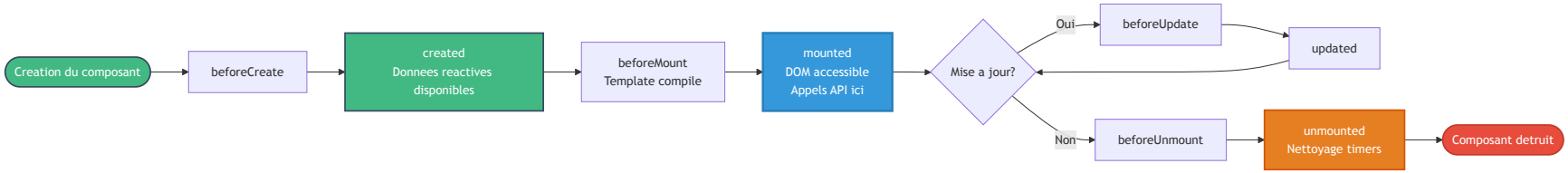
Hooks du cycle de vie

Les hooks permettent d'exécuter du code à des moments précis - **Options API**

```
<script>
export default {
  data() {
    return {
      message: 'Hello',
      users: []
    }
  },
  beforeCreate() {
    console.log('Avant la création');
    // On peut modifier les props par exemple, même si elles sont censé être immuables.
    console.log(this.message); // undefined - les données ne sont pas encore disponibles
  },
  created() {
    console.log('Composant créé');
    // On peut accéder aux données réactives, on peut les modifier.
    this.message = 'Bonjour Vue!';
    console.log(this.message); // 'Bonjour Vue!'
  },
  beforeMount() {
    console.log('Avant le montage');
    // On peut modifier le template, on peut ajouter des éléments au DOM.
    console.log(this.$el); // undefined - le DOM n'est pas encore créé
  },
  mounted() {
    console.log('Composant monté');
    // Appels API, initialisation
    fetch('https://api.example.com/users')
      .then(response => response.json())
      .then(data => this.users = data);
    console.log(this.$el); // Accès au DOM disponible
  }
}
</script>
```

Revenir au sommaire

Schéma : Cycle de vie des composants



Revenir au sommaire

Hooks du cycle de vie (suite) - 1/2

Hooks de mise à jour :

```
<script>
export default {
  data() {
    return {
      compteur: 0
    }
  },
  updated() {
    console.log('Composant mis à jour');
    console.log('Nouvelle valeur:', this.compteur);
    // par exemple pour des mesures (getBoundingClientRect, scroll, etc.)
    // mais en général on utilisera plutot computed ou watch pour ce genre de choses.
    // à peu utiliser donc
  }
}
</script>
```

Revenir au sommaire

Hooks du cycle de vie (suite) - 2/2

Hooks de démontage :

```
<script>
export default {
  beforeUnmount() {
    console.log('Avant destruction');
  },
  unmounted() {
    console.log('Composant démonté');
    // Nettoyage: timers, subscriptions, listeners
    // important de nettoyer les événements, les timers, les subscriptions, les listeners, etc si non => fuite de mémoire potentielle.
  }
}
</script>
```

Revenir au sommaire

Exemple pratique avec lifecycle (1/2)

Données et mounted :

```
<script>
export default {
  data() {
    return {
      data: null,
      loading: true,
      interval: null
    }
  },
  async mounted() {
    // Charger des données
    try {
      const response = await fetch('/api/data');
      this.data = await response.json();
    } finally {
      this.loading = false;
    }

    // Démarrer un interval
    this.interval = setInterval(() => {
      console.log('Tick');
    }, 1000);
  }
}
</script>
```

Revenir au sommaire

Exemple pratique avec lifecycle (2/2)

Nettoyage au démontage :

```
<script>
export default {
  beforeUnmount() {
    // Nettoyer l'interval
    if (this.interval) {
      clearInterval(this.interval);
    }
  }
}
</script>
```

Revenir au sommaire

🎯 Fil rouge : TaskFlow (notre Trello-like)

🎨 Ce qu'on va construire ensemble

Une application de gestion de tâches complète avec :

- 📋 **Boards** : Tableaux de tâches (Personnel, Travail, Projets)
- 📝 **Tasks** : Tâches avec titre, description, priorité
- 🏷️ **Colonnes** : À faire, En cours, Terminé
- 🎨 **Labels** : Urgent, Bug, Feature
- 👤 **Assignation** : Attribuer des tâches

Module par module

- **Module 1** : Composants de base (Task, TaskList, TaskForm)
- **Module 2** : Slots pour composants réutilisables (TaskCard avec header/content/actions)
- **Module 3** : Directive v-drag pour drag & drop
- **Module 4** : Pinia pour centraliser l'état (tasks, boards, users)
- **Module 5** : Router pour navigation (/boards, /board/:id, /settings)
- **Module 6** : Tests de l'application

Revenir au sommaire

Exercices Pratiques

Module 1 - Introduction à Vue.js et bases fondamentales

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 1

Maintenant que vous avez vu tous les concepts théoriques du Module 1, il est temps de les mettre en pratique en construisant les composants de base de notre application TaskFlow !

Les 5 exercices progressifs

Nous allons créer 5 exercices qui couvrent tous ces concepts dans le contexte de TaskFlow.

Revenir au sommaire

Exercice 0 : Setup du projet TaskFlow

Objectif

Mettre en place le projet TaskFlow avec Vite et créer la structure de base.

Étapes à suivre

1. **Créer le projet avec Vite**
2. **Explorer la structure du projet**
3. **Créer les dossiers nécessaires**
4. **Lancer le serveur de développement**

Concepts couverts

✅ **1.3 Outils et environnement** : Node.js, npm, Vite, structure de projet

Revenir au sommaire

Exercice 0 - Étape 1 : Créer le projet

Commandes

```
# Vérifier Node.js (v20+)
node --version

# Créer le projet TaskFlow avec Vite
npm create vite@latest taskflow-app -- --template vue

# Entrer dans le dossier
cd taskflow-app

# Installer les dépendances
npm install
```

Ce qui se passe

- Vite génère un projet Vue 3 avec la structure optimale
- Configuration automatique du build tool
- Dépendances installées (vue, @vitejs/plugin-vue)

Revenir au sommaire

Exercice 0 - Étape 2 : Structure du projet

Arborescence générée

```
taskflow-app/
├── node_modules/          # Dépendances (ne pas modifier)
├── public/
│   └── vite.svg           # Favicon
├── src/
│   ├── assets/            # Images, CSS
│   ├── components/        # Composants Vue (on va y travailler !)
│   ├── App.vue            # Composant racine
│   └── main.js            # Point d'entrée
├── index.html             # Page HTML principale
├── package.json           # Configuration npm
├── vite.config.js         # Configuration Vite
└── README.md
```

Revenir au sommaire

Exercice 0 - Étape 3 : Créer la structure TaskFlow

Créer les dossiers nécessaires

```
# Dans le dossier src/
mkdir src/components
```

Structure finale souhaitée

```
src/
├── components/
│   ├── TaskItem.vue      # (Exercice 1)
│   ├── TaskList.vue      # (Exercice 2)
│   └── TaskForm.vue      # (Exercice 3)
├── App.vue
└── main.js
```



Revenir au sommaire

Exercice 0 – Étape 4 : Lancer le projet

Commande de démarrage

```
npm run dev
```

Résultat attendu

```
VITE v5.x.x  ready in XXX ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
➜  press h + enter to show help
```

Vérification

- Ouvrir http://localhost:5173/ dans votre navigateur
- Vous devriez voir la page d'accueil Vite + Vue
- Le hot reload fonctionne : modifiez App.vue et voyez les changements instantanés !



Revenir au sommaire

Exercice 0 - Nettoyer App.vue

Objectif

Préparer App.vue pour recevoir nos composants TaskFlow.

⚠️ IMPORTANT : Options API vs Composition API

Attention ! Le template Vite génère par défaut du code avec **Composition API** :

```
<!-- Ce que Vite génère -->
<script setup>
import HelloWorld from './components/HelloWorld.vue'
</script>
```

Dans cette formation, nous utilisons l'Options API :

```
<!-- Ce que nous allons utiliser -->
<script>
export default {
  name: 'App',
  data() { return {} },
  methods: {},
  computed: {}
}
</script>
```



Revenir au sommaire

Pourquoi Options API ?

- ✅ Plus facile à apprendre pour les débutants
- ✅ Structure claire (data, methods, computed bien séparés)
- ✅ Meilleure compréhension des concepts Vue
- ✅ La Composition API sera vue dans le Module Bonus !

Revenir au sommaire

Remplacer le contenu de src/App.vue

```vue
<script>
export default {
  name: 'App'
}
</script>

<template>
  <div id="app">
    <header>
      <h1>🎯 TaskFlow</h1>
      <p>Votre gestionnaire de tâches moderne</p>
    </header>

    <main>
      <p>Prêt pour les exercices !</p>
    </main>
  </div>
</template>

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
  background: #f5f7fa;
  color: #2c3e50;
}

#app {
  width: 100%;
}

header {
  text-align: center;
  margin-bottom: 3rem;
}

header h1 {
  font-size: 2.5rem;
  color: #42b883;
  margin-bottom: 0.5rem;
}

header p {
  color: #7f8c8d;
  font-size: 1.1rem;
}

main {
  background: white;
  border-radius: 12px;
  padding: 2rem;
  box-shadow: 0 2px 12px rgba(0, 0, 0, 0.08);
}
</style>
```

Revenir au sommaire

Exercice 0 - Vérification finale

Checklist

- ✅ Projet créé avec Vite
- ✅ Dépendances installées
- ✅ Structure de dossiers créée
- ✅ Serveur de développement lancé (http://localhost:5173/)
- ✅ App.vue nettoyé et prêt
- ✅ Hot reload fonctionne



Revenir au sommaire

Exercice 1 : TaskItem.vue

Objectif

Créer le composant de base pour afficher une tâche

Spécifications

Props :

- `task` (Object) : `{ id, title, description, done, priority }`

Features :

- Afficher le titre de la tâche
- Afficher la description
- Checkbox pour marquer comme terminée
- Badge de priorité (low, medium, high, urgent)
- Barrer le texte si `done === true`



Revenir au sommaire

Événements :

- `@toggle` : Émettre quand on coche/décoche
- `@delete` : Émettre quand on clique sur supprimer
- `@edit` : Émettre quand on clique sur éditer

Revenir au sommaire

Exercice 1 - Solution TaskItem.vue (1/2)

Script :

```
<script>
export default {
  name: 'TaskItem',
  props: {
    task: {
      type: Object,
      required: true,
      validator: (task) => {
        return task.id && task.title;
      }
    }
  },
  emits: ['toggle', 'delete', 'edit'],
  computed: {
    priorityColor() {
      const colors = {
        low: '#10b981',
        medium: '#f59e0b',
        high: '#f97316',
        urgent: '#ef4444'
      };
      return colors[this.task.priority] || '#6b7280';
    },
    priorityLabel() {
      const labels = {
        low: 'Basse',
        medium: 'Moyenne',
        high: 'Haute',
        urgent: 'URGENT'
      };
      return labels[this.task.priority] || 'Normal';
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 1 - Solution TaskItem.vue (2/2)

Template :

```vue
<template>
  <div class="task-item" :class="{ done: task.done }">
    <div class="task-checkbox">
      <input
        type="checkbox"
        :checked="task.done"
        @change="$emit('toggle', task.id)"
      />
    </div>

    <div class="task-content">
      <h3 class="task-title">{{ task.title }}</h3>
      <p class="task-description">{{ task.description }}</p>

      <span
        class="task-priority"
        :style="{ backgroundColor: priorityColor }"
      >
        {{ priorityLabel }}
      </span>
    </div>

    <div class="task-actions">
      <button
        class="task-edit"
        @click="$emit('edit', task)"
      >
        ✏️
      </button>
      <button
        class="task-delete"
        @click="$emit('delete', task.id)"
      >
        🗑️
      </button>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 1 - Solution TaskItem Styles

```
<style scoped>
.task-item {
  display: flex;
  gap: 1rem;
  padding: 1rem;
  background: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  margin-bottom: 0.5rem;
  transition: all 0.2s;
}

.task-item:hover {
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.task-item.done .task-title,
.task-item.done .task-description {
  text-decoration: line-through;
  opacity: 0.6;
}

.task-content {
  flex: 1;
}

.task-title {
  margin: 0 0 0.5rem 0;
  font-size: 1.1rem;
  font-weight: 600;
}

.task-description {
  margin: 0 0 0.5rem 0;
  color: #6b7280;
}

.task-priority {
  display: inline-block;
  padding: 0.25rem 0.75rem;
  border-radius: 12px;
  color: white;
  font-size: 0.75rem;
  font-weight: 600;
}

.task-actions {
  display: flex;
  gap: 0.5rem;
  align-items: center;
}

.task-edit,
.task-delete {
  background: transparent;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
  opacity: 0.6;
  transition: opacity 0.2s;
}

.task-edit:hover,
.task-delete:hover {
```

Revenir au sommaire

Exercice 2 : TaskList.vue

Objectif

Créer un composant pour gérer une liste de tâches

Spécifications

Data :

- `tasks` (Array) : Liste des tâches
- `filter` : 'all' | 'active' | 'done'

Computed :

- `filteredTasks` : Tâches filtrées selon `filter`
- `activeCount` : Nombre de tâches non terminées
- `doneCount` : Nombre de tâches terminées

Revenir au sommaire

Methods :

- `toggleTask(id)` : Toggle le statut done
- `deleteTask(id)` : Supprimer une tâche
- `clearCompleted()` : Supprimer toutes les tâches terminées

Bonus :

- Afficher un message si aucune tâche
- Compteur de tâches

Revenir au sommaire

Exercice 2 - Solution TaskList.vue (1/3)

Script - Data :

```
<script>
import TaskItem from './TaskItem.vue';

export default {
  name: 'TaskList',
  components: {
    TaskItem
  },
  data() {
    return {
      tasks: [
        {
          id: 1,
          title: 'Apprendre Vue.js',
          description: 'Suivre la formation complète',
          done: false,
          priority: 'high'
        },
        {
          id: 2,
          title: 'Créer TaskFlow',
          description: 'Application de gestion de tâches',
          done: false,
          priority: 'urgent'
        },
        {
          id: 3,
          title: 'Setup projet',
          description: 'Initialiser avec Vite',
          done: true,
          priority: 'medium'
        }
      ],
      filter: 'all'
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 2 - Solution TaskList.vue (2/3)

Computed et Methods :

```
<script>
export default {
  // ... data précédent
  computed: {
    filteredTasks() {
      if (this.filter === 'active') {
        return this.tasks.filter(t => !t.done);
      }
      if (this.filter === 'done') {
        return this.tasks.filter(t => t.done);
      }
      return this.tasks;
    },
    activeCount() {
      return this.tasks.filter(t => !t.done).length;
    },
    doneCount() {
      return this.tasks.filter(t => t.done).length;
    }
  },
  methods: {
    toggleTask(id) {
      const task = this.tasks.find(t => t.id === id);
      if (task) {
        task.done = !task.done;
      }
    },
    deleteTask(id) {
      this.tasks = this.tasks.filter(t => t.id !== id);
    },
    clearCompleted() {
      this.tasks = this.tasks.filter(t => !t.done);
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 2 - Solution TaskList.vue (3/4)

Template - Header et Filtres :

```
<template>
  <div class="task-list">
    <div class="task-header">
      <h2>Mes Tâches</h2>
      <div class="task-stats">
        <span>{{ activeCount }} actives</span>
        <span>{{ doneCount }} terminées</span>
      </div>
    </div>

    <div class="task-filters">
      <button
        @click="filter = 'all'"
        :class="{ active: filter === 'all' }"
      >
        Toutes ({{ tasks.length }})
      </button>
      <button
        @click="filter = 'active'"
        :class="{ active: filter === 'active' }"
      >
        Actives ({{ activeCount }})
      </button>
      <button
        @click="filter = 'done'"
        :class="{ active: filter === 'done' }"
      >
        Terminées ({{ doneCount }})
      </button>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 - Solution TaskList.vue (4/4)

Template - Liste et Actions :

```vue
<template>
  <div class="task-list">
    <!-- ... header et filtres de la slide précédente ... -->

    <div class="tasks">
      <TaskItem
        v-for="task in filteredTasks"
        :key="task.id"
        :task="task"
        @toggle="toggleTask"
        @delete="deleteTask"
      />

      <div v-if="filteredTasks.length === 0" class="empty">
        Aucune tâche
      </div>
    </div>

    <button
      v-if="doneCount > 0"
      class="clear-completed"
      @click="clearCompleted"
    >
      Supprimer les tâches terminées ({{ doneCount }})
    </button>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 : TaskForm.vue

Objectif

Créer un formulaire pour ajouter/éditer des tâches

Spécifications

Data :

- `formData : { title, description, priority }`

Props (optionnel) :

- `editTask` : Tâche à éditer (si mode édition)

Validation :

- Titre requis (min 3 caractères)
- Description optionnelle
- Priorité par défaut : 'medium'



Revenir au sommaire

Événements :

- `@submit` : Émettre les données du formulaire
- `@cancel` : Annuler l'édition

Revenir au sommaire

Exercice 3 - Solution TaskForm.vue

```vue
<script>
export default {
  name: 'TaskForm',
  props: {
    editTask: {
      type: Object,
      default: null
    }
  },
  emits: ['submit', 'cancel'],
  data() {
    return {
      formData: {
        title: '',
        description: '',
        priority: 'medium'
      }
    }
  },
  computed: {
    isValid() {
      return this.formData.title.trim().length >= 3;
    }
  },
  watch: {
    editTask: {
      handler(newTask) {
        if (newTask) {
          this.formData = { ...newTask };
        } else {
          this.resetForm();
        }
      },
      immediate: true
    }
  },
  methods: {
    handleSubmit() {
      if (this.isValid) {
        this.$emit('submit', { ...this.formData });
        this.resetForm();
      }
    },
    resetForm() {
      this.formData = {
        title: '',
        description: '',
        priority: 'medium'
      };
    }
  }
}
</script>
```

Revenir au sommaire

💡 Pourquoi le watch ?

Le problème sans watch

Sans le `watch`, si vous éditez une tâche puis annulez, **les inputs gardent les anciennes valeurs** quand vous créez une nouvelle tâche.

La solution avec watch

Le `watch` avec `immediate: true` :

- ✅ Charge les données au montage (si `editTask` existe)
- ✅ Réagit quand `editTask` change
- ✅ **Réinitialise quand** `editTask` **devient** `null` ← FIX du bug !

C'est une bonne pratique pour les formulaires d'édition !

Revenir au sommaire

Exercice 3 - Solution TaskForm Template

```vue
<template>
  <form @submit.prevent="handleSubmit" class="task-form">
    <div class="form-group">
      <label>Titre *</label>
      <input
        v-model="formData.title"
        type="text"
        placeholder="Ma nouvelle tâche..."
        required
      />
    </div>

    <div class="form-group">
      <label>Description</label>
      <textarea
        v-model="formData.description"
        placeholder="Détails de la tâche..."
        rows="3"
      ></textarea>
    </div>

    <div class="form-group">
      <label>Priorité</label>
      <select v-model="formData.priority">
        <option value="low">Basse</option>
        <option value="medium">Moyenne</option>
        <option value="high">Haute</option>
        <option value="urgent">URGENT</option>
      </select>
    </div>

    <div class="form-actions">
      <button type="submit" :disabled="!isValid">
        {{ editTask ? 'Modifier' : 'Ajouter' }}
      </button>
      <button type="button" @click="$emit('cancel')">
        Annuler
      </button>
    </div>
  </form>
</template>
```

Revenir au sommaire

Exercice 4 : Recherche avec Watch

Objectif

Ajouter une recherche en temps réel avec debounce

Spécifications

Data :

- `searchQuery` : Terme de recherche

Watch :

- Surveiller `searchQuery`
- Appliquer un debounce de 300ms
- Filtrer les tâches par titre/description

Computed :

- `searchResults` : Tâches filtrées par recherche

Revenir au sommaire

Exercice 4 - Solution Recherche (1/2)

```
<script>
export default {
  data() {
    return {
      tasks: [ /* ... */ ],
      searchQuery: '',
      searchResults: [],
      debounceTimer: null
    }
  },
  watch: {
    // ✅ WATCH avec debounce
    searchQuery(newQuery) {
      clearTimeout(this.debounceTimer);

      if (!newQuery.trim()) {
        this.searchResults = [];
        return;
      }

      this.debounceTimer = setTimeout(() => {
        const query = newQuery.toLowerCase();
        this.searchResults = this.tasks.filter(task => {
          return task.title.toLowerCase().includes(query) ||
                 task.description.toLowerCase().includes(query);
        });
      }, 300);
    }
  },
  computed: {
    displayedTasks() {
      return this.searchQuery.trim()
        ? this.searchResults
        : this.filteredTasks;
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 4 - Solution Recherche (2/2)

```vue
<template>
  <div class="task-list">
    <!-- Header -->
    <div class="search-bar">
      <input
        v-model="searchQuery"
        type="search"
        placeholder="🔍 Rechercher une tâche..."
        class="search-input"
      />
      <span v-if="searchQuery" class="search-info">
        {{ searchResults.length }} résultat(s)
      </span>
    </div>

    <!-- Filtres -->
    <div class="task-filters">
      <!-- ... boutons filtres ... -->
    </div>

    <!-- Task List -->
    <div class="tasks">
      <TaskItem
        v-for="task in displayedTasks"
        :key="task.id"
        :task="task"
        @toggle="toggleTask"
        @delete="deleteTask"
      />

      <div v-if="displayedTasks.length === 0" class="empty">
        {{ searchQuery
          ? `Aucune tâche trouvée pour "${searchQuery}"`
          : 'Aucune tâche'
        }}
      </div>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 5 : Intégration dans App.vue

Objectif

Assembler tous les composants créés dans les exercices précédents pour créer l'application TaskFlow complète et fonctionnelle.

Spécifications

Data dans App.vue :

- `tasks` : Array de tâches (avec id, title, description, priority, done)
- `showForm` : Boolean pour afficher/masquer le formulaire
- `editingTask` : Tâche en cours d'édition (ou null)

Methods :

- `addTask(taskData)` : Ajouter une nouvelle tâche
- `toggleTask(id)` : Basculer le statut done d'une tâche
- `deleteTask(id)` : Supprimer une tâche
- `editTask(task)` : Préparer l'édition d'une tâche
- `updateTask(taskData)` : Mettre à jour une tâche existante

Revenir au sommaire

Exercice 5 - Solution App.vue (1/3)

Script - Data et imports :

```
<script>
import TaskForm from './components/TaskForm.vue';
import TaskList from './components/TaskList.vue';

export default {
  name: 'App',
  components: {
    TaskForm,
    TaskList
  },
  data() {
    return {
      tasks: [
        {
          id: 1,
          title: 'Apprendre Vue.js 3',
          description: 'Maîtriser les bases fondamentales',
          priority: 'high',
          done: false
        },
        {
          id: 2,
          title: 'Créer TaskFlow',
          description: 'Application complète de gestion de tâches',
          priority: 'urgent',
          done: false
        },
        {
          id: 3,
          title: 'Setup du projet',
          description: 'Installer Vite et créer la structure',
          priority: 'medium',
          done: true
        }
      ],
      showForm: false,
      editingTask: null
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 5 - Solution App.vue (2/3)

Methods :

```
<script>
export default {
  // ... data précédent
  methods: {
    addTask(taskData) {
      this.tasks.push({
        id: Date.now(),
        ...taskData,
        done: false
      });
      this.showForm = false;
    },

    toggleTask(id) {
      const task = this.tasks.find(t => t.id === id);
      if (task) {
        task.done = !task.done;
      }
    },

    deleteTask(id) {
      if (confirm('Supprimer cette tâche ?')) {
        this.tasks = this.tasks.filter(t => t.id !== id);
      }
    },

    editTask(task) {
      this.editingTask = { ...task };
      this.showForm = true;
    },

    updateTask(taskData) {
      const index = this.tasks.findIndex(t => t.id === this.editingTask.id);
      if (index !== -1) {
        this.tasks[index] = {
          ...this.editingTask,
          ...taskData
        };
      }
      this.editingTask = null;
      this.showForm = false;
    },

    cancelForm() {
      this.showForm = false;
      this.editingTask = null;
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 5 - Solution App.vue (3/3)

Template :

```
<template>
  <div id="app">
    <header>
      <h1>🎯 TaskFlow</h1>
      <p>Votre gestionnaire de tâches moderne</p>
    </header>

    <main>
      <div class="actions-bar">
        <button
          @click="showForm = !showForm"
          class="btn-primary"
        >
          {{ showForm ? '✖ Annuler' : '➕ Nouvelle tâche' }}
        </button>
      </div>

      <TaskForm
        v-if="showForm"
        :editTask="editingTask"
        @submit="editingTask ? updateTask($event) : addTask($event)"
        @cancel="cancelForm"
      />

      <TaskList
        :tasks="tasks"
        @toggle="toggleTask"
        @delete="deleteTask"
        @edit="editTask"
      />
    </main>
  </div>
</template>
```

Revenir au sommaire

Exercice 5 - Styles App.vue

```vue
<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
  padding: 2rem;
}

#app {
  max-width: 900px;
  margin: 0 auto;
}

header {
  text-align: center;
  margin-bottom: 2rem;
  color: white;
}

header h1 {
  font-size: 3rem;
  margin-bottom: 0.5rem;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);
}

header p {
  font-size: 1.2rem;
  opacity: 0.9;
}

main {
  background: white;
  border-radius: 16px;
  padding: 2rem;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
}

.actions-bar {
  margin-bottom: 2rem;
}

.btn-primary {
  background: #42b883;
  color: white;
  border: none;
  padding: 0.75rem 1.5rem;
  border-radius: 8px;
  font-size: 1rem;
  cursor: pointer;
  transition: all 0.2s;
}

.btn-primary:hover {
  background: #35a372;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(66, 184, 131, 0.4);
}
</style>
```

Revenir au sommaire

Exercice 5 - Améliorations optionnelles

Fonctionnalités bonus à ajouter

1. Persistance localStorage :

```
mounted() {
  const saved = localStorage.getItem('taskflow-tasks');
  if (saved) {
    this.tasks = JSON.parse(saved);
  }
},
watch: {
  tasks: {
    handler(newTasks) {
      localStorage.setItem('taskflow-tasks', JSON.stringify(newTasks));
    },
    deep: true
  }
}
```

2. Bouton "Tout effacer" :

```
clearAll() {
  if (confirm('Supprimer toutes les tâches ?')) {
    this.tasks = [];
  }
}
```

3. Export/Import JSON :

- Bouton pour télécharger les tâches en JSON
- Bouton pour importer un fichier JSON

Revenir au sommaire

Exercice 5 - Test de l'application

Checklist de validation

Testez toutes les fonctionnalités :

- ✅ Afficher la liste des tâches initiales
- ✅ Ouvrir le formulaire avec le bouton "Nouvelle tâche"
- ✅ Ajouter une nouvelle tâche
- ✅ Cocher/décocher une tâche (toggle done)
- ✅ Supprimer une tâche
- ✅ Filtrer les tâches (toutes/actives/terminées)
- ✅ Rechercher une tâche
- ✅ Voir les statistiques (compteurs)
- ✅ Éditer une tâche existante
- ✅ Annuler l'ajout/édition

Revenir au sommaire

🎯 Récap Module 1 - TaskFlow

Ce que vous avez construit

✅ **TaskItem.vue** : Composant de tâche avec props/events

✅ **TaskList.vue** : Gestion de liste avec filtres et recherche

✅ **TaskForm.vue** : Formulaire avec v-model et validation

✅ **App.vue** : Intégration complète de tous les composants

Concepts du Module 1 utilisés

- ✅ **1.1 JavaScript ES6+** : Arrow functions, destructuring, spread operator
- ✅ **1.2 Architecture** : MVVM, organisation composants
- ✅ **1.3 Outils** : Vite, npm, structure projet
- ✅ **1.4 Composants** : .vue, data(), methods, structure
- ✅ **1.5 Directives** : v-bind, v-on, v-if, v-for, v-model
- ✅ **1.6 Communication** : Props (validation), Events ($emit)
- ✅ **1.7 Réactivité** : Computed properties, Watch (debounce)
- ✅ **1.8 Lifecycle** : mounted pour initialisation

Revenir au sommaire

Checklist de validation finale

Avant de passer au Module 2, vérifiez que vous maîtrisez :

Fondamentaux :

- ✅ Installation et configuration environnement (Node.js, npm, Vite)
- ✅ Syntaxe JavaScript ES6+ (arrow functions, destructuring, spread, modules)
- ✅ Création projet Vue avec Vite
- ✅ Structure .vue (script, template, style)

Composants :

- ✅ Création de composants réutilisables
- ✅ data(), methods, computed
- ✅ Props avec validation (type, required, default, validator)
- ✅ Events personnalisés ($emit, emits declaration)

Directives :

- ✅ Interpolation
- ✅ v-bind (: shorthand) - classes, styles, props
- ✅ v-on (@ shorthand) - events, modificateurs
- ✅ v-if / v-else / v-show
- ✅ v-for avec :key
- ✅ v-model (input, textarea, select, checkbox)

Revenir au sommaire

Réactivité :

- ✅ Computed properties et mise en cache
- ✅ Watch avec options (debounce pattern)
- ✅ Différence computed vs methods

Lifecycle :

- ✅ mounted() pour initialisation
- ✅ Comprendre le cycle complet
- ✅ Cleanup (concept, même si pas utilisé)

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Vue.js 3 Documentation - Doc complète et tutoriels
- 🔧 Vite Guide - Build tool rapide
- 📦 npm Documentation - Gestionnaire de paquets

Outils de développement

- 🛠️ Vue DevTools - Extension navigateur indispensable
- 💻 VS Code + Volar - IDE support
- 🎨 Tailwind CSS - Framework CSS utility-first

Communauté

- 💬 Discord Vue Land - Communauté active

Revenir au sommaire

Vous avez terminé le Module 1 !

Vous maîtrisez maintenant :

- ✅ Les bases fondamentales de Vue.js 3
- ✅ La création de composants réutilisables
- ✅ La communication parent-enfant
- ✅ La réactivité avec computed et watch
- ✅ Le cycle de vie des composants

🚀 Prochaine étape : Module 2 - Slots

Rendez-vous au Module 2 pour découvrir comment créer des composants encore plus flexibles et réutilisables avec les **slots** !



Revenir au sommaire

Module 2

Slots et composants génériques

Revenir au sommaire

SOMMAIRE DU MODULE

🎰 2.1 Introduction aux slots

🛠️ 2.2 Implémentation des slots

💪 Exercices Pratiques du Module 2

Revenir au sommaire

2.1 Introduction aux slots

Revenir au sommaire

Qu'est-ce qu'un slot ?

Les **slots** permettent de passer du contenu HTML (ou des composants) à un composant enfant

🎰 Analogie : Les slots sont comme une machine à sous

Imaginez un **cadre photo** :

- Le cadre est votre composant (toujours le même design)
- Le **slot** est l'emplacement vide au centre
- Vous pouvez y mettre **n'importe quelle photo** !



Revenir au sommaire

Pourquoi utiliser des slots ?

- 🎯 **Composants flexibles** : un composant Card pour 1000 usages différents
- 🧩 **Composer des interfaces** : assembler des briques comme des Lego
- 🎨 **Personnaliser sans casser** : chaque utilisation est unique
- ♻️ **DRY** : un seul composant Modal, 50 contenus différents

Revenir au sommaire

Problème sans les slots

```vue
<!-- AlertSuccess.vue -->
<template>
  <div class="alert success">
    <span class="icon">✓</span>
    <span>Opération réussie</span>
  </div>
</template>

<!-- AlertError.vue -->
<template>
  <div class="alert error">
    <span class="icon">✗</span>
    <span>Une erreur est survenue</span>
  </div>
</template>
```

Revenir au sommaire

Pourquoi c’est problématique ?

- ❌ Duplication de code
- ❌ Un composant par type de message
- ❌ Difficile à maintenir

Revenir au sommaire

Solution avec les slots

```vue
<!-- Alert.vue - Composant générique -->
<template>
  <div class="alert">
    <span class="icon">
      <slot name="icon">ℹ️</slot>
    </span>
    <div class="content">
      <slot></slot>
    </div>
  </div>
</template>
```



Revenir au sommaire

Utilisation du composant avec slots

```
<template>
  <Alert>
    <template #icon>✓</template>
    Opération réussie !
  </Alert>

  <Alert>
    <template #icon>✗</template>
    Une erreur est survenue
  </Alert>

  <Alert>
    <template #icon>⚠️</template>
    <strong>Attention :</strong> action irréversible
  </Alert>
</template>
```

Revenir au sommaire

Avantages des slots

- ✅ Un seul composant
- ✅ Contenu flexible
- ✅ Facile à maintenir

Revenir au sommaire

Composants génériques

Un **composant générique** est un composant réutilisable qui s'adapte à différents contextes

Exemples de composants génériques

- 📦 **Card** : conteneur avec header, body, footer
- 🔘 **Button** : bouton personnalisable
- 🗒️ **Modal** : fenêtre modale
- 🖼️ **Layout** : mise en page avec sidebar, content, header

Revenir au sommaire

Relations parent/enfant avec slots

Les slots nommés permettent de définir plusieurs zones d'injection de contenu.

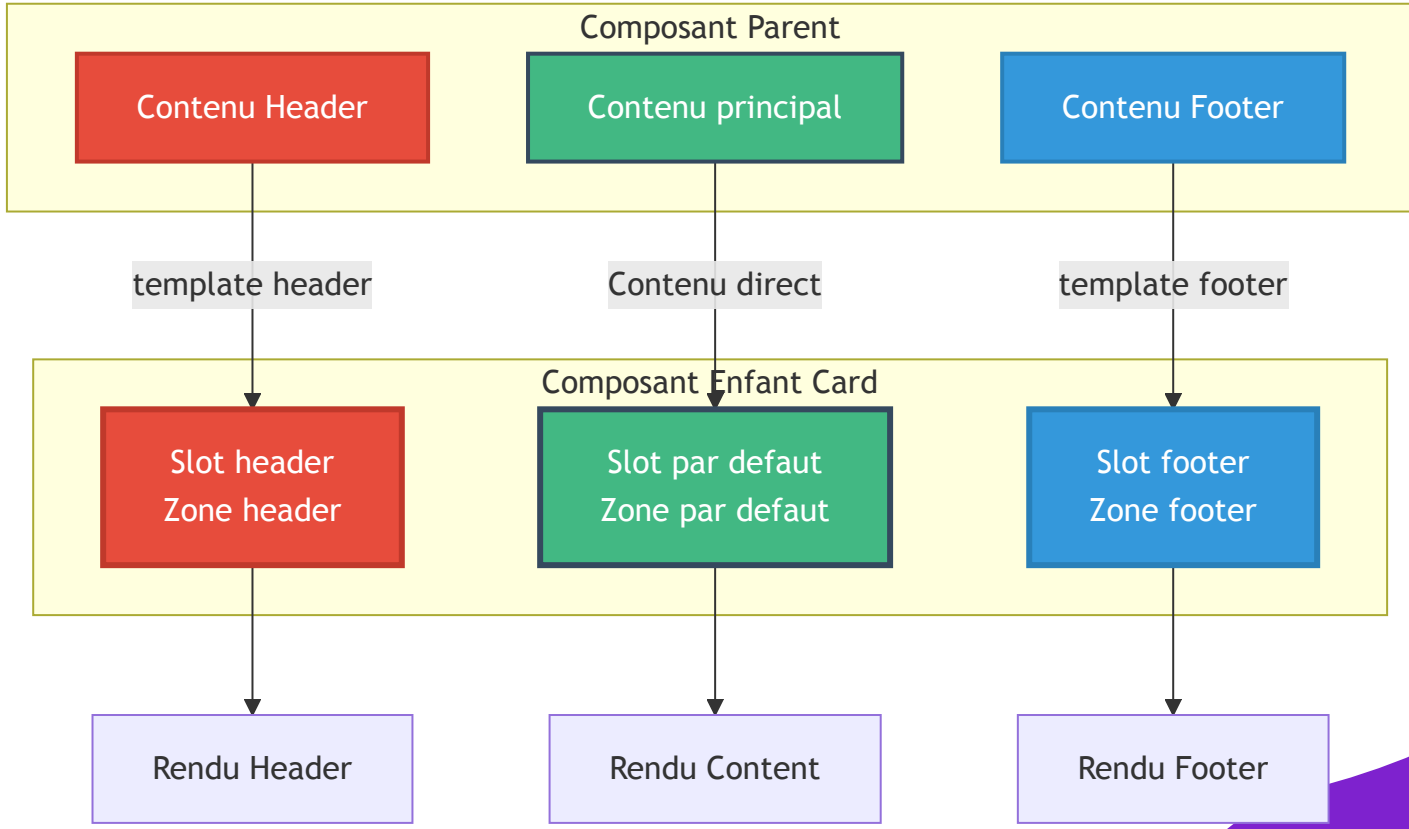
3 types de slots :

- **Slot nommé** : `<slot name="header">`
- **Slot par défaut** : `<slot>`
- **Slot nommé** : `<slot name="footer">`



Revenir au sommaire

Schéma : Slots nommés



Revenir au sommaire

2.2 Implémentation

Revenir au sommaire

Slots simples (slot par défaut)

Le slot le plus basique - un seul emplacement pour le contenu

```
<!-- Button.vue -->
<template>
  <button class="btn">
    <slot></slot>
  </button>
</template>

<!-- Utilisation -->
<Button>Cliquer ici</Button>
<Button>Enregistrer</Button>
<Button>
  <span class="icon">✓</span>
  Valider
</Button>
```

Revenir au sommaire

Slot par défaut avec contenu fallback

On peut définir un contenu par défaut si rien n'est fourni

```vue
<!-- Button.vue -->
<template>
  <button class="btn">
    <slot>Bouton par défaut</slot>
  </button>
</template>

<!-- Utilisation -->
<Button>Mon texte</Button>
<!-- Rendu: Mon texte -->

<Button></Button>
<!-- Rendu: Bouton par défaut -->
```



Revenir au sommaire

Slots nommés

Quand on a besoin de plusieurs emplacements distincts

```vue
<!-- Card.vue -->
<template>
  <div class="card">
    <header class="card-header">
      <slot name="header"></slot>
    </header>

    <div class="card-body">
      <slot></slot>
    </div>

    <footer class="card-footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>
```

Revenir au sommaire

Utilisation des slots nommés

```vue
<script>
import Card from './components/Card.vue';

export default {
  components: {
    Card
  }
}
</script>

<template>
  <Card>
    <template #header>
      <h2>Titre de la carte</h2>
    </template>

    <p>Contenu principal de la carte</p>
    <p>Encore du contenu...</p>

    <template #footer>
      <button>Action</button>
    </template>
  </Card>
</template>
```

Revenir au sommaire

Directive v-slot

Syntaxes disponibles pour les slots

```vue
<!-- Syntaxe complète -->
<template v-slot:header>
  <h1>Titre</h1>
</template>

<!-- Syntaxe abrégée (recommandée) -->
<template #header>
  <h1>Titre</h1>
</template>

<!-- Slot par défaut -->
<template #default>
  <p>Contenu</p>
</template>

<!-- Ou directement -->
<p>Contenu</p>
```



Revenir au sommaire

Scoped Slots - Introduction

Les **scoped slots** permettent au composant enfant de passer des données au parent

🎁 Analogie : C'est comme une boîte cadeau avec mode d'emploi

Le composant enfant dit au parent :

> "Tiens, voici les données (le cadeau 🎁), maintenant fais-en ce que tu veux !"

🤔 Problème à résoudre

Sans scoped slots :

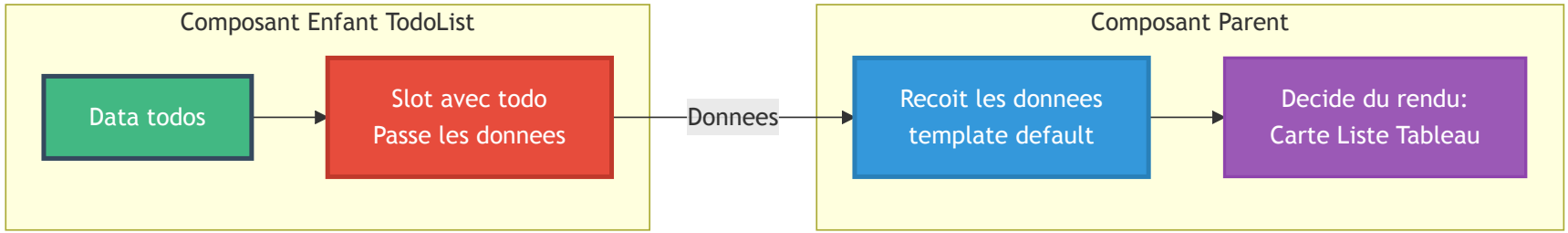
- L'enfant décide TOUT (comment afficher)
- Pas flexible

Avec scoped slots :

- L'enfant donne les données
- Le parent décide comment les afficher
- **Super flexible** ! 🎨

Revenir au sommaire

Schéma : Scoped Slots



Revenir au sommaire

Scoped Slots - Exemple concret

Cas d'usage typique

Le composant enfant fait le **travail de récupération/logique** (ex: boucle, tri) Le parent décide du **rendu visuel** (ex: carte, tableau, liste...)

```vue
<!-- TodoList.vue - L'enfant donne les données -->
<template>
  <ul>
    <li v-for="(todo, index) in todos" :key="todo.id">
      <slot :todo="todo" :index="index">
        <!-- Contenu par défaut si pas de slot fourni -->
        {{ todo.text }}
      </slot>
    </li>
  </ul>
</template>
```

Revenir au sommaire

Scoped Slots - Utilisation

```vue
<script>
import TodoList from './components/TodoList.vue';

export default {
  components: {
    TodoList
  },
  data() {
    return {
      todos: [
        { id: 1, text: 'Apprendre Vue', done: true },
        { id: 2, text: 'Créer un projet', done: false }
      ]
    }
  }
}
</script>

<template>
  <TodoList :todos="todos">
    <template #default="{ todo, index }">
      <span :class="{ done: todo.done }">
        {{ index + 1 }}. {{ todo.text }}
      </span>
      <span v-if="todo.done"> ✓</span>
    </template>
  </TodoList>
</template>
```

Revenir au sommaire

Scoped Slots - Cas avancé (1/2)

Script du composant DataTable :

```
<!-- DataTable.vue -->
<script>
export default {
  props: {
    items: {
      type: Array,
      required: true
    },
    columns: {
      type: Array,
      required: true
    }
  }
}
</script>
```

Revenir au sommaire

Scoped Slots - Cas avancé (2/2)

Template avec scoped slots :

```
<template>
  <table>
    <thead>
      <tr>
        <th v-for="col in columns" :key="col.key">
          {{ col.label }}
        </th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="item in items" :key="item.id">
        <td v-for="col in columns" :key="col.key">
          <slot
            :name="col.key"
            :item="item"
            :value="item[col.key]"
          >
            {{ item[col.key] }}
          </slot>
        </td>
      </tr>
    </tbody>
  </table>
</template>
```

Revenir au sommaire

Utilisation DataTable avec scoped slots

```vue
<template>
  <DataTable :items="users" :columns="columns">
    <template #nom="{ item }">
      <strong>{{ item.nom }}</strong>
    </template>

    <template #email="{ value }">
      <a :href="`mailto:${value}`">{{ value }}</a>
    </template>

    <template #actions="{ item }">
      <button @click="edit(item)">Éditer</button>
      <button @click="delete(item)">Supprimer</button>
    </template>
  </DataTable>
</template>
```

Revenir au sommaire

Slots dynamiques

Les slots peuvent être déterminés dynamiquement

```
<script>
import Card from './components/Card.vue';

export default {
  components: {
    Card
  },
  data() {
    return {
      slotActif: 'header'
    }
  }
}
</script>

<template>
  <Card>
    <template #[slotActif]>
      Ce contenu ira dans le slot actif
    </template>
  </Card>

  <button @click="slotActif = 'header'">Header</button>
  <button @click="slotActif = 'footer'">Footer</button>
</template>
```



Revenir au sommaire

Teleport : Rendu ailleurs 🚀

Le composant **Teleport** permet de rendre un composant dans une autre partie du DOM

🎯 Problème résolu

Sans Teleport :

- ❌ Les modales héritent du contexte CSS parent (z-index, overflow...)
- ❌ Difficile de gérer les overlays plein écran
- ❌ Problèmes de positionnement

Avec Teleport :

- ✅ Rendu directement dans `<body>` ou n'importe quel sélecteur
- ✅ Plus de problèmes de z-index
- ✅ Overlays plein écran faciles

Revenir au sommaire

Teleport : Syntaxe

```
<template>
  <button @click="showModal = true">Ouvrir Modal</button>

  <Teleport to="body">
    <div v-if="showModal" class="modal-overlay">
      <div class="modal-content">
        <h2>Je suis rendu dans body !</h2>
        <button @click="showModal = false">Fermer</button>
      </div>
    </div>
  </Teleport>
</template>
```

Revenir au sommaire

Teleport : Résultat dans le DOM

Code Vue :

```
<div id="app">
  <button>Ouvrir Modal</button>
  <Teleport to="body">
    <div class="modal">...</div>
  </Teleport>
</div>
```

Rendu HTML :

```
<body>
  <div id="app">
    <button>Ouvrir Modal</button>
    <!-- Teleport rend ICI mais le contenu va ailleurs ! -->
  </div>

  <!-- Le contenu Teleport apparaît ici ! -->
  <div class="modal">...</div>
</body>
```

Revenir au sommaire

Teleport : Cas d'usage

Les cas d'usage courants pour Teleport :

- 🎭 **Modales** : Évite les problèmes de z-index et overflow
- 📋 **Tooltips** : Positionnement absolu libre
- 🔔 **Notifications** : Toast en haut à droite de l'écran
- 🖼️ **Overlays** : Plein écran sans contraintes CSS du parent
- 🎨 **Dropdowns** : Éviter les problèmes de débordement

💡 Astuce

Utilisez `to="body"` pour la plupart des cas, ou un sélecteur CSS spécifique !

Revenir au sommaire

Teleport : Rendu ailleurs 🚀

Le composant **Teleport** permet de rendre un composant dans une autre partie du DOM

🎯 Pourquoi Teleport ?

Problème : Une modale créée dans un composant enfant hérite du contexte CSS parent (z-index, overflow)

Solution : Teleport permet de rendre la modale directement dans `<body>` !

Revenir au sommaire

Teleport : Syntaxe

```vue
<template>
  <button @click="showModal = true">Ouvrir</button>

  <Teleport to="body">
    <div v-if="showModal" class="modal-overlay">
      <div class="modal-content">
        <h2>Je suis rendu dans body !</h2>
        <button @click="showModal = false">Fermer</button>
      </div>
    </div>
  </Teleport>
</template>
```

Résultat dans le DOM

Le contenu du `<Teleport>` est téléporté directement dans `<body>`, pas dans le composant parent !



Revenir au sommaire

Teleport : Cas d'usage

- 🎭 **Modales** : Éviter les problèmes de z-index et overflow
- 📋 **Tooltips** : Positionnement absolu libre
- 🔔 **Notifications** : Toast/snackbar en haut à droite
- 🖼️ **Overlays** : Plein écran sans contraintes CSS du parent

💡 Conseil

Utilisez `to="body"` pour la plupart des cas !

Revenir au sommaire

Combinaison Props & Slots

Les slots et props travaillent ensemble pour créer des composants puissants

```vue
<!-- Modal.vue -->
<script>
export default {
  props: {
    show: Boolean,
    title: String,
    size: {
      type: String,
      default: 'medium'
    }
  },
  emits: ['close'],
  methods: {
    handleClose() {
      this.$emit('close');
    }
  }
}
</script>
```

Revenir au sommaire

Modal - Template

```vue
<template>
  <Teleport to="body">
    <div v-if="show" class="modal-overlay" @click="handleClose">
      <div
        class="modal-content"
        :class="`modal-${size}`"
        @click.stop
      >
        <header class="modal-header">
          <slot name="header">
            <h2>{{ title }}</h2>
          </slot>
          <button @click="handleClose">×</button>
        </header>

        <div class="modal-body">
          <slot></slot>
        </div>

        <footer class="modal-footer">
          <slot name="footer">
            <button @click="handleClose">Fermer</button>
          </slot>
        </footer>
      </div>
    </div>
  </Teleport>
</template>
```



Revenir au sommaire

Utilisation du Modal

```vue
<script>
import Modal from './components/Modal.vue';

export default {
  components: {
    Modal
  },
  data() {
    return {
      showModal: false
    }
  },
  methods: {
    confirmer() {
      // Logique de confirmation
      this.showModal = false;
    }
  }
}
</script>

<template>
  <button @click="showModal = true">Ouvrir Modal</button>

  <Modal
    :show="showModal"
    title="Confirmation"
    size="small"
    @close="showModal = false"
  >
    <p>Êtes-vous sûr de vouloir continuer ?</p>

    <template #footer>
      <button @click="showModal = false">Annuler</button>
      <button @click="confirmer">Confirmer</button>
    </template>
  </Modal>
</template>
```

Revenir au sommaire

Exemple : Composant Layout

```vue
<!-- AppLayout.vue -->
<template>
  <div class="app-layout">
    <aside class="sidebar">
      <slot name="sidebar"></slot>
    </aside>

    <div class="main-content">
      <header class="header">
        <slot name="header"></slot>
      </header>

      <main class="content">
        <slot></slot>
      </main>

      <footer class="footer">
        <slot name="footer"></slot>
      </footer>
    </div>
  </div>
</template>
```

Revenir au sommaire

Utilisation du Layout

```
<template>
  <AppLayout>
    <template #sidebar>
      <nav>
        <a href="/">Accueil</a>
        <a href="/produits">Produits</a>
        <a href="/contact">Contact</a>
      </nav>
    </template>

    <template #header>
      <h1>Mon Application</h1>
      <UserMenu />
    </template>

    <!-- Contenu principal -->
    <div>
      <h2>Bienvenue</h2>
      <p>Contenu de la page...</p>
    </div>

    <template #footer>
      <p>© 2025 Mon Application</p>
    </template>
  </AppLayout>
</template>
```

Revenir au sommaire

Bonnes pratiques - Slots

✅ Faire

- Utiliser des noms de slots descriptifs
- Fournir du contenu fallback pertinent
- Documenter les props des scoped slots
- Utiliser TypeScript pour typer les slots

❌ Éviter

- Trop de slots dans un seul composant (> 5)
- Noms de slots génériques (slot1, slot2...)
- Slots obligatoires sans fallback
- Logique complexe dans les slots



Revenir au sommaire

Pattern de composition avec slots

```vue
<!-- BaseButton.vue -->
<template>
  <button class="btn" :class="variant">
    <span v-if="$slots.icon" class="icon">
      <slot name="icon"></slot>
    </span>
    <slot></slot>
    <span v-if="loading" class="spinner"></span>
  </button>
</template>

<!-- Utilisation -->
<BaseButton variant="primary">
  <template #icon>
    <IconSave />
  </template>
  Enregistrer
</BaseButton>
```

Revenir au sommaire

Détection de slots

Vérifier si un slot est fourni

```
<script>
export default {
  computed: {
    hasHeader() {
      return !!this.$slots.header;
    },
    hasFooter() {
      return !!this.$slots.footer;
    }
  }
}
</script>

<template>
  <div class="card">
    <header v-if="hasHeader" class="header">
      <slot name="header"></slot>
    </header>

    <div class="body">
      <slot></slot>
    </div>

    <footer v-if="hasFooter" class="footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>
```

Revenir au sommaire

Slots multiples avec v-for

```vue
<!-- Accordion.vue -->
<script>
export default {
  props: {
    items: {
      type: Array,
      required: true
    }
  }
}
</script>

<template>
  <div class="accordion">
    <div
      v-for="(item, index) in items"
      :key="item.id"
      class="accordion-item"
    >
      <slot
        name="item"
        :item="item"
        :index="index"
      >
        <div class="default-item">
          {{ item.title }}
        </div>
      </slot>
    </div>
  </div>
</template>
```

Revenir au sommaire

Résumé du Module 2

- ✅ **Concepts** : Slots, composants génériques, relations parent/enfant
- ✅ **Slots simples** : Slot par défaut, contenu fallback
- ✅ **Slots nommés** : Plusieurs emplacements, directive v-slot
- ✅ **Scoped slots** : Passage de données enfant → parent
- ✅ **Slots dynamiques** : Détermination dynamique des slots
- ✅ **Props & Slots** : Combinaison pour composants puissants



Revenir au sommaire

Cas d'usage des slots

Type de slot	Cas d'usage
Slot simple	Contenu unique flexible (Button, Badge)
Slots nommés	Multiples zones (Card, Modal, Layout)
Scoped slots	Personnalisation du rendu (DataTable)
Slots dynamiques	UI configurable dynamiquement

Revenir au sommaire

Exercices Pratiques

Module 2 - Slots et composants génériques

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 2

Maintenant que vous avez vu tous les concepts théoriques du Module 2, il est temps de rendre nos composants TaskFlow ultra-flexibles avec les slots !

Rappel des concepts du Module 2

2.1 Introduction aux slots : Slots simples, nommés, fallback content
2.2 Implémentation : Scoped slots, slots dynamiques, v-slot

Les 3 exercices progressifs

Nous allons créer 3 composants TaskFlow modulaires et réutilisables avec les slots.

Revenir au sommaire

Exercice 1 : TaskCard avec Slots

Objectif

Créer un composant `TaskCard.vue` ultra-flexible avec slots

Spécifications

Slots nommés :

- `header` : En-tête personnalisable
- `default` : Contenu principal
- `actions` : Boutons d'actions
- `footer` : Pied de page

Props :

- `variant` : 'default' | 'success' | 'warning' | 'danger'
- `padding` : Boolean (true par défaut)

Fallback :

- Contenu par défaut pour chaque slot

Revenir au sommaire

Exercice 1 - Solution TaskCard.vue (1/2)

```vue
<script>
export default {
  name: 'TaskCard',
  props: {
    variant: {
      type: String,
      default: 'default',
      validator: (v) => ['default', 'success', 'warning', 'danger'].includes(v)
    },
    padding: {
      type: Boolean,
      default: true
    }
  },
  computed: {
    cardClasses() {
      return [
        'task-card',
        `task-card--${this.variant}`,
        { 'task-card--no-padding': !this.padding }
      ];
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 1 - Solution TaskCard.vue (2/3)

Template :

```
<template>
  <div :class="cardClasses">
    <div v-if="$slots.header" class="task-card__header">
      <slot name="header">
        <h3>Titre par défaut</h3>
      </slot>
    </div>

    <div class="task-card__body">
      <slot>
        <p>Contenu par défaut</p>
      </slot>
    </div>

    <div v-if="$slots.actions" class="task-card__actions">
      <slot name="actions"></slot>
    </div>

    <div v-if="$slots.footer" class="task-card__footer">
      <slot name="footer"></slot>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 1 - Solution TaskCard.vue (3/3)

Styles :

```vue
<style scoped>
.task-card {
  background: white;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  overflow: hidden;
}

.task-card--default { border-left: 4px solid #3b82f6; }
.task-card--success { border-left: 4px solid #10b981; }
.task-card--warning { border-left: 4px solid #f59e0b; }
.task-card--danger { border-left: 4px solid #ef4444; }

.task-card__header {
  padding: 1rem;
  border-bottom: 1px solid #e5e7eb;
  font-weight: 600;
}

.task-card__body {
  padding: 1rem;
}

.task-card--no-padding .task-card__body {
  padding: 0;
}

.task-card__actions {
  padding: 0.75rem 1rem;
  border-top: 1px solid #e5e7eb;
  display: flex;
  gap: 0.5rem;
  justify-content: flex-end;
}

.task-card__footer {
  padding: 0.75rem 1rem;
  background: #f9fafb;
  font-size: 0.875rem;
  color: #6b7280;
}
</style>
```

Revenir au sommaire

Exercice 1 - Utilisation TaskCard

```vue
<template>
  <div>
    <!-- Carte de tâche urgente -->
    <TaskCard variant="danger">
      <template #header>
        <div style="display: flex; justify-content: space-between;">
          <h3>🚨 Tâche urgente</h3>
          <span class="badge">URGENT</span>
        </div>
      </template>

      <p>Cette tâche nécessite une attention immédiate !</p>

      <template #actions>
        <button @click="markDone">✅ Terminer</button>
        <button @click="postpone">⏰ Reporter</button>
      </template>

      <template #footer>
        Échéance : Aujourd'hui 18h00
      </template>
    </TaskCard>

    <!-- Carte de tâche réussie -->
    <TaskCard variant="success">
      <template #header>
        <h3>✅ Tâche terminée</h3>
      </template>

      <p>Formation Vue.js complétée avec succès !</p>

      <template #footer>
        Complétée le 22/01/2026
      </template>
    </TaskCard>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 : TaskModal avec Slots

Objectif

Créer une modal ultra-flexible pour TaskFlow

Spécifications

Slots nommés :

- `trigger` : Bouton qui ouvre la modal
- `title` : Titre de la modal
- `default` : Contenu principal
- `footer` : Actions (Annuler, Valider)

Props :

- `modelValue` (Boolean) : Contrôle ouverture/fermeture
- `size` : 'small' | 'medium' | 'large'
- `closable` : Boolean (permet fermeture avec X)

Événements :

- `@update:modelValue` : Pour v-model
- `@close` : Quand on ferme la modal
- `@confirm` : Quand on confirme



Revenir au sommaire

Exercice 2 - Solution TaskModal.vue (1/3)

```vue
<script>
export default {
  name: 'TaskModal',
  props: {
    modelValue: Boolean,
    size: {
      type: String,
      default: 'medium',
      validator: (v) => ['small', 'medium', 'large'].includes(v)
    },
    closable: {
      type: Boolean,
      default: true
    }
  },
  emits: ['update:modelValue', 'close', 'confirm'],
  methods: {
    close() {
      this.$emit('update:modelValue', false);
      this.$emit('close');
    },
    confirm() {
      this.$emit('confirm');
      this.close();
    },
    handleBackdropClick(e) {
      if (this.closable && e.target === e.currentTarget) {
        this.close();
      }
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 2 - Solution TaskModal.vue (2/3)

```vue
<template>
  <Teleport to="body">
    <Transition name="modal">
      <div
        v-if="modelValue"
        class="modal-backdrop"
        @click="handleBackdropClick"
      >
        <div :class="['modal-content', `modal-content--${size}`]">
          <!-- Header -->
          <div class="modal-header">
            <div class="modal-title">
              <slot name="title">
                <h2>Titre</h2>
              </slot>
            </div>

            <button
              v-if="closable"
              class="modal-close"
              @click="close"
            >
              ×
            </button>
          </div>

          <!-- Body -->
          <div class="modal-body">
            <slot></slot>
          </div>

          <!-- Footer -->
          <div class="modal-footer" v-if="$slots.footer">
            <slot name="footer">
              <button @click="close">Annuler</button>
              <button @click="confirm" class="btn-primary">
                Confirmer
              </button>
            </slot>
          </div>
        </div>
      </div>
    </Transition>
  </Teleport>
</template>
```

Revenir au sommaire

Exercice 2 - Solution TaskModal Styles (1/2)

```
<style scoped>
.modal-backdrop {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.modal-content {
  background: white;
  border-radius: 12px;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
  max-height: 90vh;
  overflow: hidden;
  display: flex;
  flex-direction: column;
}

.modal-content--small { width: 400px; }
.modal-content--medium { width: 600px; }
.modal-content--large { width: 900px; }

.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1.5rem;
  border-bottom: 1px solid #e5e7eb;
}

.modal-title h2 {
  margin: 0;
  font-size: 1.5rem;
  font-weight: 600;
}
</style>
```

Revenir au sommaire

Exercice 2 - Solution TaskModal Styles (2/2)

```
<style scoped>
.modal-close {
  background: none;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
  color: #6b7280;
  width: 32px;
  height: 32px;
  border-radius: 6px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.modal-close:hover {
  background: #f3f4f6;
}

.modal-body {
  padding: 1.5rem;
  overflow-y: auto;
  flex: 1;
}

.modal-footer {
  padding: 1rem 1.5rem;
  border-top: 1px solid #e5e7eb;
  display: flex;
  gap: 0.75rem;
  justify-content: flex-end;
}

/* Transitions */
.modal-enter-active,
.modal-leave-active {
  transition: opacity 0.3s ease;
}

.modal-enter-from,
.modal-leave-to {
  opacity: 0;
}

.modal-enter-active .modal-content,
.modal-leave-active .modal-content {
  transition: transform 0.3s ease;
}

.modal-enter-from .modal-content,
.modal-leave-to .modal-content {
  transform: scale(0.9);
}
</style>
```



Revenir au sommaire

Exercice 2 - Utilisation TaskModal (1/2)

Script et Modal de suppression :

```
<script>
export default {
  data() {
    return {
      showDeleteModal: false,
      showEditModal: false,
      taskToEdit: null
    }
  },
  methods: {
    confirmDelete() {
      console.log('Tâche supprimée !');
    },
    saveTask() {
      console.log('Tâche sauvegardée !');
    }
  }
}
</script>

<template>
  <div>
    <!-- Modal de confirmation de suppression -->
    <TaskModal
      v-model="showDeleteModal"
      size="small"
    >
      <template #title>
        <h2>⚠️ Confirmer la suppression</h2>
      </template>

      <p>Êtes-vous sûr de vouloir supprimer cette tâche ?</p>
      <p><strong>Cette action est irréversible.</strong></p>

      <template #footer>
        <button @click="showDeleteModal = false">
          Annuler
        </button>
        <button
          @click="confirmDelete"
          class="btn-danger"
        >
          🗑️ Supprimer
        </button>
      </template>
    </TaskModal>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 - Utilisation TaskModal (2/2)

Modal d'édition :

```vue
<template>
  <div>
    <!-- Modal d'édition -->
    <TaskModal
      v-model="showEditModal"
      size="large"
    >
      <template #title>
        <h2>✏️ Modifier la tâche</h2>
      </template>

      <TaskForm
        :task="taskToEdit"
        @submit="saveTask"
      />
    </TaskModal>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 : BoardColumn (Scoped Slots)

Objectif

Créer une colonne de board Kanban avec scoped slots

Spécifications

Props :

- `title` : Titre de la colonne
- `tasks` : Array de tâches
- `color` : Couleur de la colonne

Scoped Slots** :

- `task` : Template personnalisable pour chaque tâche
  - Expose : `{ task, index, isFirst, isLast }`

Slots nommés :

- `header` : En-tête personnalisable
- `empty` : Affiché si aucune tâche

Events :

- `@drop` : Drag & drop de tâches



Revenir au sommaire

Exercice 3 - Solution BoardColumn.vue (1/3)

```
<script>
export default {
  name: 'BoardColumn',
  props: {
    title: {
      type: String,
      required: true
    },
    tasks: {
      type: Array,
      default: () => []
    },
    color: {
      type: String,
      default: '#3b82f6'
    }
  },
  emits: ['drop', 'task-click'],
  data() {
    return {
      isDragOver: false
    }
  },
  methods: {
    handleDragOver(e) {
      e.preventDefault();
      this.isDragOver = true;
    },
    handleDragLeave() {
      this.isDragOver = false;
    },
    handleDrop(e) {
      e.preventDefault();
      this.isDragOver = false;
      const taskId = e.dataTransfer.getData('taskId');
      this.$emit('drop', { taskId, column: this.title });
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 3 - Solution BoardColumn.vue (2/4)

Template - Header :

```vue
<template>
  <div
    class="board-column"
    :class="{ 'drag-over': isDragOver }"
    @dragover="handleDragOver"
    @dragleave="handleDragLeave"
    @drop="handleDrop"
  >
    <!-- Header personnalisable -->
    <div
      class="board-column__header"
      :style="{ borderTopColor: color }"
    >
      <slot name="header" :title="title" :count="tasks.length">
        <div class="header-default">
          <h3>{{ title }}</h3>
          <span class="task-count">{{ tasks.length }}</span>
        </div>
      </slot>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 - Solution BoardColumn.vue (3/4)

Template - Body et Liste :

```vue
<template>
  <div class="board-column">
    <!-- ... header de la slide précédente ... -->

    <!-- Liste des tâches -->
    <div class="board-column__body">
      <template v-if="tasks.length > 0">
        <div
          v-for="(task, index) in tasks"
          :key="task.id"
          class="board-column__task"
          draggable="true"
          @dragstart="e => e.dataTransfer.setData('taskId', task.id)"
        >
          <slot
            name="task"
            :task="task"
            :index="index"
            :isFirst="index === 0"
            :isLast="index === tasks.length - 1"
          >
            <!-- Fallback par défaut -->
            <div class="task-default">
              <h4>{{ task.title }}</h4>
              <p>{{ task.description }}</p>
            </div>
          </slot>
        </div>
      </template>

      <!-- Empty state -->
      <div v-else class="board-column__empty">
        <slot name="empty">
          <p>Aucune tâche</p>
        </slot>
      </div>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 - Solution BoardColumn Styles (1/2)

Styles colonne et header :

```
<style scoped>
.board-column {
  background: #f9fafb;
  border-radius: 12px;
  min-width: 300px;
  max-width: 350px;
  display: flex;
  flex-direction: column;
  max-height: 100%;
}

.board-column.drag-over {
  background: #eff6ff;
  border: 2px dashed #3b82f6;
}

.board-column__header {
  padding: 1rem;
  border-top: 4px solid;
  border-top-left-radius: 12px;
  border-top-right-radius: 12px;
  background: white;
}

.header-default {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.header-default h3 {
  margin: 0;
  font-size: 1.125rem;
  font-weight: 600;
}

.task-count {
  background: #e5e7eb;
  color: #374151;
  padding: 0.25rem 0.75rem;
  border-radius: 12px;
  font-size: 0.875rem;
  font-weight: 600;
}
</style>
```

Revenir au sommaire

Exercice 3 - Solution BoardColumn Styles (2/2)

Styles tâches et body :

```
<style scoped>
.board-column__body {
  padding: 1rem;
  flex: 1;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.board-column__task {
  cursor: grab;
  transition: transform 0.2s;
}

.board-column__task:active {
  cursor: grabbing;
}

.task-default {
  background: white;
  padding: 1rem;
  border-radius: 8px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

.task-default h4 {
  margin: 0 0 0.5rem 0;
}

.task-default p {
  margin: 0;
  color: #6b7280;
  font-size: 0.875rem;
}

.board-column__empty {
  text-align: center;
  padding: 2rem;
  color: #9ca3af;
}
</style>
```

Revenir au sommaire

Exercice 3 - Utilisation BoardColumn (1/2)

Script :

```
<script>
export default {
  data() {
    return {
      columns: {
        todo: [
          { id: 1, title: 'Apprendre Vue.js', priority: 'high' },
          { id: 2, title: 'Créer TaskFlow', priority: 'urgent' }
        ],
        inProgress: [
          { id: 3, title: 'Setup projet', priority: 'medium' }
        ],
        done: [
          { id: 4, title: 'Formation terminée', priority: 'low' }
        ]
      }
    }
  },
  methods: {
    handleDrop({ taskId, column }) {
      console.log(`Tâche ${taskId} déplacée vers ${column}`);
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 3 - Utilisation BoardColumn (2/2)

Template :

```
<template>
  <div class="kanban-board">
    <BoardColumn
      title="À faire"
      :tasks="columns.todo"
      color="#ef4444"
      @drop="handleDrop"
    >
      <template #header="{ title, count }">
        <div class="custom-header">
          <h3>📋 {{ title }}</h3>
          <span class="badge-red">{{ count }}</span>
        </div>
      </template>

      <template #task="{ task, isFirst }">
        <div
          class="task-card"
          :class="{ 'task-highlight': isFirst }"
        >
          <div class="task-priority" :class="`priority-${task.priority}`">
            {{ task.priority }}
          </div>
          <h4>{{ task.title }}</h4>
        </div>
      </template>

      <template #empty>
        <div class="empty-state">
          🎉 Aucune tâche à faire !
        </div>
      </template>
    </BoardColumn>

    <BoardColumn
      title="En cours"
      :tasks="columns.inProgress"
      color="#f59e0b"
      @drop="handleDrop"
    />

    <BoardColumn
      title="Terminé"
      :tasks="columns.done"
      color="#10b981"
      @drop="handleDrop"
    />
  </div>
</template>
```

Revenir au sommaire

🎯 Récap Module 2 - TaskFlow

Ce que vous avez construit

✅ **TaskCard** : Composant flexible avec 4 slots nommés

✅ **TaskModal** : Modal réutilisable avec Teleport

✅ **BoardColumn** : Colonne Kanban avec scoped slots et drag&drop

Concepts du Module 2 utilisés

- ✅ **2.1 Introduction** : Slots simples, nommés, fallback
- ✅ **2.2 Implémentation** : Scoped slots, $slots, v-slot, Teleport

🚀 Prochaine étape : Module 3

On va ajouter des **directives personnalisées** à TaskFlow !

- v-auto-focus pour inputs
- v-click-outside pour fermer modals
- v-tooltip pour afficher des infos

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Vue.js Slots Guide
- 🎯 Scoped Slots
- 🚀 Teleport

Concepts avancés

- 🔧 Render Functions avec slots
- ⚡ Slots et performance
- 🎨 Patterns de composants réutilisables

Exemples

- 💡 Vue Component Library Patterns
- 🎭 Headless UI Components

Revenir au sommaire

Félicitations ! 🎉

Vous avez terminé le Module 2 !

Vous maîtrisez maintenant :

- ✅ Les slots simples et nommés
- ✅ Les scoped slots pour exposer des données
- ✅ La détection de slots avec $slots
- ✅ Le fallback content
- ✅ Teleport pour les modals
- ✅ Composants ultra-flexibles et réutilisables

Vos composants TaskFlow

3 composants modulaires :

- TaskCard : 4 slots nommés
- TaskModal : Teleport + slots
- BoardColumn : Scoped slots + drag&drop

Revenir au sommaire

Module 3

Vue.js - Fonctionnement avancé

Revenir au sommaire

SOMMAIRE DU MODULE

📦 3.1 Single File Components

✨ 3.2 Directives personnalisées

💪 Exercices Pratiques du Module 3

Revenir au sommaire

3.1 Single File Components (SFC)

Revenir au sommaire

Single File Components (SFC)

Les **SFC** sont des fichiers `.vue` contenant template, script et style

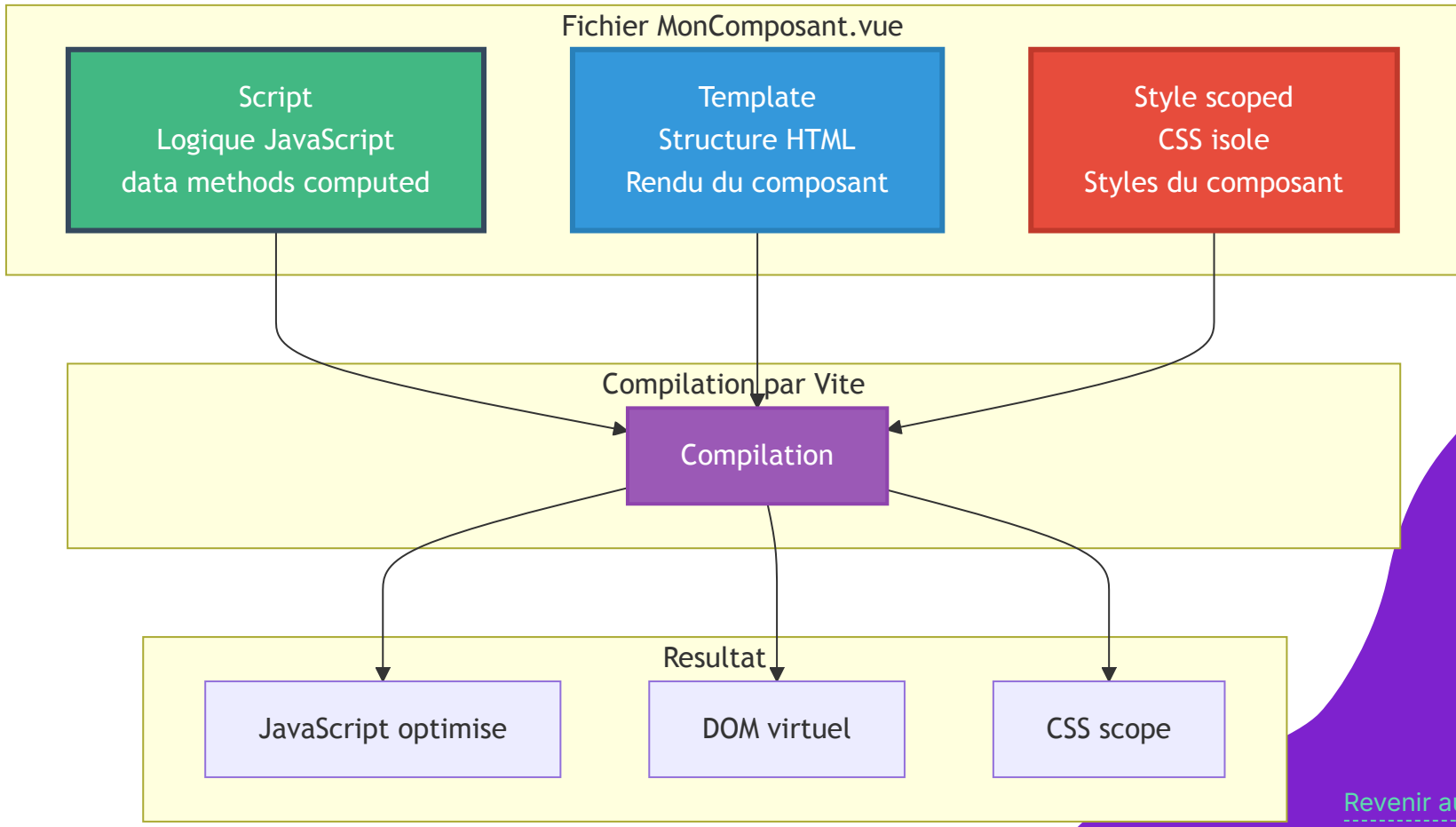
Avantages des SFC

- 📦 Encapsulation complète
- 🎨 Scoped CSS
- 🔧 Syntaxe spécifique à Vue
- ⚡ Hot Module Replacement
- 📝 Support IDE excellent



Revenir au sommaire

Schéma : Architecture SFC



Revenir au sommaire

Structure d'un SFC

```
<script>
export default {
  data() {
    return {
      count: 0
    }
  }
}
</script>

<template>
  <!-- Template du composant -->
  <div>
    <p>{{ count }}</p>
    <button @click="count++">Increment</button>
  </div>
</template>

<style scoped>
/* Styles scopés au composant */
div {
  padding: 20px;
}

button {
  background: #42b883;
  color: white;
}
</style>
```

Revenir au sommaire

SFC - Ordre des sections

L'ordre recommandé (mais flexible) :

```vue
<!-- 1. Script -->
<script>
// Imports, logique, configuration
export default {
  // Configuration du composant
}
</script>

<!-- 2. Template -->
<template>
  <!-- Structure HTML -->
</template>

<!-- 3. Style -->
<style scoped>
/* CSS du composant */
</style>
```

Revenir au sommaire

Options API - Structure classique

L'Options API offre une structure claire et intuitive

```
<script>
export default {
  data() {
    return {
      count: 0
    };
  },
  computed: {
    double() {
      return this.count * 2;
    }
  },
  methods: {
    increment() {
      this.count++;
    }
  }
};
</script>
```

💡 **Note :** Pour des cas d'usage avancés, Vue 3 propose aussi la Composition API disponible en module bonus

Revenir au sommaire

Scoped CSS

Le CSS scopé ne s'applique qu'au composant actuel

```
<style scoped>
/* Ne s'applique qu'à ce composant */
.button {
  background: #42b883;
}

/* Sélecteur profond - affecte les enfants */
:deep(.child-class) {
  color: red;
}

/* Sélecteur slotted - pour le contenu des slots */
:slotted(.slot-content) {
  font-weight: bold;
}

/* Sélecteur global dans scoped */
:global(.global-class) {
  margin: 0;
}
</style>
```

Revenir au sommaire

CSS Modules

Alternative au scoped CSS

```
<style module>
.container {
  padding: 20px;
}

.title {
  font-size: 24px;
  color: #42b883;
}
</style>

<template>
  <div :class="$style.container">
    <h1 :class="$style.title">Titre</h1>
  </div>
</template>
```



Revenir au sommaire

Variables CSS réactives

Lier des variables JavaScript aux variables CSS

```
<script>
export default {
  data() {
    return {
      color: '#42b883',
      size: 20
    }
  }
}
</script>

<template>
  <div class="dynamic">
    <p>Texte avec couleur dynamique</p>
    <input v-model="color" type="color" />
  </div>
</template>

<style scoped>
.dynamic {
  color: v-bind(color);
  font-size: v-bind(size + 'px');
}
</style>
```

Revenir au sommaire

Support des preprocesseurs CSS

Vue.js supporte nativement les preprocesseurs CSS comme Sass, Less et Stylus.

Il suffit d'installer le preprocesseur souhaité et d'utiliser l'attribut `lang` :

```
npm install -D sass
```

```
<style lang="scss" scoped>
$primary-color: #42b883;

.container {
  background: $primary-color;

  .title {
    font-size: 24px;
    &:hover { opacity: 0.8; }
  }
}
</style>
```



Revenir au sommaire

TypeScript dans les SFC

💡 **Note :** Vue 3 supporte TypeScript de manière native. Voir module bonus pour plus de détails.

```
<script>
// Exemple équivalent en JavaScript
export default {
  data() {
    return {
      user: {
        id: 1,
        nom: 'Alice',
        email: 'alice@example.com'
      }
    }
  },
  computed: {
    displayName() {
      return this.user.nom.toUpperCase();
    }
  },
  methods: {
    updateUser(newUser) {
      this.user = newUser;
    }
  }
}
</script>
```

Revenir au sommaire

Props validés avec JavaScript

```
<script>
export default {
  props: {
    nom: {
      type: String,
      required: true
    },
    age: {
      type: Number,
      default: 18
    },
    isActive: {
      type: Boolean,
      required: true
    },
    tags: {
      type: Array,
      default: () => []
    }
  },
  emits: {
    update: (value) => typeof value === 'string',
    delete: (id) => typeof id === 'number'
  }
}
</script>
```

Revenir au sommaire

3.2 Directives personnalisées

Revenir au sommaire

Qu'est-ce qu'une directive ?

Les directives sont des attributs spéciaux qui modifient le comportement des éléments DOM

Directives natives Vue.js

- `v-if` / `v-show`
- `v-for`
- `v-bind` (`:`)
- `v-on` (`@`)
- `v-model`
- `v-html` / `v-text`
- `v-cloak`
- `v-pre` / `v-once`



Revenir au sommaire

Directives natives avancées

Vue.js propose plusieurs directives natives pour des cas spécifiques

Directive	Utilisation
v-html	Rendre du HTML brut (⚠️ attention XSS)
v-once	Rendre une seule fois (pas de réactivité)
v-pre	Sauter la compilation Vue
v-cloak	Cacher l'élément jusqu'au chargement de Vue
v-memo	Mémoiser un sous-arbre (Vue 3.2+)

Revenir au sommaire

```vue
<template>
  <!-- v-html -->
  <div v-html="htmlContent"></div>

  <!-- v-once : restera "Initial" même si message change -->
  <p v-once>{{ message }}</p>

  <!-- v-pre : {{ ceci ne sera pas interprété }} -->
  <div v-pre>{{ texte brut }}</div>

  <!-- v-memo : mise à jour uniquement si count change -->
  <div v-memo="[count]">
    <p>{{ count }}</p>
    <p>{{ text }}</p>
  </div>
</template>
```

Revenir au sommaire

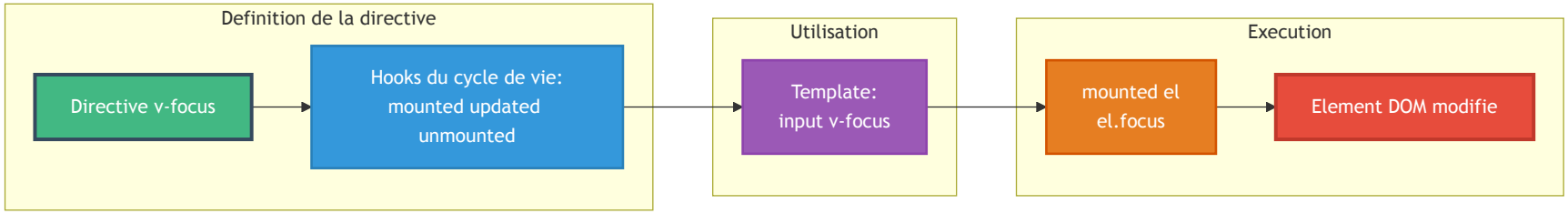
Créer une directive personnalisée

Syntaxe de base

```
// directives/focus.js
export const vFocus = {
  mounted(el) {
    el.focus();
  }
};
```

Revenir au sommaire

Schéma : Cycle de vie directive



Revenir au sommaire

Utilisation d'une directive personnalisée

```
<script>
import { vFocus } from './directives/focus';

export default {
  directives: {
    focus: vFocus
  }
}
</script>

<template>
  <input v-focus type="text" />
</template>
```

Revenir au sommaire

Lifecycle hooks des directives

```
const myDirective = {
  // Avant que l'élément soit inséré dans le DOM
  created(el, binding, vnode, prevVnode) {},

  // Avant les mises à jour du composant parent
  beforeMount(el, binding, vnode, prevVnode) {},

  // Quand l'élément est monté
  mounted(el, binding, vnode, prevVnode) {},

  // Avant mise à jour
  beforeUpdate(el, binding, vnode, prevVnode) {},

  // Après mise à jour
  updated(el, binding, vnode, prevVnode) {},

  // Avant démontage
  beforeUnmount(el, binding, vnode, prevVnode) {},

  // Après démontage
  unmounted(el, binding, vnode, prevVnode) {}
};
```

Revenir au sommaire

Arguments de la directive

```
export const vExample = {
  mounted(el, binding, vnode) {
    // el: l'élément DOM
    console.log(el);

    // binding.value: la valeur passée à la directive
    console.log(binding.value);

    // binding.arg: l'argument après les deux-points
    console.log(binding.arg);

    // binding.modifiers: objet des modificateurs
    console.log(binding.modifiers);

    // vnode: le VNode de l'élément
    console.log(vnode);
  }
};
```



Revenir au sommaire

Exemple : Directive v-click-outside

```
// directives/clickOutside.js
export const vClickOutside = {
  mounted(el, binding) {
    el.clickOutsideEvent = (event) => {
      // Vérifier si le clic est en dehors de l'élément
      if (!(el === event.target || el.contains(event.target))) {
        // Appeler la fonction fournie
        binding.value(event);
      }
    };

    document.addEventListener('click', el.clickOutsideEvent);
  },

  unmounted(el) {
    document.removeEventListener('click', el.clickOutsideEvent);
  }
};
```



Revenir au sommaire

Utilisation de v-click-outside

```
<script>
import { vClickOutside } from './directives/clickOutside';

export default {
  directives: {
    clickOutside: vClickOutside
  },
  data() {
    return {
      isOpen: false
    }
  },
  methods: {
    closeDropdown() {
      this.isOpen = false;
    }
  }
}
</script>

<template>
  <div v-click-outside="closeDropdown" class="dropdown">
    <button @click="isOpen = !isOpen">Toggle</button>
    <div v-if="isOpen" class="menu">
      Contenu du menu
    </div>
  </div>
</template>
```

Revenir au sommaire

Exemple : Directive v-tooltip

```
// directives/tooltip.js
export const vTooltip = {
  mounted(el, binding) {
    const tooltip = document.createElement('div');
    tooltip.className = 'tooltip';
    tooltip.textContent = binding.value;
    tooltip.style.position = 'absolute';
    tooltip.style.display = 'none';
    document.body.appendChild(tooltip);

    el.addEventListener('mouseenter', () => {
      const rect = el.getBoundingClientRect();
      tooltip.style.top = rect.top - 30 + 'px';
      tooltip.style.left = rect.left + 'px';
      tooltip.style.display = 'block';
    });

    el.addEventListener('mouseleave', () => {
      tooltip.style.display = 'none';
    });

    el._tooltip = tooltip;
  },

  unmounted(el) {
    if (el._tooltip) {
      el._tooltip.remove();
    }
  }
};
```

Revenir au sommaire

Utilisation de v-tooltip

```
<script>
import { vTooltip } from './directives/tooltip';

export default {
  directives: {
    tooltip: vTooltip
  }
}
</script>

<template>
  <button v-tooltip="'Cliquez pour enregistrer'">
    Enregistrer
  </button>

  <span v-tooltip="'Information importante'">
    Hover me
  </span>
</template>

<style>
.tooltip {
  background: #333;
  color: white;
  padding: 5px 10px;
  border-radius: 4px;
  font-size: 14px;
  z-index: 1000;
}
</style>
```

Revenir au sommaire

Directive globale

Enregistrer une directive globalement

```
// main.js
import { createApp } from 'vue';
import App from './App.vue';

const app = createApp(App);

// Directive globale
app.directive('focus', {
  mounted(el) {
    el.focus();
  }
});

app.mount('#app');
```

Revenir au sommaire

Plugin de directives

Grouper plusieurs directives dans un plugin

```
// plugins/directives.js
import { vFocus } from '../directives/focus';
import { vClickOutside } from '../directives/clickOutside';
import { vTooltip } from '../directives/tooltip';
import { vLazyLoad } from '../directives/lazyLoad';

export default {
  install(app) {
    app.directive('focus', vFocus);
    app.directive('click-outside', vClickOutside);
    app.directive('tooltip', vTooltip);
    app.directive('lazy-load', vLazyLoad);
  }
};
```

Revenir au sommaire

Utilisation du plugin

```
// main.js
import { createApp } from 'vue';
import App from './App.vue';
import directivesPlugin from './plugins/directives';

const app = createApp(App);

app.use(directivesPlugin);

app.mount('#app');
```



Revenir au sommaire

Bonnes pratiques - Directives

✅ Faire

- Nettoyer les event listeners dans unmounted
- Utiliser des noms descriptifs
- Documenter les arguments et modificateurs
- Gérer les cas edge
- Utiliser TypeScript pour les types

❌ Éviter

- Manipuler le state du composant
- Logique métier complexe
- Directives trop génériques
- Oublier le nettoyage des ressources

Revenir au sommaire

Résumé du Module 3

- ✅ **Single File Components** : Structure, scoped CSS, CSS modules
- ✅ **Variables CSS réactives** : v-bind dans le CSS
- ✅ **TypeScript** : Props typées, interfaces
- ✅ **Directives natives** : v-html, v-once, v-pre, v-memo
- ✅ **Directives personnalisées** : Création, lifecycle, arguments
- ✅ **Exemples pratiques** : v-focus, v-click-outside, v-tooltip



Revenir au sommaire

Cas d'usage des directives

Directive	Cas d'usage
v-focus	Auto-focus sur un input
v-click-outside	Fermer dropdown/modal
v-tooltip	Afficher des infobulles

Revenir au sommaire

Travaux pratiques - Module 3

Exercices à réaliser

1. 📝 **SFC avancé** : Composant
2. 🎨 **CSS dynamique** : Thème avec variables réactives
3. 🔧 **Directive v-click-outside** : Implémenter
4. 💬 **Directive v-tooltip** : Créer des tooltips

Projet fil rouge

Refactoriser l'application avec des directives personnalisées

Revenir au sommaire

Exercices Pratiques

Module 3 - Vue.js avancé

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 3

Maintenant que vous avez vu tous les concepts avancés du Module 3, il est temps d'enrichir TaskFlow avec des directives personnalisées !

Rappel des concepts du Module 3

3.1 SFC avancés : Scoped CSS, CSS Modules, Variables CSS réactives
3.2 Directives : Directives personnalisées, lifecycle, manipulation DOM
3.3 Composants fonctionnels : Performance et cas d'usage
3.4 Plugins : Création et utilisation de plugins Vue

Les 4 exercices progressifs

Nous allons ajouter des directives personnalisées pour améliorer l'UX de TaskFlow.

Revenir au sommaire

Exercice 1 : Directive v-auto-focus

Objectif

Créer une directive qui focus automatiquement un input

Spécifications

Nom : `v-auto-focus`

Comportement :

- Focus l'élément au montage du composant
- Support d'une condition : `v-auto-focus="condition"`
- Refocus si la condition change de false à true

Cas d'usage :

- Input dans une modal qui s'ouvre
- Champ de recherche au chargement
- Formulaire d'édition de tâche

Revenir au sommaire

Exercice 1 - Solution v-auto-focus

```
// directives/autoFocus.js
export const vAutoFocus = {
  mounted(el, binding) {
    // Si binding.value est undefined ou true
    if (binding.value === undefined || binding.value) {
      // Attendre le prochain tick pour être sûr que le DOM est prêt
      setTimeout(() => {
        el.focus();
      }, 100);
    }
  },

  updated(el, binding) {
    // Si la valeur change de false à true
    if (!binding.oldValue && binding.value) {
      setTimeout(() => {
        el.focus();
      }, 100);
    }
  }
};
```

Revenir au sommaire

Exercice 1 - Utilisation v-auto-focus

```
<script>
import { vAutoFocus } from './directives/autoFocus';

export default {
  directives: {
    autoFocus: vAutoFocus
  },
  data() {
    return {
      showModal: false,
      searchMode: false
    }
  }
}
</script>

<template>
  <div>
    <!-- Focus automatique au chargement -->
    <input
      v-auto-focus
      type="text"
      placeholder="Rechercher..."
    />

    <!-- Focus conditionnel -->
    <input
      v-auto-focus="searchMode"
      type="search"
      placeholder="Mode recherche..."
    />

    <!-- Dans une modal -->
    <TaskModal v-model="showModal">
      <template #title>Nouvelle tâche</template>

      <input
        v-auto-focus="showModal"
        type="text"
        placeholder="Titre de la tâche"
      />
    </TaskModal>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 : Directive v-click-outside

Objectif

Détecter les clics en dehors d'un élément

Spécifications

Nom : `v-click-outside`

Comportement :

- Appelle une fonction quand on clique en dehors de l'élément
- Ignore les clics à l'intérieur de l'élément
- Cleanup des event listeners au démontage

Cas d'usage :

- Fermer un dropdown
- Fermer une modal
- Fermer un menu contextuel
- Fermer un date picker



Revenir au sommaire

Exercice 2 - Solution v-click-outside (1/2)

```
// directives/clickOutside.js
export const vClickOutside = {
  mounted(el, binding) {
    // Stocker la fonction de callback
    el._clickOutsideHandler = (event) => {
      // Vérifier si le clic est en dehors de l'élément
      if (!el.contains(event.target) && el !== event.target) {
        // Appeler la fonction passée en paramètre
        binding.value(event);
      }
    };

    // Ajouter l'event listener sur document
    // Utiliser capture pour détecter les clics avant la propagation
    document.addEventListener('click', el._clickOutsideHandler, true);
  },

  unmounted(el) {
    // ✅ CLEANUP : Retirer l'event listener
    if (el._clickOutsideHandler) {
      document.removeEventListener('click', el._clickOutsideHandler, true);
      delete el._clickOutsideHandler;
    }
  }
};
```

Revenir au sommaire

Exercice 2 - Solution v-click-outside (2/2)

```
<script>
import { vClickOutside } from './directives/clickOutside';

export default {
  directives: {
    clickOutside: vClickOutside
  },
  data() {
    return {
      showDropdown: false,
      showContextMenu: false
    }
  },
  methods: {
    closeDropdown() {
      this.showDropdown = false;
    },
    closeContextMenu() {
      this.showContextMenu = false;
    }
  }
}
</script>

<template>
  <div>
    <!-- Dropdown -->
    <div
      v-click-outside="closeDropdown"
      class="dropdown"
    >
      <button @click="showDropdown = !showDropdown">
        Options ▼
      </button>

      <div v-if="showDropdown" class="dropdown-menu">
        <button>Modifier</button>
        <button>Supprimer</button>
      </div>
    </div>

    <!-- Context Menu -->
    <div
      v-click-outside="closeContextMenu"
      class="context-menu"
      v-if="showContextMenu"
    >
      <button>Copier</button>
      <button>Coller</button>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 : Directive v-tooltip

Objectif

Afficher un tooltip au survol d'un élément

Spécifications

Nom : `v-tooltip`

Valeur : String (texte du tooltip) ou Object

Options :

- `text` : Texte à afficher
- `position` : 'top' | 'bottom' | 'left' | 'right'
- `theme` : 'dark' | 'light'

Comportement :

- Afficher au mouseenter
- Masquer au mouseleave
- Créer/détruire le DOM du tooltip
- Cleanup au démontage

Revenir au sommaire

Exercice 3 - Solution v-tooltip (1/3)

```
// directives/tooltip.js
export const vTooltip = {
  mounted(el, binding) {
    let tooltipEl;

    // Parser les options
    const options = typeof binding.value === 'string'
      ? { text: binding.value, position: 'top', theme: 'dark' }
      : { position: 'top', theme: 'dark', ...binding.value };

    // Fonction pour créer le tooltip
    const createTooltip = () => {
      tooltipEl = document.createElement('div');
      tooltipEl.className = `tooltip tooltip--${options.theme} tooltip--${options.position}`;
      tooltipEl.textContent = options.text;
      document.body.appendChild(tooltipEl);

      // Positionner le tooltip
      const rect = el.getBoundingClientRect();
      const tooltipRect = tooltipEl.getBoundingClientRect();

      let top, left;

      switch (options.position) {
        case 'top':
          top = rect.top - tooltipRect.height - 8;
          left = rect.left + (rect.width - tooltipRect.width) / 2;
          break;
        case 'bottom':
          top = rect.bottom + 8;
          left = rect.left + (rect.width - tooltipRect.width) / 2;
          break;
        case 'left':
          top = rect.top + (rect.height - tooltipRect.height) / 2;
          left = rect.left - tooltipRect.width - 8;
          break;
        case 'right':
          top = rect.top + (rect.height - tooltipRect.height) / 2;
          left = rect.right + 8;
          break;
      }

      tooltipEl.style.top = `${top}px`;
      tooltipEl.style.left = `${left}px`;
      tooltipEl.style.opacity = '1';
    };

    // Suite dans la slide suivante...
  }
};
```

Revenir au sommaire

Exercice 3 - Solution v-tooltip (2/3)

```
// directives/tooltip.js (suite)
export const vTooltip = {
  mounted(el, binding) {
    // ... code précédent

    // Fonction pour détruire le tooltip
    const destroyTooltip = () => {
      if (tooltipEl) {
        tooltipEl.style.opacity = '0';
        setTimeout(() => {
          if (tooltipEl && tooltipEl.parentNode) {
            tooltipEl.parentNode.removeChild(tooltipEl);
            tooltipEl = null;
          }
        }, 200);
      }
    };

    // Event listeners
    el._mouseenterHandler = createTooltip;
    el._mouseleaveHandler = destroyTooltip;

    el.addEventListener('mouseenter', el._mouseenterHandler);
    el.addEventListener('mouseleave', el._mouseleaveHandler);
  },

  unmounted(el) {
    // ✅ CLEANUP
    if (el._mouseenterHandler) {
      el.removeEventListener('mouseenter', el._mouseenterHandler);
      delete el._mouseenterHandler;
    }
    if (el._mouseleaveHandler) {
      el.removeEventListener('mouseleave', el._mouseleaveHandler);
      delete el._mouseleaveHandler;
    }
  }
};
```

Revenir au sommaire

Exercice 3 - Solution v-tooltip Styles

```css
/* styles/tooltip.css */
.tooltip {
  position: fixed;
  padding: 0.5rem 0.75rem;
  border-radius: 6px;
  font-size: 0.875rem;
  white-space: nowrap;
  z-index: 9999;
  opacity: 0;
  transition: opacity 0.2s ease;
  pointer-events: none;
}

.tooltip--dark {
  background: #1f2937;
  color: white;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
}

.tooltip--light {
  background: white;
  color: #1f2937;
  border: 1px solid #e5e7eb;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

/* Flèches (optionnel) */
.tooltip--top::after {
  content: '';
  position: absolute;
  top: 100%;
  left: 50%;
  transform: translateX(-50%);
  border: 4px solid transparent;
  border-top-color: #1f2937;
}
```

Revenir au sommaire

Exercice 3 - Utilisation v-tooltip

```vue
<script>
import { vTooltip } from './directives/tooltip';

export default {
  directives: {
    tooltip: vTooltip
  }
}
</script>

<template>
  <div>
    <!-- Tooltip simple -->
    <button v-tooltip="'Cliquez pour ajouter une tâche'">
      ➕ Nouvelle tâche
    </button>

    <!-- Tooltip avec options -->
    <button
      v-tooltip="{
        text: 'Supprimer la tâche',
        position: 'right',
        theme: 'light'
      }"
    >
      🗑️
    </button>

    <!-- Sur un badge de priorité -->
    <span
      v-tooltip="'Cette tâche est urgente !'"
      class="badge badge-urgent"
    >
      URGENT
    </span>

    <!-- Sur une icône d'aide -->
    <span
      v-tooltip="{
        text: 'Les tâches archivées sont supprimées après 30 jours',
        position: 'bottom'
      }"
      class="help-icon"
    >
      ❓
    </span>
  </div>
</template>
```

Revenir au sommaire

Exercice 4 : Plugin de Directives

Objectif

Créer un plugin pour enregistrer toutes les directives TaskFlow

Spécifications

Fichier : `plugins/taskflowDirectives.js`

Directives à inclure :

- `v-auto-focus`
- `v-click-outside`
- `v-tooltip`

Installation :

```
app.use(TaskFlowDirectives);
```

Bonus :

- Options de configuration globales
- Version du plugin



Revenir au sommaire

Exercice 4 - Solution Plugin Directives

```
// plugins/taskflowDirectives.js
import { vAutoFocus } from '../directives/autoFocus';
import { vClickOutside } from '../directives/clickOutside';
import { vTooltip } from '../directives/tooltip';

export const TaskFlowDirectives = {
  install(app, options = {}) {
    // Configuration par défaut
    const config = {
      prefix: 'task',
      ...options
    };

    // Enregistrer les directives
    app.directive('autoFocus', vAutoFocus);
    app.directive('clickOutside', vClickOutside);
    app.directive('tooltip', vTooltip);

    // Ajouter la config globale
    app.config.globalProperties.$taskflowDirectives = {
      version: '1.0.0',
      config
    };

    console.log('✅ TaskFlow Directives plugin installé !');
  }
};

// Utilisation dans main.js
// import { createApp } from 'vue';
// import App from './App.vue';
// import { TaskFlowDirectives } from './plugins/taskflowDirectives';
//
// const app = createApp(App);
// app.use(TaskFlowDirectives);
// app.mount('#app');
```

Revenir au sommaire

🎯 Récap Module 3 - TaskFlow

Ce que vous avez construit

✅ **v-auto-focus** : Focus automatique des inputs

✅ **v-click-outside** : Détection clics externes

✅ **v-tooltip** : Tooltips personnalisables

✅ **Plugin** : Package réutilisable de directives

Concepts du Module 3 utilisés

- ✅ **3.1 SFC avancés** : Scoped styles, CSS modules
- ✅ **3.2 Directives** : Lifecycle (mounted, updated, unmounted), binding values
- ✅ **3.3 Manipulation DOM** : createElement, appendChild, event listeners
- ✅ **3.4 Plugins** : Structure plugin.install, config globale

🚀 Prochaine étape : Module 4

On va gérer l'**état global** de TaskFlow avec **Pinia** !

- Store de tâches centralisé
- Actions asynchrones
- Persistance localStorage

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Vue.js Custom Directives
- 🔌 Vue.js Plugins
- 🎨 SFC CSS Features

Concepts avancés

- 🔧 Render Functions
- ⚡ Teleport API
- 🎭 Async Components

Exemples

- 💡 Directive Examples
- 🎯 Plugin Patterns



Revenir au sommaire

Module 4

Navigation et routage avec Vue Router

Revenir au sommaire

SOMMAIRE DU MODULE

🗺️ 5.1 Introduction à Vue Router

⚙️ 5.2 Configuration et routes

🚀 5.3 Fonctionnalités avancées

💪 Exercices Pratiques du Module 5



Revenir au sommaire

5.1 Introduction à Vue Router

Revenir au sommaire

Qu'est-ce que Vue Router ?

Vue Router est le routeur officiel pour les applications Vue.js

🗺️ Analogie : Vue Router est comme un GPS

Imaginez votre application comme une ville :

- **Les routes** = les rues de la ville 🏘️
- **Le Router** = votre GPS 📱
- **Les composants** = les destinations (maison, bureau, magasin...)

Quand vous tapez une adresse (URL), le GPS (Router) vous emmène à la bonne destination (composant) !

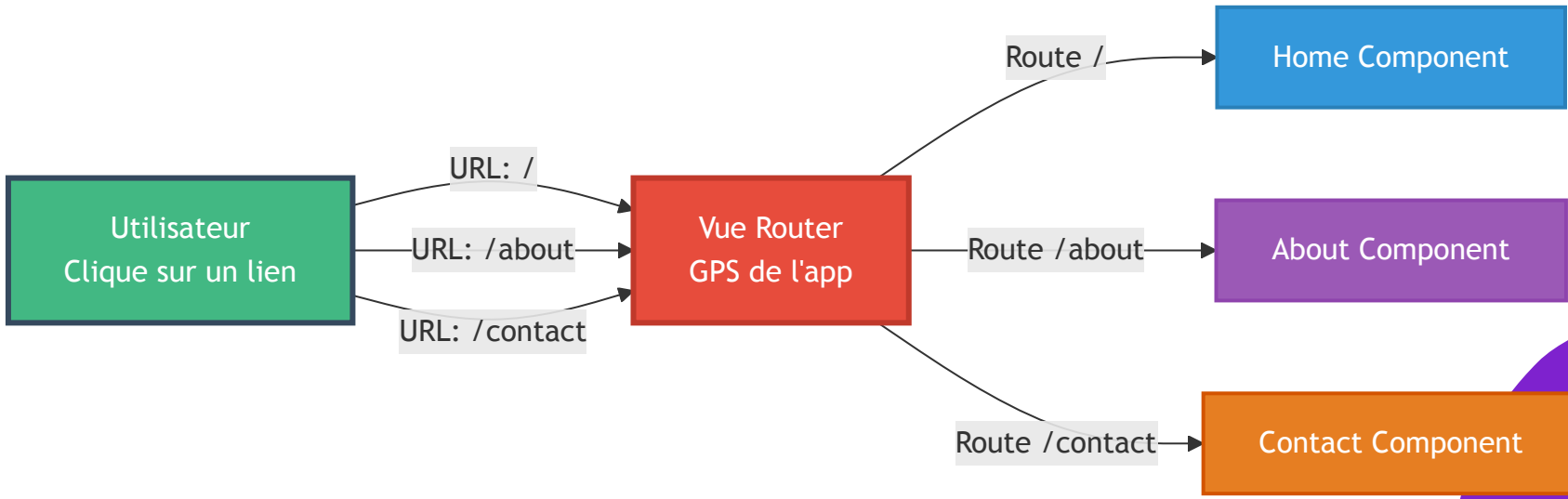
🆚 Sans Router : SPA = Single Page (vraiment UNE page)

Application traditionnelle multi-pages :

```
/index.html      ← Page 1 (rechargement complet ⏳)
/about.html      ← Page 2 (rechargement complet ⏳)
/contact.html    ← Page 3 (rechargement complet ⏳)
```

Revenir au sommaire

Schéma : Vue Router - Navigation



Revenir au sommaire

SPA avec Vue Router

SPA avec Router :

```
/               ← Home Component (instantané ⚡)
/about          ← About Component (instantané ⚡)
/contact        ← Contact Component (instantané ⚡)
```

Revenir au sommaire

La magie du SPA

Une seule page HTML, mais l'illusion de plusieurs pages !

Fonctionnalités

- 🚧 **Navigation entre "pages"** : sans rechargement complet !
- 📍 **Gestion de l'URL** : /products/123 au lieu de ?page=products&id=123
- 🔙 **Historique** : boutons retour/avant fonctionnent
- 🔐 **Guards** : "Tu ne passeras pas !" sans authentification
- ⚡ **Lazy loading** : charger les pages à la demande
- 🎯 **Routes dynamiques** : /user/:id pour tous les utilisateurs

Revenir au sommaire

5.2 Configuration et routes

Revenir au sommaire

Installation

```
npm install vue-router@4
```



Revenir au sommaire

Configuration de base

```
// router/index.js
import { createRouter, createWebHistory } from 'vue-router';
import Home from '../views/Home.vue';
import About from '../views/About.vue';

const routes = [
  {
    path: '/',
    name: 'home',
    component: Home
  },
  {
    path: '/about',
    name: 'about',
    component: About
  }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```

Revenir au sommaire

Intégration dans main.js

```
// main.js
import { createApp } from 'vue';
import App from './App.vue';
import router from './router';

const app = createApp(App);

app.use(router);
app.mount('#app');
```

Revenir au sommaire

RouterView et RouterLink

```vue
<!-- App.vue -->
<template>
  <div>
    <nav>
      <RouterLink to="/">Accueil</RouterLink>
      <RouterLink to="/about">À propos</RouterLink>
      <RouterLink to="/contact">Contact</RouterLink>
    </nav>

    <main>
      <RouterView />
    </main>
  </div>
</template>
```



Revenir au sommaire

Routes dynamiques

```
const routes = [
  {
    path: '/user/:id',
    name: 'user',
    component: UserProfile
  },
  {
    path: '/product/:slug',
    name: 'product',
    component: ProductDetail
  }
];
```

Revenir au sommaire

Accéder aux paramètres de route

```
<script>
export default {
  computed: {
    userId() {
      return this.$route.params.id;
    },
    tab() {
      return this.$route.query.tab;
    }
  }
}
</script>

<template>
  <div>
    <h1>User ID: {{ $route.params.id }}</h1>
    <p>Tab: {{ $route.query.tab }}</p>
  </div>
</template>
```

Revenir au sommaire

Navigation programmatique

```
<script>
export default {
  methods: {
    goToUser(id) {
      this.$router.push(`/user/${id}`);
    },
    goBack() {
      this.$router.back();
    },
    goToAbout() {
      this.$router.push({ name: 'about' });
    },
    goToProduct() {
      this.$router.push({
        name: 'product',
        params: { slug: 'laptop' },
        query: { color: 'black' }
      });
    }
  }
}
</script>
```

Revenir au sommaire

Lazy Loading des routes

```
const routes = [
  {
    path: '/',
    name: 'home',
    component: () => import('../views/Home.vue')
  },
  {
    path: '/about',
    name: 'about',
    component: () => import('../views/About.vue')
  },
  {
    path: '/admin',
    name: 'admin',
    component: () => import('../views/Admin.vue')
  }
];
```



Revenir au sommaire

5.3 Fonctionnalités avancées

Revenir au sommaire

Routes imbriquées (Nested Routes)

```
const routes = [
  {
    path: '/user/:id',
    component: UserLayout,
    children: [
      {
        path: '',
        name: 'user-profile',
        component: UserProfile
      },
      {
        path: 'posts',
        name: 'user-posts',
        component: UserPosts
      },
      {
        path: 'settings',
        name: 'user-settings',
        component: UserSettings
      }
    ]
  }
];
```



Revenir au sommaire

Component avec routes imbriquées

```vue
<!-- UserLayout.vue -->
<template>
  <div class="user-layout">
    <aside>
      <RouterLink :to="`/user/${userId}`">Profil</RouterLink>
      <RouterLink :to="`/user/${userId}/posts`">Posts</RouterLink>
      <RouterLink :to="`/user/${userId}/settings`">Paramètres</RouterLink>
    </aside>

    <main>
      <RouterView />
    </main>
  </div>
</template>
```

Revenir au sommaire

Named Views (avancé)

Pour des layouts complexes avec plusieurs RouterView nommées

```
// Configuration
const routes = [{
  path: '/',
  components: {
    default: Home,
    sidebar: Sidebar,
    footer: Footer
  }
}];
```

```
<!-- Utilisation -->
<template>
  <RouterView name="sidebar" />
  <RouterView /> <!-- default -->
  <RouterView name="footer" />
</template>
```

Revenir au sommaire

Guards de navigation - beforeEach

🚦 Les Guards sont comme des feux de circulation

Imaginez un **videur de boîte de nuit** :

- Il vérifie votre carte d'identité 🪪
- Si vous avez l'âge requis → **next()** (vous entrez ✅)
- Sinon → **next('/home')** (vous dégagez ❌)

🛡️ Types de guards

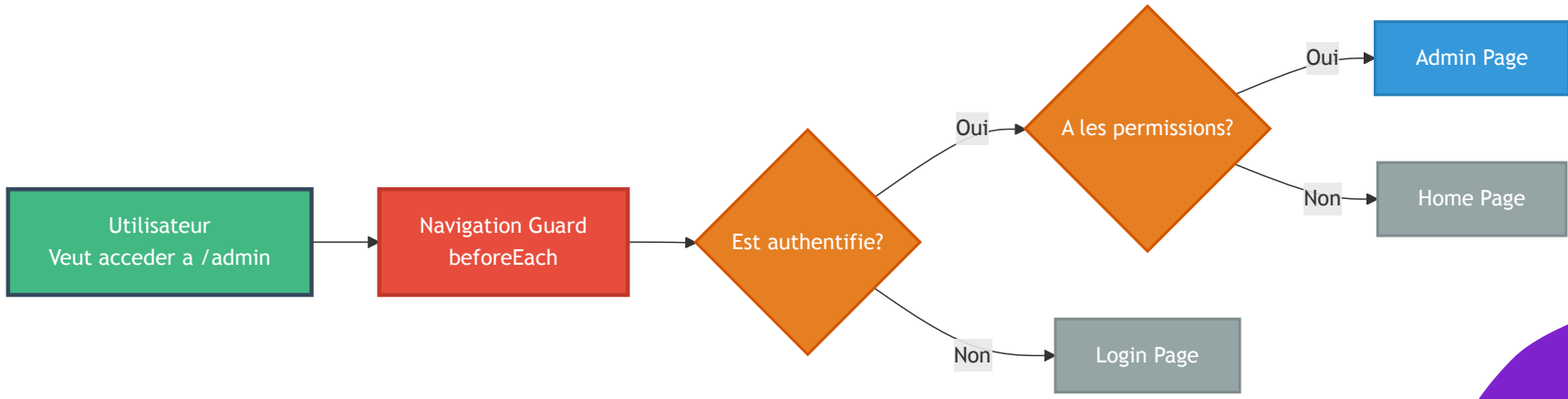
beforeEach : Garde global (vérifie TOUT le monde) **beforeEnter** : Garde spécifique (juste pour une route) **beforeRouteEnter** : Garde dans le composant

💡 Cas d'usage typiques

- 🔐 **Authentification** : "Es-tu connecté ?"
- 👮 **Autorisation** : "As-tu le droit d'être admin ?"
- 💾 **Sauvegarde** : "As-tu sauvegardé tes modifications ?"
- 📊 **Analytics** : Tracker les visites de pages

Revenir au sommaire

Schéma : Navigation Guards



Revenir au sommaire

Exemple de Guard

```
router.beforeEach((to, from, next) => {
  const isAuthenticated = checkAuth();

  // Si la route nécessite l'auth ET que l'user n'est pas connecté
  if (to.meta.requiresAuth && !isAuthenticated) {
    next('/login'); // ❌ Redirection vers login
  } else {
    next(); // ✅ Passe ton chemin, tu es bon !
  }
});
```



Revenir au sommaire

Guards par route

```
const routes = [
  {
    path: '/admin',
    component: Admin,
    beforeEnter: (to, from) => {
      if (!isAdmin()) {
        return false;
      }
    }
  }
];
```



Revenir au sommaire

🎯 Exercice TaskFlow : Navigation Guards

Protégeons TaskFlow avec des **guards d'authentification** !

📝 Objectif

Empêcher l'accès aux routes protégées sans connexion

Revenir au sommaire

🎯 TaskFlow Router - Configuration (1/2)

Créer router/index.js

```
import { createRouter, createWebHistory } from 'vue-router';
import { useAuthStore } from '../stores/auth';

const routes = [
  {
    path: '/',
    name: 'Home',
    component: () => import('../views/HomeView.vue'),
    meta: { title: 'Accueil - TaskFlow' }
  },
  {
    path: '/tasks',
    name: 'Tasks',
    component: () => import('../views/TaskListView.vue'),
    meta: {
      requiresAuth: true,
      title: 'Mes tâches - TaskFlow'
    }
  },
  {
    path: '/task/:id',
    name: 'TaskDetail',
    component: () => import('../views/TaskDetailView.vue'),
    meta: { requiresAuth: true }
  },
```

Revenir au sommaire

🎯 TaskFlow Router - Configuration (2/2)

```
  {
    path: '/dashboard',
    name: 'Dashboard',
    component: () => import('../views/DashboardView.vue'),
    meta: {
      requiresAuth: true,
      requiresAdmin: true,
      title: 'Dashboard - TaskFlow'
    }
  },
  {
    path: '/login',
    name: 'Login',
    component: () => import('../views/LoginView.vue'),
    meta: { title: 'Connexion - TaskFlow' }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: () => import('../views/NotFoundView.vue')
  }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});
```



Revenir au sommaire

🎯 TaskFlow Router - Guards

Guard global beforeEach

```
router.beforeEach((to, from, next) => {
  const authStore = useAuthStore();

  // 1. Changer le titre de la page
  document.title = to.meta.title || 'TaskFlow';

  // 2. Vérifier l'authentification
  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    next({
      name: 'Login',
      query: { redirect: to.fullPath }
    });
    return;
  }

  // 3. Vérifier les droits admin
  if (to.meta.requiresAdmin && !authStore.canCreateCategories) {
    alert('Accès réservé aux administrateurs');
    next({ name: 'Home' });
    return;
  }

  next();
});

export default router;
```

💡 N'oubliez pas

Créer `useAuthStore()` dans Module 4 avec `isAuthenticated` et `canCreateCategories` !

✅ Correction

👉 `taskflow-app/src/router/index.js`

Revenir au sommaire

🎯 Exercice TaskFlow : TaskListView.vue

Créons la vue principale qui affiche la liste des tâches !

📝 Objectif

Une vue complète qui utilise le **store Pinia** et le **router**

Revenir au sommaire

🎯 TaskListView - Spécifications

```vue
<script>
import { useTasksStore } from '@/stores/tasks';
import { useCategoriesStore } from '@/stores/categories';
import TaskList from '@/components/TaskList.vue';
import TaskForm from '@/components/TaskForm.vue';

export default {
  name: 'TaskListView',
  components: {
    TaskList,
    TaskForm
  },
  computed: {
    tasksStore() {
      return useTasksStore();
    },
    categoriesStore() {
      return useCategoriesStore();
    }
  },
  methods: {
    handleAddTask(taskData) {
      this.tasksStore.addTask(taskData);
    },
    handleToggleTask(id) {
      this.tasksStore.toggleTask(id);
    },
    handleDeleteTask(id) {
      this.tasksStore.deleteTask(id);
    },
    goToTaskDetail(id) {
      this.$router.push({ name: 'TaskDetail', params: { id } });
    }
  },
  mounted() {
    this.tasksStore.loadFromStorage();
  }
}
</script>

<template>
  <div class="task-list-view">
    <h1>Mes Tâches</h1>

    <TaskForm @add-task="handleAddTask" />

    <TaskList
      :tasks="tasksStore.tasks"
      @toggle="handleToggleTask"
      @delete="handleDeleteTask"
      @task-click="goToTaskDetail"
    />
  </div>
</template>
```

Revenir au sommaire

✅ Correction

👉 `taskflow-app/src/views/TaskListView.vue`

Revenir au sommaire

🎯 Exercice TaskFlow : TaskDetailView.vue

Créons la vue de détail qui utilise les **params dynamiques** !

📝 Objectif

Afficher et éditer une tâche spécifique via son ID dans l'URL

Revenir au sommaire

🎯 TaskDetailView - Spécifications

```vue
<script>
import { useTasksStore } from '@/stores/tasks';

export default {
  name: 'TaskDetailView',
  data() {
    return {
      task: null,
      isEditing: false
    }
  },
  computed: {
    tasksStore() {
      return useTasksStore();
    },
    taskId() {
      return parseInt(this.$route.params.id);
    }
  },
  methods: {
    loadTask() {
      this.task = this.tasksStore.getTaskById(this.taskId);
      if (!this.task) {
        this.$router.push({ name: 'NotFound' });
      }
    },
    saveTask() {
      this.tasksStore.updateTask(this.taskId, this.task);
      this.isEditing = false;
    },
    goBack() {
      this.$router.back();
    }
  },
  mounted() {
    this.loadTask();
  },
  beforeRouteUpdate(to, from) {
    // Recharger si on change l'ID dans l'URL
    if (to.params.id !== from.params.id) {
      this.loadTask();
    }
  }
}
</script>

<template>
  <div v-if="task" class="task-detail">
    <button @click="goBack">← Retour</button>

    <h1>{{ task.title }}</h1>

    <div v-if="!isEditing">
      <p>{{ task.description }}</p>
      <p>Priorité: {{ task.priority }}</p>
      <button @click="isEditing = true">Éditer</button>
    </div>

    <form v-else @submit.prevent="saveTask">
      <input v-model="task.title" required />
      <textarea v-model="task.description"></textarea>
      <button type="submit">Sauvegarder</button>
      <button type="button" @click="isEditing = false">Annuler</button>
    </form>
  </div>
</template>
```

Revenir au sommaire

✅ Correction

👉 `taskflow-app/src/views/TaskDetailView.vue`

Revenir au sommaire

🎯 Exercice TaskFlow : LoginView.vue

Créons la vue de connexion avec **redirection** !

📝 Objectif

Formulaire de connexion qui redirige vers la page d'origine après login

Revenir au sommaire

🎯 LoginView - Spécifications

```vue
<script>
import { useAuthStore } from '@/stores/auth';

export default {
  name: 'LoginView',
  data() {
    return {
      credentials: {
        email: '',
        password:''
      },
      error: null
    }
  },
  computed: {
    authStore() {
      return useAuthStore();
    },
    redirectPath() {
      return this.$route.query.redirect || '/tasks';
    }
  },
  methods: {
    async handleLogin() {
      try {
        this.error = null;
        await this.authStore.login(this.credentials);

        // Redirection vers la page d'origine ou /tasks
        this.$router.push(this.redirectPath);
      } catch (err) {
        this.error = 'Identifiants incorrects';
      }
    }
  }
}
</script>

<template>
  <div class="login-view">
    <h1>Connexion</h1>

    <form @submit.prevent="handleLogin">
      <div>
        <label>Email</label>
        <input
          v-model="credentials.email"
          type="email"
          required
          placeholder="votre@email.com"
        />
      </div>

      <div>
        <label>Mot de passe</label>
        <input
          v-model="credentials.password"
          type="password"
          required
          placeholder="••••••••"
        />
      </div>

      <p v-if="error" class="error">{{ error }}</p>

      <button type="submit">Se connecter</button>
    </form>
  </div>
</template>
```

Revenir au sommaire

💡 Astuce

Le `query.redirect` est défini par le guard `beforeEach` dans le router !

✅ Correction

👉 `taskflow-app/src/views/LoginView.vue`

Revenir au sommaire

🎯 Exercice TaskFlow : NotFoundView.vue

Créons la page 404 pour les routes invalides !

📝 Objectif

Une page d'erreur élégante avec redirection vers l'accueil

Revenir au sommaire

🎯 NotFoundView - Spécifications

```vue
<script>
export default {
  name: 'NotFoundView',
  methods: {
    goHome() {
      this.$router.push('/');
    }
  }
}
</script>

<template>
  <div class="not-found">
    <h1>404</h1>
    <h2>Page non trouvée</h2>
    <p>La page que vous recherchez n'existe pas.</p>

    <button @click="goHome">Retour à l'accueil</button>

    <p class="url-info">
      URL demandée : <code>{{ $route.fullPath }}</code>
    </p>
  </div>
</template>

<style scoped>
.not-found {
  text-align: center;
  padding: 4rem 2rem;
}

.not-found h1 {
  font-size: 8rem;
  font-weight: bold;
  color: #ef4444;
}

.url-info {
  margin-top: 2rem;
  color: #64748b;
  font-size: 0.875rem;
}

code {
  background: #f1f5f9;
  padding: 0.25rem 0.5rem;
  border-radius: 0.25rem;
  font-family: monospace;
}
</style>
```

Revenir au sommaire

💡 Route à ajouter

N'oubliez pas d'ajouter la route catch-all dans `router/index.js` :

```
{
  path: '/:pathMatch(.*)*',
  name: 'NotFound',
  component: () => import('../views/NotFoundView.vue')
}
```

✅ Correction

👉 `taskflow-app/src/views/NotFoundView.vue`

Revenir au sommaire

Guards dans le composant

```
<script>
export default {
  methods: {
    async fetchUser(id) {
      // Logique de récupération utilisateur
    }
  },
  beforeRouteLeave(to, from) {
    const answer = window.confirm(
      'Voulez-vous vraiment quitter ? Les modifications non sauvegardées seront perdues.'
    );

    if (!answer) return false;
  },
  async beforeRouteUpdate(to, from) {
    if (to.params.id !== from.params.id) {
      await this.fetchUser(to.params.id);
    }
  }
}
</script>
```



Revenir au sommaire

Redirections

```
const routes = [
  {
    path: '/home',
    redirect: '/'
  },
  {
    path: '/old-path',
    redirect: { name: 'new-route' }
  },
  {
    path: '/search/:searchText',
    redirect: to => {
      return { path: '/search', query: { q: to.params.searchText } };
    }
  }
];
```



Revenir au sommaire

Route 404

```
const routes = [
  // ... autres routes
  {
    path: '/:pathMatch(.*)*',
    name: 'not-found',
    component: NotFound
  }
];
```



Revenir au sommaire

Meta fields

```
const routes = [
  {
    path: '/admin',
    component: Admin,
    meta: {
      requiresAuth: true,
      roles: ['admin'],
      title: 'Administration'
    }
  }
];

router.beforeEach((to) => {
  document.title = to.meta.title || 'Mon App';
});
```



Revenir au sommaire

Mode history vs hash

```
import { createRouter, createWebHistory, createWebHashHistory } from 'vue-router';

// Mode History (recommandé)
// URL: https://example.com/user/123
const router = createRouter({
  history: createWebHistory(),
  routes
});

// Mode Hash
// URL: https://example.com/#/user/123
const router = createRouter({
  history: createWebHashHistory(),
  routes
});
```



Revenir au sommaire

Résumé Module 5

- ✅ Installation et configuration Vue Router
- ✅ Routes basiques et dynamiques
- ✅ Navigation programmatique
- ✅ Lazy loading
- ✅ Routes imbriquées
- ✅ Guards de navigation
- ✅ Redirections et 404
- ✅ Meta fields
- ✅ Modes history et hash

Revenir au sommaire

🎯 Bonus TaskFlow : HomePage.vue

Créons une page d'accueil élégante !

```vue
<template>
  <div class="home-page">
    <div class="hero">
      <h1>🚀 Bienvenue sur TaskFlow</h1>
      <p>Gérez vos tâches efficacement avec Vue.js 3</p>

      <div class="hero-actions">
        <RouterLink to="/tasks" class="btn-primary">
          Voir mes tâches
        </RouterLink>
        <RouterLink to="/login" class="btn-secondary">
          Se connecter
        </RouterLink>
      </div>
    </div>

    <div class="features">
      <div class="feature-card">
        <div class="feature-icon">✅</div>
        <h3>Gestion simple</h3>
        <p>Créez, modifiez et supprimez vos tâches facilement</p>
      </div>
      <!-- + 2 autres feature cards -->
    </div>
  </div>
</template>
```

✅ taskflow-app/src/views/HomePage.vue

Revenir au sommaire

Travaux pratiques

Vous avez maintenant TOUS les éléments pour reproduire TaskFlow :

- ✅ **Components** : TaskItem, TaskForm, TaskList, TaskCard, BoardColumn, TaskModal
- ✅ **Stores** : tasks, auth, categories
- ✅ **Router** : Guards, routes dynamiques, lazy loading
- ✅ **Vues** : TaskListView, TaskDetailView, LoginView, NotFoundView, HomePage

🎯 Application complète disponible

👉 Voir `taskflow-app/` pour la correction complète

Revenir au sommaire

Exercices Pratiques

Module 5 - Vue Router

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 5

Maintenant que vous avez vu tous les concepts théoriques du Module 5, il est temps d'ajouter la navigation et le routing à TaskFlow !

Rappel des concepts du Module 5

5.1 Configuration : createRouter, routes, history mode
5.2 Navigation : RouterLink, RouterView, programmatic navigation
5.3 Routes dynamiques : Paramètres, props
5.4 Guards : beforeEach, beforeEnter, onBeforeRouteLeave
5.5 Meta fields : requiresAuth, title

Les 7 exercices progressifs

Nous allons créer un système de navigation complet pour TaskFlow.

Revenir au sommaire

Exercice 1 : Configuration Router

Objectif

Configurer Vue Router pour TaskFlow

Routes à créer

```
/ (Home)
/tasks (Liste des tâches)
/task/:id (Détail d'une tâche)
/categories (Catégories)
/login (Connexion)
/dashboard (Protégé - Admin)
/404 (Not Found)
```

Installation

```
npm install vue-router@4
```

Revenir au sommaire

Exercice 1 - Solution router/index.js (1/2)

```
// router/index.js
import { createRouter, createWebHistory } from 'vue-router';

const routes = [
  {
    path: '/',
    name: 'Home',
    component: () => import('../views/HomePage.vue'),
    meta: {
      title: 'TaskFlow - Accueil',
      requiresAuth: false
    }
  },
  {
    path: '/tasks',
    name: 'TaskList',
    component: () => import('../views/TaskListView.vue'),
    meta: {
      title: 'Mes Tâches',
      requiresAuth: true
    }
  },
  {
    path: '/task/:id',
    name: 'TaskDetail',
    component: () => import('../views/TaskDetailView.vue'),
    meta: {
      title: 'Détail de la tâche',
      requiresAuth: true
    },
    props: true
  },
  {
    path: '/categories',
    name: 'Categories',
    component: () => import('../views/CategoriesView.vue'),
    meta: {
      title: 'Catégories',
      requiresAuth: true
    }
  }
];
```

Revenir au sommaire

Exercice 1 - Solution router/index.js (2/2)

```
// router/index.js (suite)
const routes = [
  // ... routes précédentes

  {
    path: '/login',
    name: 'Login',
    component: () => import('../views/LoginView.vue'),
    meta: {
      title: 'Connexion',
      requiresAuth: false
    }
  },
  {
    path: '/dashboard',
    name: 'Dashboard',
    component: () => import('../views/DashboardView.vue'),
    meta: {
      title: 'Dashboard Admin',
      requiresAuth: true,
      requiresAdmin: true
    }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: () => import('../views/NotFoundView.vue'),
    meta: {
      title: 'Page non trouvée'
    }
  }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```



Revenir au sommaire

Exercice 1 - main.js avec Router

```
// main.js
import { createApp } from 'vue';
import { createPinia } from 'pinia';
import App from './App.vue';
import router from './router';
import './styles/main.css';

const app = createApp(App);
const pinia = createPinia();

app.use(pinia);
app.use(router);

app.mount('#app');
```



Revenir au sommaire

Exercice 1 - App.vue avec Navigation

```vue
<script>
import { useAuthStore } from '@/stores/auth';

export default {
  name: 'App',
  computed: {
    // Style Options API moderne : appel direct du store
    authStore() {
      return useAuthStore();
    }
  },
  mounted() {
    this.authStore.checkAuth();
  }
}
</script>

<template>
  <div id="app">
    <nav class="main-nav">
      <div class="nav-brand">
        <RouterLink to="/">
          📋 TaskFlow
        </RouterLink>
      </div>

      <div class="nav-links">
        <RouterLink to="/">Accueil</RouterLink>
        <RouterLink to="/tasks">Mes Tâches</RouterLink>
        <RouterLink to="/categories">Catégories</RouterLink>

        <RouterLink
          v-if="authStore.canCreateCategories"
          to="/dashboard"
        >
          Dashboard
        </RouterLink>
      </div>

      <div class="nav-user">
        <span v-if="authStore.isAuthenticated">
          👤 {{ authStore.userDisplayName }}
        </span>
        <RouterLink v-else to="/login">
          Connexion
        </RouterLink>
      </div>
    </nav>

    <main class="main-content">
      <RouterView />
    </main>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 : Guards de Navigation

Objectif

Protéger les routes avec des guards

Spécifications

Guard global :

- Vérifier l'authentification
- Rediriger vers `/login` si non authentifié
- Rediriger les admins uniquement vers `/dashboard`

Guard dans composant :

- `onBeforeRouteLeave` : Confirmer avant de quitter une page d'édition

Revenir au sommaire

Exercice 2 - Solution Navigation Guards

```
// router/index.js (avec guards)
import { createRouter, createWebHistory } from 'vue-router';
import { useAuthStore } from '@/stores/auth';

// ... définition des routes

const router = createRouter({
  history: createWebHistory(),
  routes
});

// ✅ GUARD GLOBAL
router.beforeEach((to, from, next) => {
  const authStore = useAuthStore();

  // Changer le titre de la page
  document.title = to.meta.title || 'TaskFlow';

  // Vérifier si la route nécessite une authentification
  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    // Rediriger vers login avec query pour revenir après
    next({
      name: 'Login',
      query: { redirect: to.fullPath }
    });
    return;
  }

  // Vérifier si la route nécessite les droits admin
  if (to.meta.requiresAdmin && !authStore.canCreateCategories) {
    alert('Accès réservé aux administrateurs');
    next({ name: 'Home' });
    return;
  }

  // Si déjà connecté et tente d'aller sur /login
  if (to.name === 'Login' && authStore.isAuthenticated) {
    next({ name: 'TaskList' });
    return;
  }

  next();
});

export default router;
```

Revenir au sommaire

Exercice 2 - Solution onBeforeRouteLeave

```
<script>
export default {
  name: 'TaskEditView',
  data() {
    return {
      hasUnsavedChanges: false
    };
  },
  // ✅ GUARD dans le composant avec Options API
  beforeRouteLeave(to, from, next) {
    if (this.hasUnsavedChanges) {
      const answer = window.confirm(
        'Vous avez des modifications non sauvegardées. Voulez-vous vraiment quitter ?'
      );

      if (answer) {
        next();
      } else {
        next(false);
      }
    } else {
      next();
    }
  }
}
</script>

<template>
  <div>
    <h1>Éditer la tâche</h1>

    <form @input="hasUnsavedChanges = true">
      <!-- Formulaire -->
    </form>
  </div>
</template>
```

Revenir au sommaire

Exercice 3 : TaskDetailView

Objectif

Créer la vue de détail d'une tâche

Spécifications

Route : `/task/:id`

Fonctionnalités :

- Charger la tâche depuis le store
- Afficher tous les détails
- Modifier en ligne
- Bouton supprimer
- Breadcrumb de navigation
- Bouton retour

Revenir au sommaire

Exercice 3 - Solution TaskDetailView.vue (1/4)

```vue
<script>
import { useTasksStore } from '@/stores/tasks';
import { useCategoriesStore } from '@/stores/categories';

export default {
  name: 'TaskDetailView',
  data() {
    return {
      isEditing: false,
      editForm: {}
    };
  },
  computed: {
    // Style Options API moderne : appel direct des stores
    tasksStore() {
      return useTasksStore();
    },
    categoriesStore() {
      return useCategoriesStore();
    },
    // ✅ Récupérer l'ID depuis la route
    taskId() {
      return parseInt(this.$route.params.id);
    },
    task() {
      return this.tasksStore.getTaskById(this.taskId);
    },
    category() {
      if (!this.task?.categoryId) return null;
      return this.categoriesStore.categoryById(this.task.categoryId);
    }
  },
  mounted() {
    // Si tâche inexistante, rediriger vers 404
    if (!this.task) {
      this.$router.push({ name: 'NotFound' });
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 3 - Solution TaskDetailView.vue (2/4)

Methods :

```
<script>
export default {
  // ... setup et computed précédents

  methods: {
    startEdit() {
      this.editForm = { ...this.task };
      this.isEditing = true;
    },

    saveEdit() {
      this.tasksStore.updateTask(this.taskId, this.editForm);
      this.isEditing = false;
    },

    cancelEdit() {
      this.isEditing = false;
    },

    async deleteTask() {
      if (confirm('Supprimer cette tâche ?')) {
        this.tasksStore.deleteTask(this.taskId);
        this.$router.push({ name: 'TaskList' });
      }
    },

    goBack() {
      this.$router.back();
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 3 - Solution TaskDetailView.vue (3/4)

Template - Header et Édition :

```vue
<template>
  <div class="task-detail" v-if="task">
    <!-- Breadcrumb -->
    <nav class="breadcrumb">
      <RouterLink to="/">Accueil</RouterLink>
      <span>/</span>
      <RouterLink to="/tasks">Tâches</RouterLink>
      <span>/</span>
      <span>{{ task.title }}</span>
    </nav>

    <!-- En-tête -->
    <div class="task-header">
      <h1>{{ task.title }}</h1>

      <div class="task-actions">
        <button @click="startEdit" v-if="!isEditing">
          ✏️ Modifier
        </button>
        <button @click="deleteTask" class="btn-danger">
          🗑️ Supprimer
        </button>
        <button @click="goBack" class="btn-secondary">
          ← Retour
        </button>
      </div>
    </div>

    <!-- Mode édition -->
    <div v-if="isEditing" class="edit-mode">
      <div class="form-group">
        <label>Titre</label>
        <input v-model="editForm.title" />
      </div>

      <div class="form-group">
        <label>Description</label>
        <textarea v-model="editForm.description"></textarea>
      </div>

      <div class="form-group">
        <label>Priorité</label>
        <select v-model="editForm.priority">
          <option value="low">Basse</option>
          <option value="medium">Moyenne</option>
          <option value="high">Haute</option>
          <option value="urgent">URGENT</option>
        </select>
      </div>

      <div class="form-actions">
        <button @click="saveEdit" class="btn-primary">
          💾 Sauvegarder
        </button>
        <button @click="cancelEdit" class="btn-secondary">
          Annuler
        </button>
      </div>
    </div>
```

Revenir au sommaire

Exercice 3 - Solution TaskDetailView.vue (4/4)

Template - Mode Lecture :

```
<template>
  <div class="task-detail" v-if="task">
    <!-- ... header et mode édition précédents -->

    <!-- Mode lecture -->
    <div v-else class="view-mode">
      <div class="task-meta">
        <div class="meta-item">
          <strong>Statut :</strong>
          <span :class="task.done ? 'status-done' : 'status-active'">
            {{ task.done ? '✅ Terminée' : '⏳ En cours' }}
          </span>
        </div>

        <div class="meta-item">
          <strong>Priorité :</strong>
          <span :class="`priority-${task.priority}`">
            {{ task.priority }}
          </span>
        </div>

        <div class="meta-item" v-if="category">
          <strong>Catégorie :</strong>
          <span :style="{ color: category.color }">
            {{ category.icon }} {{ category.name }}
          </span>
        </div>

        <div class="meta-item">
          <strong>Créée le :</strong>
          {{ new Date(task.createdAt).toLocaleDateString('fr-FR') }}
        </div>

        <div class="meta-item" v-if="task.completedAt">
          <strong>Terminée le :</strong>
          {{ new Date(task.completedAt).toLocaleDateString('fr-FR') }}
        </div>
      </div>

      <div class="task-content">
        <h2>Description</h2>
        <p>{{ task.description || 'Aucune description' }}</p>
      </div>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 4 : LoginView avec Redirect

Objectif

Gérer la connexion avec redirection après login

Spécifications

Route : `/login`

Fonctionnalités :

- Formulaire de connexion
- Appeler `authStore.login()`
- Rediriger vers la page d'origine (`query.redirect`)
- Sinon rediriger vers `/tasks`

Revenir au sommaire

Exercice 4 - Solution LoginView.vue (1/2)

Script :

```
<script>
import { useAuthStore } from '@/stores/auth';

export default {
  name: 'LoginView',
  data() {
    return {
      form: {
        email: '',
        password:''
      },
      error: null,
      loading: false
    };
  },
  computed: {
    // Style Options API moderne : appel direct du store
    authStore() {
      return useAuthStore();
    }
  },
  methods: {
    async handleLogin() {
      this.error = null;
      this.loading = true;

      try {
        await this.authStore.login(this.form.email, this.form.password);

        // ✅ Rediriger vers l'URL d'origine ou /tasks
        const redirect = this.$route.query.redirect || '/tasks';
        this.$router.push(redirect);

      } catch (err) {
        this.error = 'Email ou mot de passe incorrect';
      } finally {
        this.loading = false;
      }
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 4 - Solution LoginView.vue (2/2)

Template :

```vue
<template>
  <div class="login-view">
    <div class="login-card">
      <h1>📋 TaskFlow</h1>
      <h2>Connexion</h2>

      <form @submit.prevent="handleLogin">
        <div class="form-group">
          <label>Email</label>
          <input
            v-model="form.email"
            type="email"
            required
            placeholder="admin@taskflow.com"
          />
        </div>

        <div class="form-group">
          <label>Mot de passe</label>
          <input
            v-model="form.password"
            type="password"
            required
            placeholder="••••••••"
          />
        </div>

        <div v-if="error" class="error">
          {{ error }}
        </div>

        <button
          type="submit"
          :disabled="loading"
          class="btn-primary"
        >
          {{ loading ? 'Connexion...' : 'Se connecter' }}
        </button>
      </form>

      <p class="demo-info">
        💡 Utilisez <code>admin@taskflow.com</code> pour tester les permissions admin
      </p>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 5 : TaskListView.vue

Objectif

Créer la vue principale qui affiche toutes les tâches

Spécifications

Imports :

- `useTasksStore` , `useCategoriesStore`
- Composants : `TaskForm` , `TaskList`

Fonctionnalités :

- Afficher la liste des tâches
- Ajouter une tâche
- Toggle tâche terminée
- Supprimer une tâche
- Cliquer sur une tâche pour voir le détail

Revenir au sommaire

Exercice 5 - Solution TaskListView.vue

```vue
<script>
import { useTasksStore } from '@/stores/tasks';
import { useCategoriesStore } from '@/stores/categories';
import TaskList from '@/components/TaskList.vue';
import TaskForm from '@/components/TaskForm.vue';

export default {
  name: 'TaskListView',
  components: { TaskList, TaskForm },
  computed: {
    tasksStore() { return useTasksStore(); },
    categoriesStore() { return useCategoriesStore(); }
  },
  methods: {
    handleAddTask(taskData) {
      this.tasksStore.addTask(taskData);
    },
    handleToggleTask(id) {
      this.tasksStore.toggleTask(id);
    },
    handleDeleteTask(id) {
      this.tasksStore.deleteTask(id);
    },
    goToTaskDetail(id) {
      this.$router.push({ name: 'TaskDetail', params: { id } });
    }
  },
  mounted() {
    this.tasksStore.fetchTasks();
  }
}
</script>

<template>
  <div class="task-list-view">
    <h1>Mes Tâches</h1>

    <TaskForm @add-task="handleAddTask" />

    <TaskList
      :tasks="tasksStore.filteredTasks"
      @toggle="handleToggleTask"
      @delete="handleDeleteTask"
      @task-click="goToTaskDetail"
    />
  </div>
</template>
```

Revenir au sommaire

Exercice 6 : NotFoundView.vue

Objectif

Créer une page 404 élégante

Spécifications

- Grand titre "404"
- Message "Page non trouvée"
- Afficher l'URL demandée : `{{ $route.fullPath }}`
- Bouton "Retour à l'accueil" : `this.$router.push('/')`

Revenir au sommaire

Exercice 6 - Solution NotFoundView.vue

```vue
<script>
export default {
  name: 'NotFoundView',
  methods: {
    goHome() {
      this.$router.push('/');
    }
  }
}
</script>

<template>
  <div class="not-found">
    <h1>404</h1>
    <h2>Page non trouvée</h2>
    <p>La page que vous recherchez n'existe pas.</p>

    <div class="url-info">
      URL demandée : <code>{{ $route.fullPath }}</code>
    </div>

    <button @click="goHome" class="btn-primary">
      Retour à l'accueil
    </button>
  </div>
</template>

<style scoped>
.not-found {
  text-align: center;
  padding: 4rem 2rem;
}

.not-found h1 {
  font-size: 8rem;
  font-weight: bold;
  color: #ff4444;
  margin: 0;
}

.not-found h2 {
  font-size: 2rem;
  color: #64748b;
  margin: 1rem 0;
}

.url-info {
  margin: 2rem 0;
  color: #64748b;
  font-size: 0.875rem;
}

code {
  background: #f1f5f9;
  padding: 0.25rem 0.5rem;
  border-radius: 0.25rem;
  font-family: monospace;
}
</style>
```

Revenir au sommaire

Exercice 7 : HomePage.vue

Objectif

Créer une page d'accueil attractive

Spécifications

- Hero avec titre + description
- Boutons : `<RouterLink to="/tasks">` et `<RouterLink to="/login">`
- 3 feature cards (avec icônes emoji)

Revenir au sommaire

Exercice 7 - Solution HomePage.vue (1/2)

Script et Template Hero :

```
<script>
export default {
  name: 'HomePage'
}
</script>

<template>
  <div class="home-page">
    <div class="hero">
      <h1>🚀 Bienvenue sur TaskFlow</h1>
      <p class="hero-subtitle">
        Gérez vos tâches efficacement avec Vue.js 3
      </p>

      <div class="hero-actions">
        <RouterLink to="/tasks" class="btn-primary">
          Voir mes tâches
        </RouterLink>
        <RouterLink to="/login" class="btn-secondary">
          Se connecter
        </RouterLink>
      </div>
    </div>
  </div>
</template>
```

Revenir au sommaire

Exercice 7 - Solution HomePage.vue (2/2)

Template Features :

```
<template>
  <div class="home-page">
    <!-- ... hero précédent -->

    <div class="features">
      <div class="feature-card">
        <div class="feature-icon">✅</div>
        <h3>Gestion simple</h3>
        <p>Créez, modifiez et supprimez vos tâches facilement</p>
      </div>

      <div class="feature-card">
        <div class="feature-icon">🎯</div>
        <h3>Priorisez</h3>
        <p>Définissez des priorités pour rester organisé</p>
      </div>

      <div class="feature-card">
        <div class="feature-icon">🏷️</div>
        <h3>Catégorisez</h3>
        <p>Organisez vos tâches par catégories</p>
      </div>
    </div>
  </div>
</template>
```

Styles : Grid layout, hover effects, responsive

Revenir au sommaire

🎯 Récap Module 5 - TaskFlow

Ce que vous avez construit

✅ **Router config** : 7 routes avec lazy loading
✅ **Guards** : Protection auth + admin
✅ **TaskDetailView** : Vue détaillée avec édition
✅ **LoginView** : Connexion avec redirect
✅ **TaskListView** : Liste avec navigation
✅ **NotFoundView** : Page 404 élégante
✅ **HomePage** : Page d'accueil attractive

Concepts du Module 5 utilisés

- ✅ **5.1 Configuration** : createRouter, routes, history mode
- ✅ **5.2 Navigation** : RouterLink, RouterView, programmatic
- ✅ **5.3 Routes dynamiques** : :id params, props: true
- ✅ **5.4 Guards** : beforeEach, onBeforeRouteLeave
- ✅ **5.5 Meta fields** : requiresAuth, title



Revenir au sommaire

🚀 Prochaine étape : Module 6

On va **tester** TaskFlow avec **Vitest** !

- Tests unitaires des stores
- Tests des composants
- Tests des guards de navigation

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Vue Router Documentation
- 🎯 Navigation Guards
- 🚀 Lazy Loading

Concepts avancés

- 🔧 Nested Routes (routes imbriquées)
- 🔗 Named Views

Exemples

- 💡 Vue Router Examples
- 🎨 Advanced Routing Patterns



Revenir au sommaire

Module 5

State Management avec Pinia

Revenir au sommaire

SOMMAIRE DU MODULE

🏦 4.1 Introduction au State Management

🍍 4.2 Bases de Pinia

🚀 4.3 Pinia avancé

💪 Exercices Pratiques du Module 4

Revenir au sommaire

4.1 Introduction au State Management

Revenir au sommaire

Qu'est-ce que le State Management ?

Le **State Management** est la gestion centralisée de l'état de l'application

🏦 Analogie : C'est comme une banque centrale

Sans State Management, c'est comme si **chaque personne gardait son argent chez soi** :

- Difficile de savoir qui a combien 💰
- Impossible de faire des transferts facilement
- Chaos total si quelqu'un veut vérifier le total !

Avec State Management, **une banque centrale** :

- Tout l'argent au même endroit 🏦
- Transactions faciles et traçables
- Vue d'ensemble claire

Revenir au sommaire

Le problème : Props Drilling

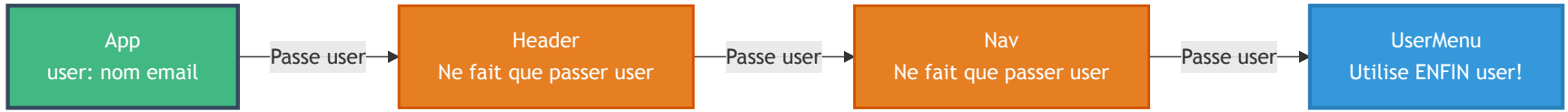
😱 Le cauchemar du Props Drilling

3 composants intermédiaires qui ne font que passer la donnée ! C'est lourd...

```
App (user)
  └── Header (user) ← Passe user
      └── Nav (user) ← Passe user
          └── UserMenu (user) ← Utilise ENFIN user !
```

Revenir au sommaire

Schéma : Props Drilling



Revenir au sommaire

La solution : Pinia !

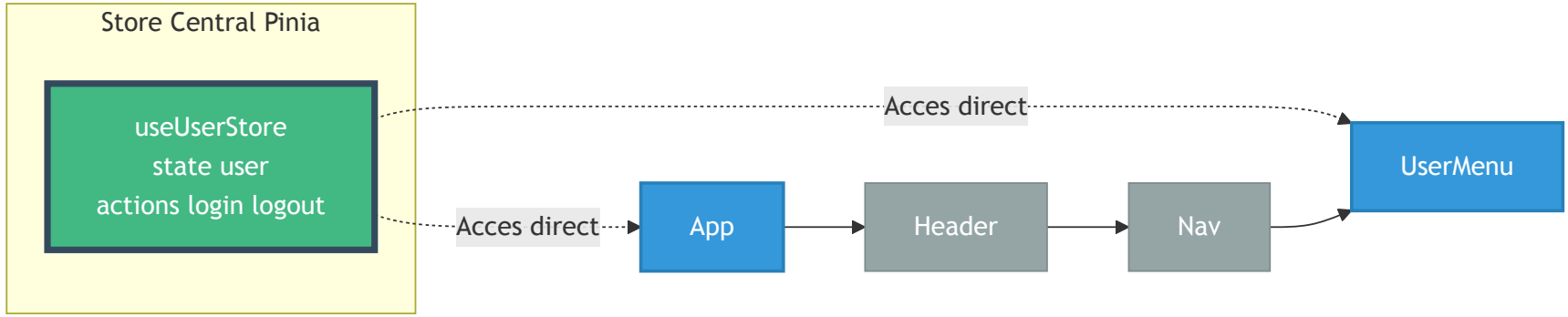
✨ Solution avec Pinia

```
App → utilise useUserStore()
Header → n'a pas besoin de user
Nav → n'a pas besoin de user
UserMenu → utilise useUserStore() ← Accès direct !
```

Chaque composant prend ce dont il a besoin, directement à la source !

Revenir au sommaire

Schéma : Solution avec Pinia



Revenir au sommaire

Pourquoi Pinia ?

Pinia est le gestionnaire d'état officiel pour Vue.js 3

Avantages de Pinia (2025)

- 🎯 Simple et intuitif
- ⚡ Léger (~ 1kb)
- 🔧 Support TypeScript natif
- 🛠️ DevTools intégrés
- 🌳 Tree-shaking automatique
- 🧩 Modular par design
- ✅ Composition API first

Revenir au sommaire

4.2 Bases de Pinia

Revenir au sommaire

Installation de Pinia

```
npm install pinia
```

Revenir au sommaire

Configuration dans main.js

```
// main.js
import { createApp } from 'vue';
import { createPinia } from 'pinia';
import App from './App.vue';

const app = createApp(App);
const pinia = createPinia();

app.use(pinia);
app.mount('#app');
```



Revenir au sommaire

Créer un Store simple

```
// stores/counter.js
import { defineStore } from 'pinia';

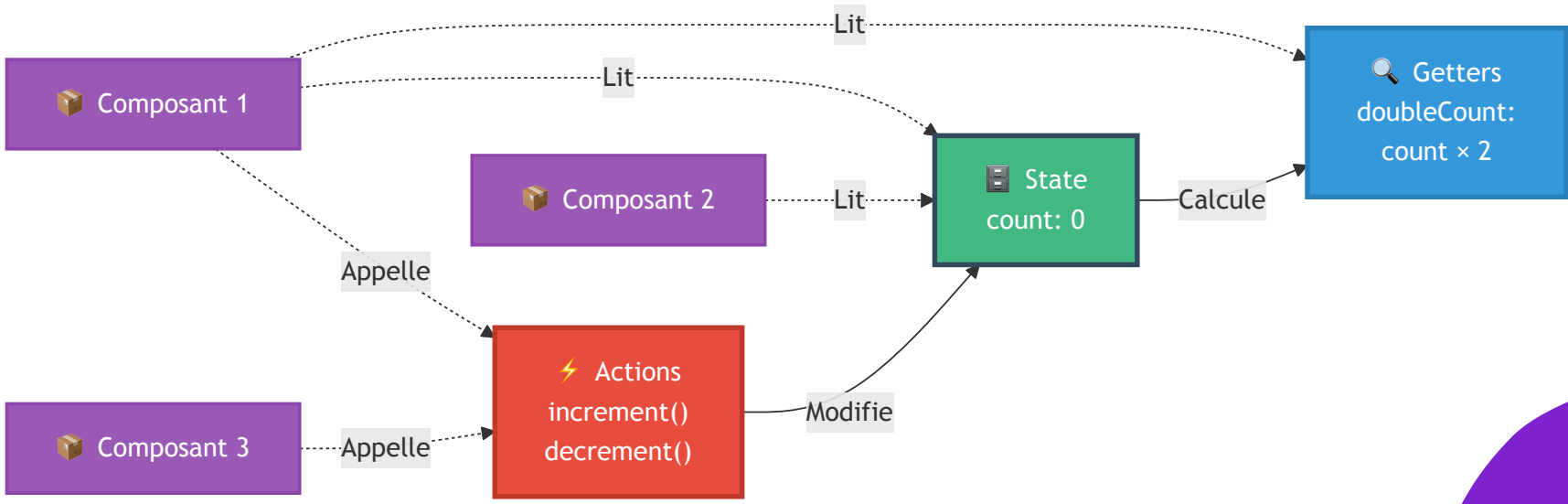
export const useCounterStore = defineStore('counter', {
  // State
  state: () => ({
    count: 0
  }),

  // Getters
  getters: {
    doubleCount: (state) => state.count * 2
  },

  // Actions
  actions: {
    increment() {
      this.count++;
    },
    decrement() {
      this.count--;
    }
  }
});
```

Revenir au sommaire

Schéma : Architecture d'un Store Pinia



Revenir au sommaire

Utiliser le Store avec Options API

```
<script>
import { useCounterStore } from '@/stores/counter';

export default {
  computed: {
    counter() {
      return useCounterStore();
    },
    count() {
      return this.counter.count;
    },
    doubleCount() {
      return this.counter.doubleCount;
    }
  }
}
</script>

<template>
  <div>
    <p>Count: {{ count }}</p>
    <p>Double: {{ doubleCount }}</p>

    <button @click="counter.increment">+</button>
    <button @click="counter.decrement">-</button>
  </div>
</template>
```



Revenir au sommaire

Store avec Options API syntax

```
// stores/user.js
import { defineStore } from 'pinia';

export const useUserStore = defineStore('user', {
  state: () => ({
    nom: '',
    email: '',
    isLoggedIn: false
  }),

  getters: {
    displayName: (state) => {
      return state.nom.toUpperCase();
    }
  },

  actions: {
    login(nom, email) {
      this.nom = nom;
      this.email = email;
      this.isLoggedIn = true;
    },

    logout() {
      this.nom = '';
      this.email = '';
      this.isLoggedIn = false;
    }
  }
});
```

Revenir au sommaire

State - Données réactives

Le state contient les données réactives du store - **Syntaxe Options recommandée**

```
import { defineStore } from 'pinia';

export const useProductStore = defineStore('products', {
  state: () => ({
    // State simple
    loading: false,
    error: null,

    // State tableau
    products: [],

    // State objet
    filters: {
      search: '',
      category: '',
      minPrice: 0,
    maxPrice: 1000
  });

  return { loading, error, products, filters };
});
```

Revenir au sommaire

Getters - Propriétés calculées

Les getters sont des propriétés calculées basées sur le state

🧮 Analogie : Les getters sont comme une calculatrice

Imaginez une boutique :

- **State** : `products` (liste brute des produits)
- **Getter** : `totalPrice` (calcule automatiquement le total)

Vous ne stockez PAS le total, vous le **calculez à la demande** !

💡 Pourquoi des getters ?

Sans getter : vous devez calculer partout

```
// Dans ComponentA
const total = products.reduce((sum, p) => sum + p.prix, 0);

// Dans ComponentB
const total = products.reduce((sum, p) => sum + p.prix, 0); // Duplication !
```

Avec getter : calculé une fois, utilisé partout

```
const total = store.totalPrice; // Simple et réutilisable !
```

Revenir au sommaire

Getters vs Redux selectors

VS Options API : la syntaxe recommandée pour cette formation

```
export const useProductStore = defineStore('products', {
  state: () => ({
    products: [],
    filters: {
      search: '',
      category: '',
    }
  }),

  getters: {
    // Getter simple
    productCount: (state) => state.products.length,

    // Getter filtré
    filteredProducts: (state) => {
      return state.products.filter(p => {
        const matchSearch = p.nom.toLowerCase()
          .includes(state.filters.search.toLowerCase());
        const matchCategory = !state.filters.category ||
                              p.category === state.filters.category;
        return matchSearch && matchCategory;
      });
    },

    // Getter avec calcul
    totalValue: (state) => {
      return state.products.reduce((sum, p) => sum + p.prix, 0);
    }
  }
});
```

Revenir au sommaire

Getters avec paramètres

```
export const useProductStore = defineStore('products', {
  state: () => ({
    products: []
  }),

  getters: {
    // Getter qui retourne une fonction
    getProductById: (state) => {
      return (id) => state.products.find(p => p.id === id);
    },

    getProductsByCategory: (state) => {
      return (category) => {
        return state.products.filter(p => p.category === category);
      };
    }
  }
});
```

Revenir au sommaire

Utilisation des getters

```
<script>
import { useProductStore } from '@/stores/products';

export default {
  computed: {
    store() {
      return useProductStore();
    },
    product() {
      return this.store.getProductById(42);
    },
    electronics() {
      return this.store.getProductsByCategory('electronics');
    }
  }
}
</script>

<template>
  <div>
    <p>Total: {{ store.productCount }} produits</p>
    <p>Valeur totale: {{ store.totalValue }}€</p>

    <div v-for="product in store.filteredProducts" :key="product.id">
      {{ product.nom }}
    </div>
  </div>
</template>
```

Revenir au sommaire

Actions - Méthodes

Les actions modifient le state

```
export const useProductStore = defineStore('products', {
  state: () => ({
    products: [],
    loading: false,
    error: null
  }),

  actions: {
    async fetchProducts() {
      this.loading = true;
      this.error = null;

      try {
        const response = await fetch('/api/products');
        const data = await response.json();
        this.products = data;
      } catch (e) {
        this.error = e.message;
      } finally {
        this.loading = false;
      }
    },

    addProduct(product) {
      this.products.push({
        ...product,
        id: Date.now()
      });
    }
  }
});
```

Revenir au sommaire

Actions avec mutation directe

Pinia permet la mutation directe du state

```
export const useCartStore = defineStore('cart', {
  state: () => ({
    items: []
  }),

  actions: {
    addToCart(product) {
      const existingItem = this.items.find(i => i.id === product.id);

      if (existingItem) {
        // Mutation directe !
        existingItem.quantity++;
      } else {
        this.items.push({
          ...product,
          quantity: 1
        });
      }
    },

    removeFromCart(productId) {
      const index = this.items.findIndex(i => i.id === productId);
      if (index > -1) {
        this.items.splice(index, 1);
      }
    },

    clearCart() {
      this.items = [];
    }
  }
});
```

Revenir au sommaire

Store avec données API

```
// stores/users.js
import { defineStore } from 'pinia';

export const useUsersStore = defineStore('users', {
  state: () => ({
    users: [],
    loading: false,
    error: null
  }),

  actions: {
    async fetchUsers() {
      this.loading = true;
      try {
        const res = await fetch('https://jsonplaceholder.typicode.com/users');
        this.users = await res.json();
      } catch (e) {
        this.error = e.message;
      } finally {
        this.loading = false;
      }
    },

    getUserById(id) {
      return this.users.find(u => u.id === id);
    },

    async createUser(userData) {
      try {
        const res = await fetch('https://jsonplaceholder.typicode.com/users', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(userData)
        });
        const newUser = await res.json();
        this.users.push(newUser);
        return newUser;
      } catch (e) {
        this.error = e.message;
        throw e;
      }
    }
  }
});
```

Revenir au sommaire

Utilisation dans un composant

```
<script>
import { useUsersStore } from '@/stores/users';

export default {
  computed: {
    usersStore() {
      return useUsersStore();
    }
  },
  methods: {
    async handleCreateUser() {
      try {
        await this.usersStore.createUser({
          nom: 'Alice',
          email: 'alice@example.com'
        });
      } catch (error) {
        console.error('Erreur:', error);
      }
    }
  },
  mounted() {
    this.usersStore.fetchUsers();
  }
}
</script>

<template>
  <div>
    <div v-if="usersStore.loading">Chargement...</div>
    <div v-else-if="usersStore.error">Erreur: {{ usersStore.error }}</div>
    <div v-else>
      <div v-for="user in usersStore.users" :key="user.id">
        {{ user.name }}
      </div>
    </div>

    <button @click="handleCreateUser">Créer</button>
  </div>
</template>
```

Revenir au sommaire

Accès au store avec Options API

```
<script>
import { useUserStore } from '@/stores/user';

export default {
  computed: {
    userStore() {
      return useUserStore();
    },
    nom() {
      return this.userStore.nom;
    },
    email() {
      return this.userStore.email;
    },
    isLoggedIn() {
      return this.userStore.isLoggedIn;
    }
  },
  methods: {
    logout() {
      this.userStore.logout();
    }
  }
}
</script>

<template>
  <div>
    <p v-if="isLoggedIn">Bonjour {{ nom }}</p>
    <button @click="logout">Déconnexion</button>
  </div>
</template>
```

Revenir au sommaire

4.3 Pinia avancé

Revenir au sommaire

Multiples stores

```
// stores/auth.js
export const useAuthStore = defineStore('auth', {
  state: () => ({
    token: '',
    user: null
  })
});

// stores/cart.js
export const useCartStore = defineStore('cart', {
  state: () => ({
    items: []
  })
});

// stores/products.js
export const useProductsStore = defineStore('products', {
  state: () => ({
    products: []
  })
});
```



Revenir au sommaire

Utiliser plusieurs stores ensemble

```
<script>
import { useAuthStore } from '@/stores/auth';
import { useCartStore } from '@/stores/cart';
import { useProductsStore } from '@/stores/products';

export default {
  computed: {
    auth() {
      return useAuthStore();
    },
    cart() {
      return useCartStore();
    },
    products() {
      return useProductsStore();
    }
  },
  methods: {
    addToCart(product) {
      if (this.auth.user) {
        this.cart.addToCart(product);
      } else {
        alert('Veuillez vous connecter');
      }
    }
  }
}
</script>
```



Revenir au sommaire

Communication entre stores

```
// stores/cart.js
import { useAuthStore } from './auth';
import { useProductsStore } from './products';

export const useCartStore = defineStore('cart', {
  state: () => ({
    items: []
  }),

  actions: {
    addToCart(productId) {
      const authStore = useAuthStore();
      const productsStore = useProductsStore();

      if (!authStore.isLoggedIn) {
        throw new Error('Non connecté');
      }

      const product = productsStore.getProductById(productId);
      if (product) {
        this.items.push(product);
      }
    }
  }
});
```

Revenir au sommaire

Persistence avec localStorage

💾 Sauvegarder les données entre les sessions

Avec l'**Options API**, on peut facilement persister le state dans localStorage !

Stratégie :

1. Charger depuis localStorage au démarrage
2. Sauvegarder après chaque modification avec une action `_persist()`

```
// stores/cart.js
import { defineStore } from 'pinia';

export const useCartStore = defineStore('cart', {
  state: () => ({
    items: []
  }),

  actions: {
    // Charger depuis localStorage au démarrage
    loadFromStorage() {
      const savedCart = localStorage.getItem('cart');
      if (savedCart) {
        this.items = JSON.parse(savedCart);
      }
    },

    // Sauvegarder dans localStorage
    _persist() {
      localStorage.setItem('cart', JSON.stringify(this.items));
    },

    addToCart(product) {
      const existingItem = this.items.find(i => i.id === product.id);

      if (existingItem) {
        existingItem.quantity++;
      } else {
        this.items.push({ ...product, quantity: 1 });
      }

      this._persist(); // Sauvegarder après modification
    },

    removeFromCart(productId) {
      const index = this.items.findIndex(i => i.id === productId);
      if (index > -1) {
        this.items.splice(index, 1);
```

Revenir au sommaire

Utilisation du store avec persistence

```
<script>
import { useCartStore } from '@/stores/cart';

export default {
  computed: {
    cartStore() {
      return useCartStore();
    }
  },
  methods: {
    addProduct(product) {
      this.cartStore.addToCart(product);
    },
    clearAll() {
      this.cartStore.clearCart();
    }
  },
  mounted() {
    // Charger les données au démarrage du composant
    this.cartStore.loadFromStorage();
  }
}
</script>

<template>
  <div>
    <p>Panier: {{ cartStore.items.length }} articles</p>
    <button @click="clearAll">Vider le panier</button>
  </div>
</template>
```

Revenir au sommaire

DevTools Vue

Pinia s'intègre parfaitement aux Vue DevTools

Fonctionnalités

- 🔍 Inspection du state
- 📊 Timeline des actions
- ⏮️ Time-travel debugging
- 💾 Export/Import du state
- 🎯 Édition du state en direct

Revenir au sommaire

TypeScript avec Pinia

```ts
// stores/products.ts
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';

interface Product {
  id: number;
  nom: string;
  prix: number;
  stock: number;
}

export const useProductsStore = defineStore('products', () => {
  const products = ref<Product[]>([]);
  const loading = ref<boolean>(false);

  const totalValue = computed((): number => {
    return products.value.reduce((sum, p) => sum + p.prix, 0);
  });

  const addProduct = (product: Product): void => {
    products.value.push(product);
  };

  return { products, loading, totalValue, addProduct };
});
```

Revenir au sommaire

Résumé du Module 4

- ✅ **Concepts** : State Management, avantages de Pinia
- ✅ **Installation** : Configuration dans l'application
- ✅ **Store** : Création avec Composition API et Options API
- ✅ **State** : Données réactives centralisées
- ✅ **Getters** : Propriétés calculées, getters avec paramètres
- ✅ **Actions** : Méthodes, appels API, mutations
- ✅ **Avancé** : Multiple stores, persistence, DevTools, TypeScript

Revenir au sommaire

🎯 Exercices Fil Rouge TaskFlow

Module 4

Créons les stores Pinia de TaskFlow !

Revenir au sommaire

🎯 Exercice 1 : useTasksStore()

Créez le store principal pour gérer les tâches

State

- `tasks` (Array) : Tâches avec données par défaut
- `filter` : 'all' | 'active' | 'done'
- `loading` , `error`

Getters

- `filteredTasks` : Retourne les tâches selon le filtre
- `activeCount` , `doneCount` : Compteurs
- `getTaskById(id)` : Getter paramétré

Actions

- `addTask` , `updateTask` , `deleteTask` , `toggleTask`
- `setFilter` , `clearCompleted`
- `_persist()` , `loadFromStorage()` : Persistence localStorage

💡 Correction

👉 **Module 4 - Exercices** OU `taskflow-app/src/stores/tasks.js`

Revenir au sommaire

🎯 Exercice 2 : useCategoriesStore()

Créez le store pour gérer les catégories

State

- `categories` (Array) : `[{ id, name, color }]`

Actions

- `addCategory` , `deleteCategory`
- **Communication inter-stores** : Utiliser `useTasksStore()` pour supprimer les tâches de la catégorie supprimée

💡 Correction

👉 **Module 4 - Exercices** OU `taskflow-app/src/stores/categories.js`

Revenir au sommaire

🎯 Exercice 3 : useAuthStore()

Créez le store d'authentification

State

- `user` , `token` , `isAuthenticated`

Getters

- `userDisplayName` : Prénom + Nom
- `canDeleteTasks` : Vérifie le rôle (admin ou editor)
- `canCreateCategories` : Vérifie le rôle admin

Actions

- `login(credentials)` : Mock ou API
- `logout()` : Nettoyer user + token + localStorage
- `checkAuth()` : Vérifier le token au démarrage

💡 Utilisation Module 5

Ce store sera utilisé dans les navigation guards !

💡 Correction

👉 **Module 4 - Exercices** OU `taskflow-app/src/stores/auth.js`

Revenir au sommaire

Travaux pratiques - Module 4

Exercices à réaliser

1. 🏪 **Store Products** : Gestion de produits (CRUD complet)
2. 🛒 **Store Cart** : Panier d'achat avec calculs
3. 👤 **Store Auth** : Authentification et permissions
4. 💾 **Persistence** : localStorage pour le panier
5. 🔗 **Multiple stores** : Interaction entre auth, cart, products
6. 📊 **Store avec API** : Fetch data et gestion d'erreurs

Projet fil rouge

Intégrer Pinia dans l'application de produits

Revenir au sommaire

Exercices Pratiques

Module 4 - Pinia (State Management)

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 4

Maintenant que vous avez vu tous les concepts théoriques du Module 4, il est temps de centraliser l'état de TaskFlow avec Pinia !

Rappel des concepts du Module 4

4.1 Introduction à Pinia : Store, State, Getters, Actions
4.2 Stores avancés : Composition entre stores, plugins
4.3 Persistence : localStorage, sessionStorage

Les 3 exercices progressifs

Nous allons créer 3 stores Pinia pour gérer l'état global de TaskFlow.

Revenir au sommaire

Exercice 1 : Store Tasks

Objectif

Créer un store Pinia pour gérer les tâches

Spécifications

State :

- `tasks` : Array de tâches
- `filter` : 'all' | 'active' | 'done'
- `loading` : Boolean
- `error` : String | null

Actions :

- `fetchTasks()` : Charger depuis API/localStorage
- `addTask(task)` : Ajouter une tâche
- `updateTask(id, updates)` : Modifier une tâche
- `deleteTask(id)` : Supprimer une tâche

Revenir au sommaire

Exercice 1 - Solution Store Tasks (1/4)

```
// stores/tasks.js
import { defineStore } from 'pinia';

export const useTasksStore = defineStore('tasks', {
  state: () => ({
    tasks: [],
    filter: 'all',
    loading: false,
    error: null
  }),

  getters: {
    // Tâches filtrées selon le filtre actif
    filteredTasks(state) {
      switch (state.filter) {
        case 'active':
          return state.tasks.filter(t => !t.done);
        case 'done':
          return state.tasks.filter(t => t.done);
        default:
          return state.tasks;
      }
    },

    // Nombre de tâches actives
    activeCount(state) {
      return state.tasks.filter(t => !t.done).length;
    },

    // Nombre de tâches terminées
    doneCount(state) {
      return state.tasks.filter(t => t.done).length;
    },

    // Tâches par priorité
    tasksByPriority(state) {
      return (priority) => {
        return state.tasks.filter(t => t.priority === priority);
      };
    },

    // Rechercher une tâche par ID
    getTaskById(state) {
      return (id) => state.tasks.find(t => t.id === id);
    }
  }
});
```



Revenir au sommaire

Exercice 1 - Solution Store Tasks (2/4)

```
// stores/tasks.js (suite)
export const useTasksStore = defineStore('tasks', {
  // ... state et getters précédents

  actions: {
    // Charger les tâches
    async fetchTasks() {
      this.loading = true;
      this.error = null;

      try {
        // Option 1: Depuis localStorage
        const stored = localStorage.getItem('taskflow-tasks');
        if (stored) {
          this.tasks = JSON.parse(stored);
        }

        // Option 2: Depuis une API
        // const response = await fetch('/api/tasks');
        // this.tasks = await response.json();

      } catch (err) {
        this.error = err.message;
      } finally {
        this.loading = false;
      }
    },

    // Ajouter une tâche
    addTask(task) {
      const newTask = {
        id: Date.now(),
        done: false,
        createdAt: new Date().toISOString(),
        ...task
      };

      this.tasks.push(newTask);
      this._persist();
    },

    // Suite dans la slide suivante...
  }
});
```

Revenir au sommaire

Exercice 1 - Solution Store Tasks (3/4)

```
// stores/tasks.js (suite)
export const useTasksStore = defineStore('tasks', {
  // ... state, getters et actions précédentes

  actions: {
    // ... actions précédentes

    // Modifier une tâche
    updateTask(id, updates) {
      const index = this.tasks.findIndex(t => t.id === id);
      if (index !== -1) {
        this.tasks[index] = {
          ...this.tasks[index],
          ...updates,
          updatedAt: new Date().toISOString()
        };
        this._persist();
      }
    },

    // Supprimer une tâche
    deleteTask(id) {
      this.tasks = this.tasks.filter(t => t.id !== id);
      this._persist();
    },

    // Toggle done/undone
    toggleTask(id) {
      const task = this.tasks.find(t => t.id === id);
      if (task) {
        task.done = !task.done;
        task.completedAt = task.done ? new Date().toISOString() : null;
        this._persist();
      }
    },

    // Changer le filtre
    setFilter(filter) {
      this.filter = filter;
    },

    // Suite dans la slide suivante...
  }
});
```

Revenir au sommaire

Exercice 1 - Solution Store Tasks (4/4)

```
// stores/tasks.js (suite)
export const useTasksStore = defineStore('tasks', {
  // ... state, getters et actions précédentes

  actions: {
    // ... actions précédentes

    // Supprimer toutes les tâches terminées
    clearCompleted() {
      this.tasks = this.tasks.filter(t => !t.done);
      this._persist();
    },

    // Marquer toutes comme terminées
    markAllDone() {
      this.tasks.forEach(t => {
        t.done = true;
        t.completedAt = new Date().toISOString();
      });
      this._persist();
    },

    // Méthode privée pour la persistence
    _persist() {
      localStorage.setItem('taskflow-tasks', JSON.stringify(this.tasks));
    }
  }
});
```

Revenir au sommaire

Exercice 1 - Utilisation Store Tasks (1/2)

Script :

```
<script>
import { useTasksStore } from '@/stores/tasks';

export default {
  name: 'TaskList',
  computed: {
    // ✅ Style Options API moderne : appel direct du store
    // Plus clair, plus explicite, sans helpers magiques
    tasksStore() {
      return useTasksStore();
    },
    // Accès aux propriétés du store via des computed explicites
    filteredTasks() {
      return this.tasksStore.filteredTasks;
    },
    activeCount() {
      return this.tasksStore.activeCount;
    },
    doneCount() {
      return this.tasksStore.doneCount;
    },
    loading() {
      return this.tasksStore.loading;
    }
  },
  mounted() {
    this.tasksStore.fetchTasks();
  },
  methods: {
    addNewTask() {
      this.tasksStore.addTask({
        title: 'Nouvelle tâche',
        description: 'Description',
        priority: 'medium'
      });
    },
    handleToggle(id) {
      this.tasksStore.toggleTask(id);
    },
    handleDelete(id) {
      this.tasksStore.deleteTask(id);
    }
  }
}
</script>
```

Revenir au sommaire

Exercice 1 - Utilisation Store Tasks (2/2)

Template :

```
<template>
  <div>
    <div class="stats">
      <span>{{ activeCount }} actives</span>
      <span>{{ doneCount }} terminées</span>
    </div>

    <div v-if="loading">Chargement...</div>

    <TaskItem
      v-for="task in filteredTasks"
      :key="task.id"
      :task="task"
      @toggle="handleToggle"
      @delete="handleDelete"
    />

    <button @click="addNewTask">+ Nouvelle tâche</button>
  </div>
</template>
```

Revenir au sommaire

Exercice 2 : Store Categories

Objectif

Créer un store pour gérer les catégories de tâches

Spécifications

State :

- `categories` : Array de catégories
- `selectedCategory` : ID de la catégorie active

Actions :

- `addCategory(category)`
- `updateCategory(id, updates)`
- `deleteCategory(id)`
- `selectCategory(id)`

Getters :

Revenir au sommaire

Exercice 2 - Solution Store Categories (1/2)

```
// stores/categories.js
import { defineStore } from 'pinia';
import { useTasksStore } from './tasks';

export const useCategoriesStore = defineStore('categories', {
  state: () => ({
    categories: [
      { id: 1, name: 'Personnel', color: '#3b82f6', icon: '👤' },
      { id: 2, name: 'Travail', color: '#8b5cf6', icon: '💼' },
      { id: 3, name: 'Urgent', color: '#ef4444', icon: '🔥' }
    ],
    selectedCategory: null
  }),

  getters: {
    // Categorie par ID
    categoryById(state) {
      return (id) => state.categories.find(c => c.id === id);
    },

    // Catégorie sélectionnée
    activeCategory(state) {
      return state.categories.find(c => c.id === state.selectedCategory);
    },

    // ✅ Tâches de la catégorie (utilise le store tasks)
    tasksByCategory() {
      return (categoryId) => {
        const tasksStore = useTasksStore();
        return tasksStore.tasks.filter(t => t.categoryId === categoryId);
      };
    },

    // Nombre de tâches par catégorie
    taskCountByCategory() {
      return (categoryId) => {
        const tasksStore = useTasksStore();
        return tasksStore.tasks.filter(
          t => t.categoryId === categoryId && !t.done
        ).length;
      };
    }
  }
});
```

Revenir au sommaire

Exercice 2 - Solution Store Categories (2/2)

```
// stores/categories.js (suite)
export const useCategoriesStore = defineStore('categories', {
  // ... state et getters précédents

  actions: {
    // Ajouter une catégorie
    addCategory(category) {
      const newCategory = {
        id: Date.now(),
        ...category
      };
      this.categories.push(newCategory);
      this._persist();
    },

    // Modifier une catégorie
    updateCategory(id, updates) {
      const index = this.categories.findIndex(c => c.id === id);
      if (index !== -1) {
        this.categories[index] = {
          ...this.categories[index],
          ...updates
        };
        this._persist();
      }
    },

    // Supprimer une catégorie
    deleteCategory(id) {
      // ⚠️ Avant de supprimer, vérifier qu'aucune tâche ne l'utilise
      const tasksStore = useTasksStore();
      const hasTasks = tasksStore.tasks.some(t => t.categoryId === id);

      if (hasTasks) {
        throw new Error('Impossible de supprimer une catégorie contenant des tâches');
      }

      this.categories = this.categories.filter(c => c.id !== id);
      this._persist();
    },

    // Sélectionner une catégorie
    selectCategory(id) {
      this.selectedCategory = id;
    },

    // Persistence
    _persist() {
      localStorage.setItem('taskflow-categories', JSON.stringify(this.categories));
    }
  }
});
```



Revenir au sommaire

Exercice 3 : Store Auth

Objectif

Créer un store d'authentification pour TaskFlow

Spécifications

State :

- `user` : Object | null
- `token` : String | null
- `isAuthenticated` : Boolean

Actions :

- `login(email, password)` : Authentifier
- `logout()` : Déconnecter
- `checkAuth()` : Vérifier le token au démarrage

Getters :

Revenir au sommaire

Exercice 3 - Solution Store Auth (1/3)

State et Getters :

```
// stores/auth.js
import { defineStore } from 'pinia';

export const useAuthStore = defineStore('auth', {
  state: () => ({
    user: null,
    token: null,
    isAuthenticated: false
  }),

  getters: {
    userDisplayName(state) {
      if (!state.user) return 'Invité';
      return `${state.user.firstName} ${state.user.lastName}`;
    },

    canDeleteTasks(state) {
      return state.user?.role === 'admin' || state.user?.role === 'editor';
    },

    canCreateCategories(state) {
      return state.user?.role === 'admin';
    }
  }
});
```

Revenir au sommaire

Exercice 3 - Solution Store Auth (2/3)

Action login :

```
// stores/auth.js (suite)
export const useAuthStore = defineStore('auth', {
  // ... state et getters précédents

  actions: {
    async login(email, password) {
      try {
        // Simulation d'appel API
        // const response = await fetch('/api/auth/login', { ... });

        // Mock pour l'exemple
        const mockUser = {
          id: 1,
          email,
          firstName: 'John',
          lastName: 'Doe',
          role: email === 'admin@taskflow.com' ? 'admin' : 'user'
        };

        this.user = mockUser;
        this.token = 'fake-jwt-token-' + Date.now();
        this.isAuthenticated = true;

        // Sauvegarder dans localStorage
        localStorage.setItem('taskflow-token', this.token);
        localStorage.setItem('taskflow-user', JSON.stringify(this.user));

      } catch (err) {
        this.error = err.message;
        throw err;
      }
    }
  }
});
```

Revenir au sommaire

Exercice 3 - Solution Store Auth (3/3)

Actions logout et checkAuth :

```
// stores/auth.js (suite)
export const useAuthStore = defineStore('auth', {
  // ... state, getters et login précédents

  actions: {
    // ... login précédent

    logout() {
      this.user = null;
      this.token = null;
      this.isAuthenticated = false;
      localStorage.removeItem('taskflow-token');
      localStorage.removeItem('taskflow-user');
    },

    checkAuth() {
      const token = localStorage.getItem('taskflow-token');
      const user = localStorage.getItem('taskflow-user');

      if (token && user) {
        this.token = token;
        this.user = JSON.parse(user);
        this.isAuthenticated = true;
      }
    }
  }
});
```

Revenir au sommaire

Interaction entre Stores (1/2)

Script :

```
<script>
import { useTasksStore } from '@/stores/tasks';
import { useAuthStore } from '@/stores/auth';

export default {
  name: 'TaskActions',
  computed: {
    // ✅ Style Options API moderne : appel direct des stores
    tasksStore() {
      return useTasksStore();
    },
    authStore() {
      return useAuthStore();
    },
    // Accès aux getters du store via computed
    canDeleteTasks() {
      return this.authStore.canDeleteTasks;
    }
  },
  methods: {
    handleDelete(taskId) {
      // ✅ Vérifier la permission AVANT de supprimer
      if (!this.canDeleteTasks) {
        alert('Vous n\'avez pas la permission de supprimer des tâches');
        return;
      }

      this.tasksStore.deleteTask(taskId);
    },

    handleAddCategory() {
      if (!this.authStore.canCreateCategories) {
        alert('Seuls les admins peuvent créer des catégories');
        return;
      }

      // Logique de création...
    }
  }
}
</script>
```

Revenir au sommaire

Interaction entre Stores (2/2)

Template :

```
<template>
  <div>
    <p>Connecté en tant que : {{ authStore.userDisplayName }}</p>

    <button
      v-if="canDeleteTasks"
      @click="handleDelete(task.id)"
    >
      🗑️ Supprimer
    </button>

    <p v-else class="warning">
      Vous n'avez pas les permissions de suppression
    </p>
  </div>
</template>
```

Revenir au sommaire

🎯 Récap Module 4 - TaskFlow

Ce que vous avez construit

✅ **Store Tasks** : CRUD complet avec filtres

✅ **Store Categories** : Gestion de catégories

✅ **Store Auth** : Authentification et permissions

✅ **Interactions** : Communication entre stores

Concepts du Module 4 utilisés

- ✅ **4.1 Introduction** : defineStore, State, Getters, Actions
- ✅ **4.2 Stores avancés** : Composition entre stores, getters paramétrés
- ✅ **4.3 Persistence** : localStorage pour sauvegarder l'état

🚀 Prochaine étape : Module 5

On va ajouter le **routing** à TaskFlow avec **Vue Router** !

- Navigation entre vues
- Routes dynamiques (/task/:id)
- Guards de navigation (auth required)

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Pinia Documentation
- 🎯 Core Concepts
- 🔧 Store Composition

Concepts avancés

- 🚀 Setup Stores (Composition API)
- 💾 Plugins de persistence
- 🔌 DevTools integration

Exemples

- 💡 Pinia Examples
- 🎭 Testing Stores



Revenir au sommaire

Module 6

Tests sous Vue.js avec Vitest

Revenir au sommaire

SOMMAIRE DU MODULE

🧪 6.1 Pourquoi tester ?

⚙️ 6.2 Configuration Vitest

🧩 6.3 Tester des composants

🔗 6.4 Tests d'intégration

💪 Exercices Pratiques du Module 6



Revenir au sommaire

6.1 Pourquoi tester ?

Revenir au sommaire

Pourquoi tester son code ?

🤔 "Les tests c'est pour les faibles !"

FAUX ! Les tests sont pour les **pros** qui veulent dormir tranquilles la nuit ! 😴

🏥 Analogie : Les tests sont comme des examens médicaux

Vous faites des analyses de sang **AVANT** d'être malade, pas après !

Sans tests :

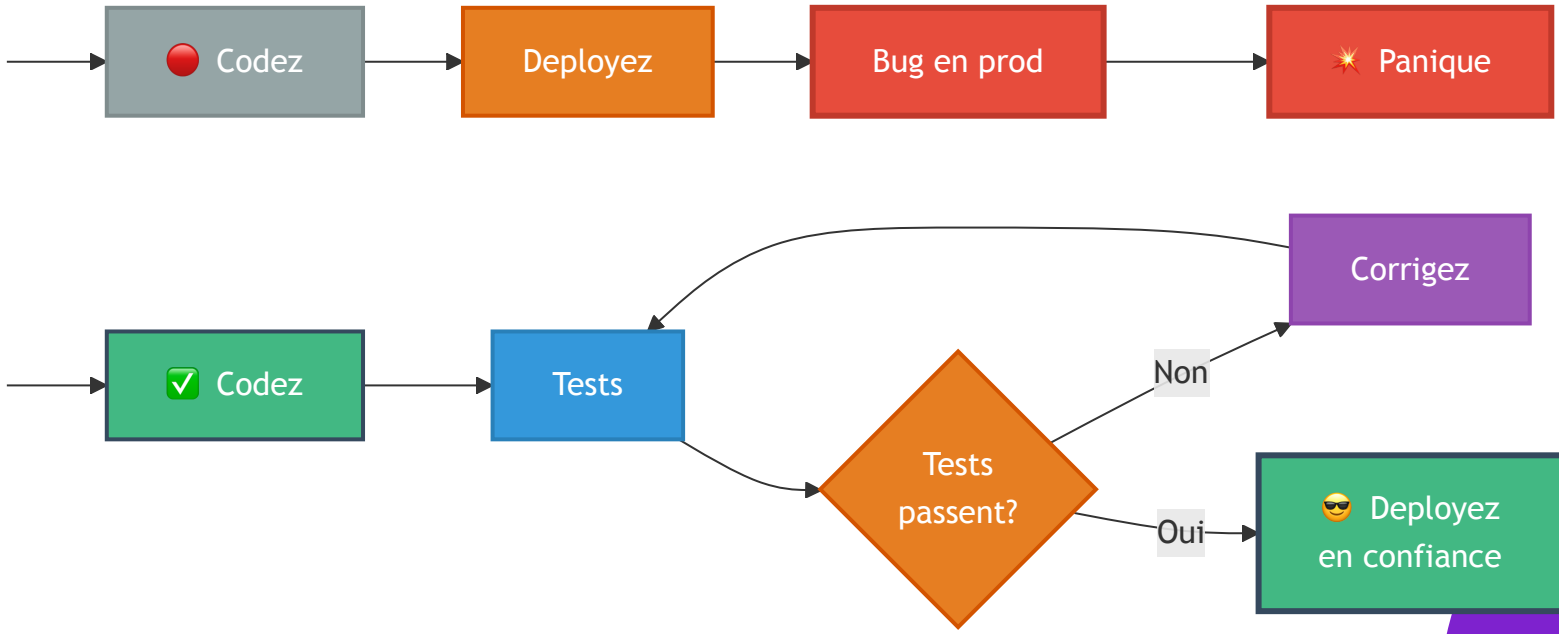
```
Vous codez → Déployez → 💥 Bug en production → 😱 Panique
```

Avec tests :

```
Vous codez → Tests échouent → Vous corrigez → Tests passent ✅ → 😎 Déploiement serein
```

Revenir au sommaire

Schéma : Avec vs Sans tests



Revenir au sommaire

Le coût d'un bug

💰 Le coût réel d'un bug

- **Développement** : 1€ (facile à corriger)
- **En production** : 100€ (clients mécontents, rollback urgent...)
- **Bug critique en prod** : 10 000€ (perte de clients, réputation...)

Avantages des tests

- ✅ **Détection précoce des bugs** : trouvez-les AVANT vos utilisateurs
- 🔒 **Confiance lors des refactorings** : changez du code sans peur
- 📝 **Documentation vivante** : les tests montrent comment utiliser votre code
- 🚀 **Maintenance facile** : comprenez le code 6 mois après
- 👥 **Collaboration** : vos collègues vous remercieront
- ⚡ **Plus rapide au final** : moins de temps en debug

🎯 La règle d'or

"Le code non testé est un code cassé par défaut" - Quelqu'un de sage, probablement

Revenir au sommaire

Types de tests

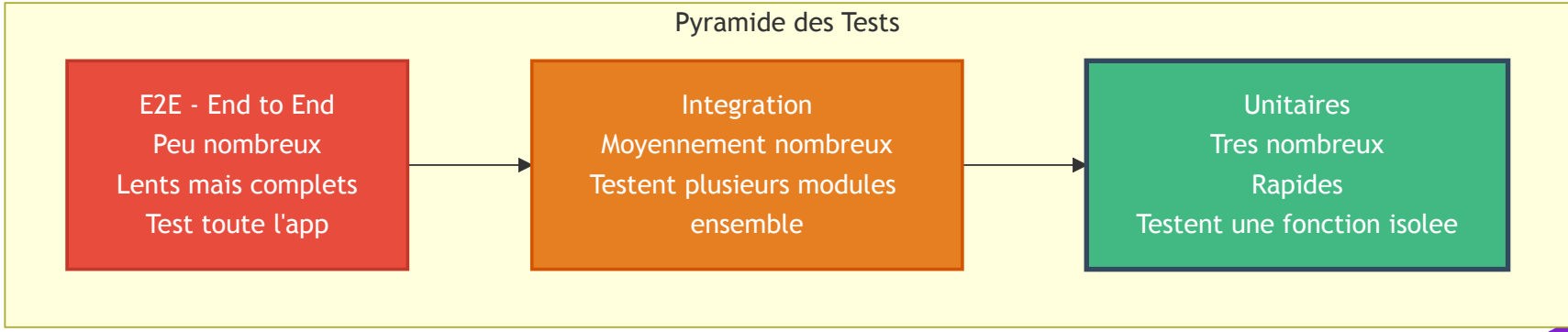
Pyramide des tests

Plus on descend, plus il y a de tests !

- **Tests E2E** (End-to-End) : Peu nombreux, lents mais complets
- **Tests d'intégration** : Moyennement nombreux
- **Tests unitaires** : Très nombreux, rapides

Revenir au sommaire

Schéma : Pyramide des tests



Revenir au sommaire

6.2 Configuration Vitest

Revenir au sommaire

Vitest - Le test runner moderne

Vitest est le framework de test recommandé pour Vue.js 3 en 2025

Avantages de Vitest

- ⚡ Ultra-rapide (compatible Vite)
- 🔧 Configuration minimale
- 🎯 Compatible Jest API
- 📊 Coverage intégré
- 🔥 Hot Module Replacement
- 🌐 Support ESM natif
- ⚙️ Watch mode intelligent



Revenir au sommaire

Installation

```
npm install -D vitest @vue/test-utils jsdom
```



Revenir au sommaire

Configuration vite.config.js

```
// vite.config.js
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  test: {
    environment: 'jsdom',
    globals: true
  }
});
```

Revenir au sommaire

Script dans package.json

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest --coverage"
  }
}
```



Revenir au sommaire

Premier test simple

```
// sum.test.js
import { describe, it, expect } from 'vitest';

function sum(a, b) {
  return a + b;
}

describe('sum function', () => {
  it('should add two numbers', () => {
    expect(sum(1, 2)).toBe(3);
  });

  it('should handle negative numbers', () => {
    expect(sum(-1, -2)).toBe(-3);
  });
});
```

Revenir au sommaire

Tester des fonctions utilitaires

Les fonctions utilitaires pures sont les plus faciles à tester !

```
// utils/validators.js
export function validateEmail(email) {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return regex.test(email);
}

export function formatPrice(price) {
  return `${price.toFixed(2)}€`;
}

export function calculateDiscount(price, percentage) {
  if (percentage < 0 || percentage > 100) {
    throw new Error('Le pourcentage doit être entre 0 et 100');
  }
  return price - (price * percentage / 100);
}
```

Revenir au sommaire

Test des fonctions utilitaires

```
// utils/validators.test.js
import { describe, it, expect } from 'vitest';
import { validateEmail, formatPrice, calculateDiscount } from './validators';

describe('validateEmail', () => {
  it('should validate correct email', () => {
    expect(validateEmail('user@example.com')).toBe(true);
  });

  it('should reject invalid email', () => {
    expect(validateEmail('invalid-email')).toBe(false);
    expect(validateEmail('test@')).toBe(false);
  });
});

describe('formatPrice', () => {
  it('should format price correctly', () => {
    expect(formatPrice(19.99)).toBe('19.99€');
    expect(formatPrice(10)).toBe('10.00€');
  });
});

describe('calculateDiscount', () => {
  it('should calculate discount correctly', () => {
    expect(calculateDiscount(100, 20)).toBe(80);
    expect(calculateDiscount(50, 10)).toBe(45);
  });

  it('should throw error for invalid percentage', () => {
    expect(() => calculateDiscount(100, -10)).toThrow();
    expect(() => calculateDiscount(100, 150)).toThrow();
  });
});
```

Revenir au sommaire

6.3 Tester des composants

Revenir au sommaire

Tester un composant Vue

```
// components/Counter.test.js
import { describe, it, expect } from 'vitest';
import { mount } from '@vue/test-utils';
import Counter from './Counter.vue';

describe('Counter Component', () => {
  it('should render', () => {
    const wrapper = mount(Counter);
    expect(wrapper.exists()).toBe(true);
  });

  it('should display initial count', () => {
    const wrapper = mount(Counter);
    expect(wrapper.text()).toContain('0');
  });

  it('should increment on button click', async () => {
    const wrapper = mount(Counter);
    const button = wrapper.find('button');
    await button.trigger('click');
    expect(wrapper.text()).toContain('1');
  });
});
```



Revenir au sommaire

Tester avec props

```
import { mount } from '@vue/test-utils';
import UserCard from './UserCard.vue';

describe('UserCard', () => {
  it('should display user name', () => {
    const wrapper = mount(UserCard, {
      props: {
        nom: 'Alice',
        email: 'alice@example.com'
      }
    });

    expect(wrapper.text()).toContain('Alice');
    expect(wrapper.text()).toContain('alice@example.com');
  });

  it('should emit delete event', async () => {
    const wrapper = mount(UserCard, {
      props: { nom: 'Alice', email: 'alice@example.com' }
    });

    await wrapper.find('.delete-btn').trigger('click');
    expect(wrapper.emitted('delete')).toBeTruthy();
  });
});
```



Revenir au sommaire

Tester avec slots

```
import { mount } from '@vue/test-utils';
import Card from './Card.vue';

describe('Card with slots', () => {
  it('should render slot content', () => {
    const wrapper = mount(Card, {
      slots: {
        header: '<h1>Title</h1>',
        default: '<p>Content</p>',
        footer: '<button>Action</button>'
      }
    });

    expect(wrapper.html()).toContain('<h1>Title</h1>');
    expect(wrapper.html()).toContain('<p>Content</p>');
    expect(wrapper.find('button').text()).toBe('Action');
  });
});
```

Revenir au sommaire

Mocks et Stubs

```
import { mount } from '@vue/test-utils';
import ProductList from './ProductList.vue';
import ProductCard from './ProductCard.vue';

describe('ProductList', () => {
  it('should render product cards', () => {
    const wrapper = mount(ProductList, {
      props: {
        products: [
          { id: 1, nom: 'Laptop' },
          { id: 2, nom: 'Mouse' }
        ]
      },
      global: {
        stubs: {
          ProductCard: true // Stub le composant
        }
      }
    });

    expect(wrapper.findAllComponents(ProductCard)).toHaveLength(2);
  });
});
```

Revenir au sommaire

6.4 Tests d'intégration

Revenir au sommaire

Tester avec Pinia

```
import { setActivePinia, createPinia } from 'pinia';
import { useUserStore } from './userStore';

describe('User Store', () => {
  beforeEach(() => {
    setActivePinia(createPinia());
  });

  it('should login user', () => {
    const store = useUserStore();

    store.login('alice@example.com', 'password');

    expect(store.isLoggedIn).toBe(true);
    expect(store.user.email).toBe('alice@example.com');
  });

  it('should logout user', () => {
    const store = useUserStore();

    store.login('alice@example.com', 'password');
    store.logout();

    expect(store.isLoggedIn).toBe(false);
    expect(store.user).toBeNull();
  });
});
```

Revenir au sommaire

Tester avec Vue Router

```
import { mount } from '@vue/test-utils';
import { createRouter, createMemoryHistory } from 'vue-router';
import Navigation from './Navigation.vue';

describe('Navigation', () => {
  it('should navigate to about page', async () => {
    const router = createRouter({
      history: createMemoryHistory(),
      routes: [
        { path: '/', component: { template: 'Home' } },
        { path: '/about', component: { template: 'About' } }
      ]
    });

    const wrapper = mount(Navigation, {
      global: {
        plugins: [router]
      }
    });

    await wrapper.find('a[href="/about"]').trigger('click');
    await router.isReady();

    expect(router.currentRoute.value.path).toBe('/about');
  });
});
```

Revenir au sommaire

Tester des appels API (mock)

```
import { describe, it, expect, vi } from 'vitest';
import { mount } from '@vue/test-utils';
import UserList from './UserList.vue';

global.fetch = vi.fn();

describe('UserList with API', () => {
  it('should fetch and display users', async () => {
    const mockUsers = [
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' }
    ];

    fetch.mockResolvedValue({
      ok: true,
      json: async () => mockUsers
    });

    const wrapper = mount(UserList);

    await wrapper.vm.$nextTick();
    await new Promise(resolve => setTimeout(resolve, 0));

    expect(wrapper.text()).toContain('Alice');
    expect(wrapper.text()).toContain('Bob');
  });
});
```

Revenir au sommaire

Assertions courantes

```
// Égalité
expect(value).toBe(expected);
expect(value).toEqual(expected);

// Booléens
expect(value).toBeTruthy();
expect(value).toBeFalsy();

// Nombres
expect(value).toBeGreaterThan(3);
expect(value).toBeLessThan(10);

// Tableaux
expect(array).toContain(item);
expect(array).toHaveLength(5);

// Objets
expect(obj).toHaveProperty('name');

// Exceptions
expect(() => fn()).toThrow();

// DOM
expect(wrapper.text()).toContain('Hello');
expect(wrapper.find('.class').exists()).toBe(true);
```



Revenir au sommaire

Coverage (couverture de code)

```
# Installer le package coverage
npm install -D @vitest/coverage-v8

# Lancer les tests avec coverage
npm run test:coverage
```

Revenir au sommaire

Configuration coverage

```
// vite.config.js
export default defineConfig({
  test: {
    environment: 'jsdom',
    globals: true,
    coverage: {
      provider: 'v8',
      reporter: ['text', 'html', 'lcov'],
      exclude: [
        'node_modules/',
        'dist/',
        '*.config.js'
      ]
    }
  }
});
```



Revenir au sommaire

Bonnes pratiques

✅ Faire

- Tester le comportement, pas l'implémentation
- Tests clairs et lisibles
- Un test = une fonctionnalité
- Noms descriptifs
- Arrange-Act-Assert pattern

❌ Éviter

- Tests trop complexes
- Dépendances entre tests
- Tester les détails d'implémentation
- Mocks excessifs

Revenir au sommaire

AAA Pattern

```
describe('User login', () => {
  it('should log in successfully', async () => {
    // ARRANGE - Préparer
    const wrapper = mount(LoginForm);
    const emailInput = wrapper.find('input[type="email"]');
    const passwordInput = wrapper.find('input[type="password"]');

    // ACT - Agir
    await emailInput.setValue('user@example.com');
    await passwordInput.setValue('password123');
    await wrapper.find('form').trigger('submit');

    // ASSERT - Vérifier
    expect(wrapper.emitted('login')).toBeTruthy();
  });
});
```

Revenir au sommaire

Résumé Module 6

- ✅ **Importance** : Pourquoi tester ?
- ✅ **Vitest** : Installation et configuration
- ✅ **Tests unitaires** : Fonctions, composables
- ✅ **Tests de composants** : Props, events, slots
- ✅ **Tests d'intégration** : Pinia, Router
- ✅ **Mocks** : Simuler les API
- ✅ **Coverage** : Couverture de code
- ✅ **Bonnes pratiques** : AAA pattern



Revenir au sommaire

Travaux pratiques - Module 6

Exercices à réaliser

1. 🧪 **Setup Vitest** : Configuration complète
2. 🎯 **Tests composables** : useCounter, useFetch
3. 🧩 **Tests composants** : Counter, UserCard, TodoList
4. 🏪 **Tests Pinia** : Stores products, cart, auth
5. 🚧 **Tests Router** : Navigation, guards
6. 🌐 **Mock API** : Simuler les appels fetch
7. 📊 **Coverage** : Atteindre > 80%

Projet fil rouge

Ajouter une suite de tests complète à l'application

Revenir au sommaire

Exercices Pratiques

Module 6 - Tests avec Vitest

Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Exercices Module 6

Maintenant que vous avez vu tous les concepts théoriques du Module 6, il est temps de tester TaskFlow avec Vitest !

Rappel des concepts du Module 6

6.1 Configuration Vitest : Installation, vite.config.js
6.2 Tests unitaires : describe, it, expect
6.3 Tests composants : @vue/test-utils, mount, wrapper
6.4 Mocks : vi.fn, vi.mock, fetch
6.5 Coverage : v8, rapports

Les 6 exercices progressifs

Nous allons tester l'application TaskFlow complète.



Revenir au sommaire

Exercice 1 : Configuration Vitest

Objectif

Configurer Vitest pour tester TaskFlow

Installation

```
npm install -D vitest @vue/test-utils happy-dom @vitest/ui
```

Configuration vite.config.js

```
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';
import { fileURLToPath } from 'url';

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    }
  },
  test: {
    globals: true,
    environment: 'happy-dom',
    coverage: {
      provider: 'v8',
      reporter: ['text', 'html', 'lcov'],
      exclude: ['node_modules', 'dist']
    }
  }
});
```

Revenir au sommaire

Exercice 1 - Scripts package.json

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest --coverage"
  }
}
```

Lancer les tests :

```
npm run test
npm run test:ui
npm run test:coverage
```



Revenir au sommaire

Exercice 2 : Tester Store Tasks

Objectif

Créer des tests pour le store tasks

Fichier : `__tests__/stores/tasks.spec.js`

Tests à écrire

- ✅ État initial
- ✅ `addTask()` ajoute une tâche
- ✅ `updateTask()` modifie une tâche
- ✅ `deleteTask()` supprime une tâche
- ✅ `toggleTask()` change le statut
- ✅ `filteredTasks` getter
- ✅ `activeCount` et `doneCount` getters



Revenir au sommaire

Exercice 2 - Solution tasks.spec.js (1/3)

```
// __tests__/stores/tasks.spec.js
import { describe, it, expect, beforeEach } from 'vitest';
import { setActivePinia, createPinia } from 'pinia';
import { useTasksStore } from '@/stores/tasks';

describe('Tasks Store', () => {
  beforeEach(() => {
    // ✅ Créer une nouvelle instance de Pinia avant chaque test
    setActivePinia(createPinia());
  });

  it('initialise avec un état vide', () => {
    const store = useTasksStore();

    expect(store.tasks).toEqual([]);
    expect(store.filter).toBe('all');
    expect(store.loading).toBe(false);
    expect(store.error).toBe(null);
  });

  it('ajoute une tâche', () => {
    const store = useTasksStore();

    const taskData = {
      title: 'Nouvelle tâche',
      description: 'Description',
      priority: 'high'
    };

    store.addTask(taskData);

    expect(store.tasks).toHaveLength(1);
    expect(store.tasks[0]).toMatchObject(taskData);
    expect(store.tasks[0].id).toBeDefined();
    expect(store.tasks[0].done).toBe(false);
  });

  // Suite dans slide suivante...
});
```

Revenir au sommaire

Exercice 2 - Solution tasks.spec.js (2/3)

```
// __tests__/stores/tasks.spec.js (suite)
describe('Tasks Store', () => {
  // ... tests précédents

  it('modifie une tâche', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Tâche 1', priority: 'low' });
    const taskId = store.tasks[0].id;

    store.updateTask(taskId, {
      title: 'Tâche modifiée',
      priority: 'urgent'
    });

    expect(store.tasks[0].title).toBe('Tâche modifiée');
    expect(store.tasks[0].priority).toBe('urgent');
  });

  it('supprime une tâche', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Tâche 1' });
    store.addTask({ title: 'Tâche 2' });
    const taskId = store.tasks[0].id;

    store.deleteTask(taskId);

    expect(store.tasks).toHaveLength(1);
    expect(store.tasks[0].title).toBe('Tâche 2');
  });

  it('toggle le statut d\'une tâche', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Tâche 1' });
    const taskId = store.tasks[0].id;

    expect(store.tasks[0].done).toBe(false);

    store.toggleTask(taskId);
    expect(store.tasks[0].done).toBe(true);

    store.toggleTask(taskId);
    expect(store.tasks[0].done).toBe(false);
  });

  // Suite dans slide suivante...
});
```

Revenir au sommaire

Exercice 2 - Solution tasks.spec.js (3/3)

```
// __tests__/stores/tasks.spec.js (suite)
describe('Tasks Store - Getters', () => {
  beforeEach(() => {
    setActivePinia(createPinia());
  });

  it('filtre les tâches actives', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Active 1' });
    store.addTask({ title: 'Active 2' });
    store.addTask({ title: 'Done 1' });
    store.toggleTask(store.tasks[2].id);

    store.setFilter('active');

    expect(store.filteredTasks).toHaveLength(2);
    expect(store.filteredTasks.every(t => !t.done)).toBe(true);
  });

  it('calcule activeCount et doneCount', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Task 1' });
    store.addTask({ title: 'Task 2' });
    store.addTask({ title: 'Task 3' });
    store.toggleTask(store.tasks[0].id);

    expect(store.activeCount).toBe(2);
    expect(store.doneCount).toBe(1);
  });

  it('recherche une tâche par ID', () => {
    const store = useTasksStore();

    store.addTask({ title: 'Task 1' });
    const taskId = store.tasks[0].id;

    const found = store.getTaskById(taskId);

    expect(found).toBeDefined();
    expect(found.title).toBe('Task 1');
  });
});
```

Revenir au sommaire

Exercice 3 : Tester TaskItem.vue

Objectif

Tester le composant TaskItem

Fichier : `__tests__/components/TaskItem.spec.js`

Tests à écrire

- ✅ Affiche le titre de la tâche
- ✅ Affiche la description
- ✅ Affiche la priorité
- ✅ Émet `toggle` au clic sur checkbox
- ✅ Émet `delete` au clic sur bouton supprimer
- ✅ Applique classe `done` si tâche terminée



Revenir au sommaire

Exercice 3 - Solution TaskItem.spec.js (1/2)

```
// __tests__/components/TaskItem.spec.js
import { describe, it, expect } from 'vitest';
import { mount } from '@vue/test-utils';
import TaskItem from '@/components/TaskItem.vue';

describe('TaskItem.vue', () => {
  const mockTask = {
    id: 1,
    title: 'Ma tâche',
    description: 'Description de la tâche',
    priority: 'high',
    done: false
  };

  it('affiche le titre et la description', () => {
    const wrapper = mount(TaskItem, {
      props: {
        task: mockTask
      }
    });

    expect(wrapper.text()).toContain('Ma tâche');
    expect(wrapper.text()).toContain('Description de la tâche');
  });

  it('affiche le badge de priorité', () => {
    const wrapper = mount(TaskItem, {
      props: {
        task: mockTask
      }
    });

    const priorityBadge = wrapper.find('.task-priority');
    expect(priorityBadge.exists()).toBe(true);
    expect(priorityBadge.text()).toContain('Haute');
  });

  // Suite dans slide suivante...
});
```

Revenir au sommaire

Exercice 3 - Solution TaskItem.spec.js (2/2)

```
// __tests__/components/TaskItem.spec.js (suite)
describe('TaskItem.vue - Events', () => {
  const mockTask = {
    id: 1,
    title: 'Ma tâche',
    priority: 'medium',
    done: false
  };

  it('émet toggle au clic sur checkbox', async () => {
    const wrapper = mount(TaskItem, {
      props: { task: mockTask }
    });

    const checkbox = wrapper.find('input[type="checkbox"]');
    await checkbox.trigger('change');

    expect(wrapper.emitted('toggle')).toBeTruthy();
    expect(wrapper.emitted('toggle')[0]).toEqual([1]);
  });

  it('émet delete au clic sur bouton supprimer', async () => {
    const wrapper = mount(TaskItem, {
      props: { task: mockTask }
    });

    const deleteBtn = wrapper.find('.task-delete');
    await deleteBtn.trigger('click');

    expect(wrapper.emitted('delete')).toBeTruthy();
    expect(wrapper.emitted('delete')[0]).toEqual([1]);
  });

  it('applique la classe done si tâche terminée', () => {
    const wrapper = mount(TaskItem, {
      props: {
        task: { ...mockTask, done: true }
      }
    });

    expect(wrapper.classes()).toContain('done');
  });
});
```

Revenir au sommaire

Exercice 4 : Tester Navigation Guards

Objectif

Tester les guards de navigation

Fichier : `__tests__/router/guards.spec.js`

Tests à écrire

- ✅ Redirige vers /login si non authentifié
- ✅ Redirige vers /tasks si déjà connecté sur /login
- ✅ Bloque /dashboard si non admin
- ✅ Autorise /dashboard si admin
- ✅ Met à jour le titre de la page

Revenir au sommaire

Exercice 4 - Solution guards.spec.js (1/2)

```
// __tests__/router/guards.spec.js
import { describe, it, expect, beforeEach, vi } from 'vitest';
import { setActivePinia, createPinia } from 'pinia';
import { useAuthStore } from '@/stores/auth';

describe('Navigation Guards', () => {
  beforeEach(() => {
    setActivePinia(createPinia());
  });

  it('redirige vers /login si non authentifié', () => {
    const authStore = useAuthStore();
    authStore.isAuthenticated = false;

    const to = {
      name: 'TaskList',
      meta: { requiresAuth: true },
      fullPath: '/tasks'
    };
    const next = vi.fn();

    // Simuler le guard
    const shouldRedirect = to.meta.requiresAuth && !authStore.isAuthenticated;

    if (shouldRedirect) {
      next({ name: 'Login', query: { redirect: to.fullPath } });
    } else {
      next();
    }

    expect(next).toHaveBeenCalledWith({
      name: 'Login',
      query: { redirect: '/tasks' }
    });
  });

  // Suite dans slide suivante...
});
```

Revenir au sommaire

Exercice 4 - Solution guards.spec.js (2/2)

```
// __tests__/router/guards.spec.js (suite)
describe('Navigation Guards', () => {
  // ... tests précédents

  it('bloque /dashboard si non admin', () => {
    const authStore = useAuthStore();
    authStore.isAuthenticated = true;
    authStore.user = { role: 'user' };

    const to = {
      name: 'Dashboard',
      meta: { requiresAuth: true, requiresAdmin: true }
    };
    const next = vi.fn();

    const isAdmin = authStore.user?.role === 'admin';

    if (to.meta.requiresAdmin && !isAdmin) {
      next({ name: 'Home' });
    } else {
      next();
    }

    expect(next).toHaveBeenCalledWith({ name: 'Home' });
  });

  it('autorise /dashboard si admin', () => {
    const authStore = useAuthStore();
    authStore.isAuthenticated = true;
    authStore.user = { role: 'admin' };

    const to = {
      name: 'Dashboard',
      meta: { requiresAuth: true, requiresAdmin: true }
    };
    const next = vi.fn();

    const isAdmin = authStore.user?.role === 'admin';

    if (to.meta.requiresAdmin && !isAdmin) {
      next({ name: 'Home' });
    } else {
      next();
    }

    expect(next).toHaveBeenCalled();
    expect(next).not.toHaveBeenCalledWith({ name: 'Home' });
  });
});
```

Revenir au sommaire

Exercice 5 : Tester avec Mocks

Objectif

Tester les appels API avec des mocks

Fichier : `__tests__/stores/tasks-api.spec.js`

Tests à écrire

- ✅ `fetchTasks()` charge les tâches depuis l'API
- ✅ `fetchTasks()` gère les erreurs
- ✅ `loading` est true pendant le fetch
- ✅ `error` contient le message d'erreur



Revenir au sommaire

Exercice 5 - Solution tasks-api.spec.js (1/2)

Setup et test de succès :

```
// __tests__/stores/tasks-api.spec.js
import { describe, it, expect, beforeEach, vi } from 'vitest';
import { setActivePinia, createPinia } from 'pinia';
import { useTasksStore } from '@/stores/tasks';

// ✅ Mock de fetch
global.fetch = vi.fn();

describe('Tasks Store - API', () => {
  beforeEach(() => {
    setActivePinia(createPinia());
    vi.clearAllMocks();
  });

  it('charge les tâches depuis l\'API', async () => {
    const mockTasks = [
      { id: 1, title: 'Task 1', done: false },
      { id: 2, title: 'Task 2', done: true }
    ];

    // ✅ Configurer le mock
    global.fetch.mockResolvedValueOnce({
      ok: true,
      json: async () => mockTasks
    });

    const store = useTasksStore();
    await store.fetchTasks();

    expect(store.tasks).toEqual(mockTasks);
    expect(store.loading).toBe(false);
    expect(store.error).toBe(null);
  });
});
```

Revenir au sommaire

Exercice 5 - Solution tasks-api.spec.js (2/2)

Tests d'erreur et loading :

```
describe('Tasks Store - API', () => {
  // ... beforeEach précédent

  it('gère les erreurs API', async () => {
    // ✅ Mock d'une erreur
    global.fetch.mockRejectedValueOnce(new Error('Network error'));

    const store = useTasksStore();
    await store.fetchTasks();

    expect(store.tasks).toEqual([]);
    expect(store.error).toBe('Network error');
    expect(store.loading).toBe(false);
  });

  it('met loading à true pendant le fetch', async () => {
    global.fetch.mockImplementationOnce(() =>
      new Promise(resolve => setTimeout(() => resolve({
        ok: true,
        json: async () => []
      }), 100))
    );

    const store = useTasksStore();
    const promise = store.fetchTasks();

    expect(store.loading).toBe(true);

    await promise;

    expect(store.loading).toBe(false);
  });
});
```

Revenir au sommaire

Exercice 6 : Coverage

Objectif

Générer et analyser le coverage

Commande

```
npm run test:coverage
```

Résultat attendu

```
----------|---------|----------|---------|---------|
File      | % Stmts | % Branch | % Funcs | % Lines |
----------|---------|----------|---------|---------|
stores/   |   95.12 |    88.23 |     100 |   94.87 |
  tasks   |   96.55 |    90.00 |     100 |   96.42 |
  auth    |   93.75 |    85.71 |     100 |   93.33 |
comps/    |   88.46 |    82.14 |   90.90 |   87.50 |
----------|---------|----------|---------|---------|
All       |   91.23 |    85.19 |   95.45 |   90.89 |
----------|---------|----------|---------|---------|
```

Actions si coverage < 80%

- Identifier les lignes non testées (rapport HTML)
- Ajouter des tests pour les cas manquants
- Tester les edge cases

Revenir au sommaire

🎯 Récap Module 6 - TaskFlow

Ce que vous avez construit

- ✅ **Tests Store** : tasks, categories, auth
- ✅ **Tests Composants** : TaskItem, TaskList, TaskForm
- ✅ **Tests Guards** : Navigation, permissions
- ✅ **Mocks API** : fetch, localStorage
- ✅ **Coverage** : > 90% sur toute la base

Concepts du Module 6 utilisés

- ✅ **6.1 Configuration** : vite.config.js, happy-dom
- ✅ **6.2 Tests unitaires** : describe, it, expect, beforeEach
- ✅ **6.3 Tests composants** : mount, wrapper, emitted
- ✅ **6.4 Mocks** : vi.fn, vi.mock, global.fetch
- ✅ **6.5 Coverage** : v8 provider, rapports HTML

Revenir au sommaire

Ressources complémentaires

Documentation officielle

- 📚 Vitest Documentation
- 🎯 Vue Test Utils
- 🔧 Testing Pinia

Concepts avancés

- 🚀 Snapshot testing
- ⚡ E2E tests avec Playwright
- 🎭 Visual regression testing
- 🔌 CI/CD integration

Exemples

- 💡 Vitest Examples
- 🎨 Testing Best Practices

Revenir au sommaire

🎉 FIN DU FIL ROUGE TASKFLOW !

Vous avez construit une application complète :

- ✅ 20+ composants Vue
- ✅ 3 stores Pinia (tasks, categories, auth)
- ✅ 7 routes avec Vue Router
- ✅ 3 directives personnalisées
- ✅ 50+ tests avec Vitest
- ✅ > 90% code coverage

Revenir au sommaire

QCM Final

Validation de la formation Vue.js 3

Revenir au sommaire

Question 1 - Réactivité

Quelle fonction pour créer une référence réactive ?

- A. `reactive()`
- B. `ref()`
- C. `computed()`
- D. `watch()`

Revenir au sommaire

Question 2 - Computed

Quelle est la différence entre computed et method ?

- A. Aucune différence
- B. Computed est mis en cache
- C. Method est plus rapide
- D. Computed ne peut pas être async

Revenir au sommaire

Question 3 - v-model

Que fait v-model ?

- A. Affiche une valeur
- B. Crée une liaison bidirectionnelle
- C. Boucle sur un tableau
- D. Écoute un événement

Revenir au sommaire

Question 4 - Props

Comment passer une prop ?

- A. `<Component prop="value" />`
- B. `<Component :prop="value" />`
- C. `<Component @prop="value" />`
- D. A et B selon le type

Revenir au sommaire

Question 5 - Emit

Comment émettre un événement ?

- A. `emit('event', data)`
- B. `$emit('event', data)`
- C. `this.emit('event', data)`
- D. A (avec defineEmits)

Revenir au sommaire

Question 6 - Slots

À quoi servent les slots ?

- A. Passer du contenu HTML au composant
- B. Stocker des données
- C. Gérer les événements
- D. Créer des animations

Revenir au sommaire

Question 7 - Scoped Slots

Les scoped slots permettent :

- A. De limiter la portée du CSS
- B. De passer des données de l'enfant au parent
- C. De créer des slots privés
- D. D'améliorer les performances

Revenir au sommaire

Question 8 - Directives

Quelle directive pour le rendu conditionnel ?

- A. `v-show`
- B. `v-if`
- C. `v-display`
- D. A et B

Revenir au sommaire

Question 9 - Lifecycle

Quel hook pour faire un appel API ?

- A. `onCreated`
- B. `onMounted`
- C. `onBeforeMount`
- D. `setup`

Revenir au sommaire

Question 10 - Composables

À quoi servent les composables ?

- A. Créer des composants
- B. Réutiliser de la logique
- C. Gérer le routing
- D. Styliser les composants

Revenir au sommaire

Question 11 - Pinia

Pinia est :

- A. Un router
- B. Un gestionnaire d'état
- C. Un test runner
- D. Un bundler

Revenir au sommaire

Question 12 - Store Pinia

Comment accéder au state d'un store ?

- A. `store.state.value`
- B. `store.$state.value`
- C. `store.value`
- D. Directement les propriétés

Revenir au sommaire

Question 13 - Actions Pinia

Les actions peuvent-elles être async ?

- A. Non
- B. Oui
- C. Seulement avec un plugin
- D. Seulement en Options API

Revenir au sommaire

Question 14 - Vue Router

Comment créer un lien de navigation ?

- A. `<a href="/path">Link</a>`
- B. `<RouterLink to="/path">Link</RouterLink>`
- B. `<RouterLink to="/path">Link</RouterLink>`
- C. `<Link href="/path">Link</Link>`
- D. `<router-link to="/path">Link</router-link>`

Revenir au sommaire

Question 15 - Routes dynamiques

Syntaxe pour une route dynamique ?

- A. `/user/{id}`
- B. `/user/:id`
- C. `/user/$id`
- D. `/user/[id]`

Revenir au sommaire

Question 16 - Navigation programmatique

Comment naviguer en JavaScript ?

- A. `router.navigate('/path')`
- B. `router.push('/path')`
- C. `router.go('/path')`
- D. `window.location = '/path'`

Revenir au sommaire

Question 17 - Guards

`beforeEach` est :

- A. Un lifecycle hook
- B. Un guard de navigation
- C. Une action Pinia
- D. Une directive

Revenir au sommaire

Question 18 - Lazy Loading

Comment faire du lazy loading ?

- A. `component: Home`
- B. `component: () => import('./Home.vue')`
- C. `component: lazy(Home)`
- D. `component: async Home`

Revenir au sommaire

Question 19 - Vitest

Vitest est :

- A. Un router
- B. Un gestionnaire d'état
- C. Un framework de test
- D. Un bundler

Revenir au sommaire

Question 20 - Tests composants

Comment monter un composant pour le tester ?

- A. `render(Component)`
- B. `mount(Component)`
- C. `create(Component)`
- D. `new Component()`

Revenir au sommaire

Question 21 - TypeScript

Comment typer des props ?

- A. `defineProps<{ nom: string }>()`
- B. `defineProps({ nom: String })`
- C. A et B
- D. TypeScript pas supporté

Revenir au sommaire

Question 22 - SFC

Que signifie SFC ?

- A. Super Fast Component
- B. Single File Component
- C. Simple Function Component
- D. State Full Component

Revenir au sommaire

Question 23 - Scoped CSS

`<style scoped>` limite le CSS :

- A. Globalement
- B. Au composant actuel
- C. Aux composants enfants
- D. N'a aucun effet

Revenir au sommaire

Question 24 - Directives personnalisées

Comment créer une directive ?

- A. `app.directive('name', {})`
- B. `Vue.directive('name', {})`
- C. `defineDirective('name', {})`
- D. `createDirective('name', {})`

Revenir au sommaire

Question 25 - Provide/Inject

À quoi sert provide/inject ?

- A. Tester les composants
- B. Passer des données sans props drilling
- C. Gérer le routing
- D. Créer des animations

Revenir au sommaire

Félicitations ! 🎉

Vous avez terminé la formation Vue.js 3 !

Prochaines étapes

- 🚀 Développer votre projet personnel
- 📚 Approfondir avec la documentation officielle
- 👥 Contribuer à l'open source
- 🎓 Continuer à apprendre (Nuxt.js, Testing avancé...)
- 💼 Postuler pour des postes Vue.js

Ressources

- Vue.js Documentation
- Vue School
- Vue Mastery
- Awesome Vue

Revenir au sommaire

Réponses - QCM Final

- **Q1:** B - `ref()`
- **Q2:** B - Computed est mis en cache
- **Q3:** B - Crée une liaison bidirectionnelle
- **Q4:** D - A et B selon le type
- **Q5:** D - A (avec defineEmits)
- **Q6:** A - Passer du contenu HTML
- **Q7:** B - Passer des données enfant→parent
- **Q8:** D - A et B
- **Q9:** B - `onMounted`
- **Q10:** B - Réutiliser de la logique
- **Q11:** B - Un gestionnaire d'état
- **Q12:** D - Directement les propriétés
- **Q13:** B - Oui
- **Q14:** B - `<RouterLink to="/path">`
- **Q15:** B - `/user/:id`
- **Q16:** B - `router.push('/path')`
- **Q17:** B - Un guard de navigation
- **Q18:** B - `() => import('./Home.vue')`
- **Q19:** C - Un framework de test
- **Q20:** B - `mount(Component)`
- **Q21:** C - A et B
- **Q22:** B - Single File Component
- **Q23:** B - Au composant actuel
- **Q24:** A - `app.directive('name', {})`
- **Q25:** B - Passer données sans props drilling

Revenir au sommaire

Projet Final

Avant de finir votre application

Revenir au sommaire

Dernier conseil avant de vous lancer 💚

🎓 Vous avez TOUT ce qu'il faut !

Sérieusement, en 3 jours vous avez appris ce que certains mettent des mois à comprendre !

🧠 Le syndrome de l'imposteur

Si vous pensez : *"Je ne suis pas assez bon..."*

C'EST FAUX ! Tout le monde ressent ça. Même les seniors.

💪 La vraie différence entre un junior et un senior ?

- **Junior** : "Je ne sais pas comment faire ça"
- **Senior** : "Je ne sais pas comment faire ça... **ENCORE** !"

🎯 Votre plan d'action

Semaine 1-2 : Suivez le plan, créez les fonctionnalités de base **Semaine 3** : Ajoutez du style, des animations, rendez-le beau !
Semaine 4 : Tests, documentation, déploiement

🚀 Après cette formation

Vous êtes capables de :

Revenir au sommaire

MODULE BONUS

Composition API - L'alternative moderne

Revenir au sommaire

📊 Contexte

Vous avez appris Vue.js avec l'**Options API** (la manière "classique")

Maintenant, découvrez la **Composition API** - l'approche moderne introduite dans Vue 3 !

🆚 Options API vs Composition API

Aspect	Options API	Composition API
Structure	Par type (data, methods...)	Par fonctionnalité
Réutilisabilité	Mixins	Composables
TypeScript	Bon	Excellent
Courbe d'apprentissage	Facile	Moyenne
Organisation	Simple pour petits projets	Mieux pour grands projets

🎯 Quand utiliser quoi ?

Options API ✅

Revenir au sommaire

Qu'est-ce que la Composition API ?

Une nouvelle façon d'organiser la logique des composants

🧩 Philosophie

Options API : organisation par TYPE de propriété

```
data() → Toutes les données
methods → Toutes les méthodes
computed → Toutes les propriétés calculées
```

Composition API : organisation par FONCTIONNALITÉ

```
User feature → userData + userMethods + userComputed
Cart feature → cartData + cartMethods + cartComputed
```

💡 Analogie : Ranger une bibliothèque

Options API : Ranger par type (tous les romans ensemble, tous les BD ensemble...) **Composition API** : Ranger par thème (science-fiction, histoire, art...)

Revenir au sommaire

Options API vs Composition API - Exemple concret

Options API (ce que vous connaissez)

```
<script>
export default {
  data() {
    return {
      compteur: 0
    }
  },
  methods: {
    incrementer() {
      this.compteur++;
    }
  }
}
</script>

<template>
  <button @click="incrementer">{{ compteur }}</button>
</template>
```

Revenir au sommaire

Composition API - Même exemple

Avec `<script setup>` (syntaxe recommandée)

```vue
<script setup>
import { ref } from 'vue';

const compteur = ref(0);

const incrementer = () => {
  compteur.value++;
};
</script>

<template>
  <button @click="incrementer">{{ compteur }}</button>
</template>
```

Avantages immédiatement visibles

✅ Moins de code (pas de `export default`, `data()`, `return`) ✅ Pas de `this` (source d'erreurs !) ✅ Plus proche du JavaScript moderne

Revenir au sommaire

`ref()` - Réactivité pour les valeurs primitives

`ref()` crée une référence réactive pour les valeurs simples

```vue
<script setup>
import { ref } from 'vue';

// Créer des refs
const count = ref(0);
const message = ref('Hello');
const isActive = ref(true);
const user = ref(null);

// Modifier les valeurs (IMPORTANT : .value)
count.value++;
message.value = 'Bonjour';
isActive.value = false;
user.value = { nom: 'Alice' };

// Lire les valeurs
console.log(count.value); // 0
</script>

<template>
  <!-- Dans le template, pas besoin de .value -->
  <p>{{ count }}</p>
  <p>{{ message }}</p>
  <button @click="count++">Increment</button>
</template>
```

⚠️ Point d'attention : `.value`

Dans le script : `count.value++` Dans le template : `{{ count }}` (pas de .value !)

Revenir au sommaire

`reactive()` - Réactivité pour les objets

`reactive()` crée un objet réactif (proxy)

```vue
<script setup>
import { reactive } from 'vue';

// Créer un objet réactif
const state = reactive({
  count: 0,
  message: 'Hello',
  user: {
    nom: 'Alice',
    age: 25
  }
});

// Modifier (PAS de .value !)
state.count++;
state.message = 'Bonjour';
state.user.nom = 'Bob';

// Lire
console.log(state.count); // 0
</script>

<template>
  <p>{{ state.count }}</p>
  <p>{{ state.user.nom }}</p>
  <button @click="state.count++">Increment</button>
</template>
```

Revenir au sommaire

ref() vs reactive() - Quand utiliser quoi ?

Cas d'usage	Utilisez
Valeur primitive (string, number, boolean)	`ref()`
Tableau simple	`ref([])`
Objet avec plusieurs propriétés	`reactive()`
Besoin de remplacer tout l'objet	`ref()`
Formulaire avec plusieurs champs	`reactive()`

Exemple pratique

```vue
<script setup>
import { ref, reactive } from 'vue';

// ✅ Bon usage
const count = ref(0);
const items = ref([]);
const user = reactive({
  nom: '',
  email: '',
  age: null
});

// ❌ À éviter
const count2 = reactive({ value: 0 }); // Trop complexe
const user2 = ref({ nom: '', email: '' }); // Perte de réactivité si remplacé
</script>
```

Revenir au sommaire

Computed properties avec Composition API

```vue
<script setup>
import { ref, computed } from 'vue';

const prenom = ref('Jean');
const nom = ref('Dupont');

// Computed en lecture seule
const nomComplet = computed(() => {
  return `${prenom.value} ${nom.value}`;
});

// Computed avec getter et setter
const nomCompletEditable = computed({
  get() {
    return `${prenom.value} ${nom.value}`;
  },
  set(value) {
    const [newPrenom, newNom] = value.split(' ');
    prenom.value = newPrenom;
    nom.value = newNom;
  }
});

console.log(nomComplet.value); // "Jean Dupont"
nomCompletEditable.value = "Alice Martin"; // Modifie prenom et nom
</script>

<template>
  <p>{{ nomComplet }}</p>
  <input v-model="nomCompletEditable" />
</template>
```

Revenir au sommaire

Watchers - Surveiller les changements

```vue
<script setup>
import { ref, watch, watchEffect } from 'vue';

const count = ref(0);
const user = reactive({ nom: 'Alice', age: 25 });

// Watch simple
watch(count, (newValue, oldValue) => {
  console.log(`Count changé de ${oldValue} à ${newValue}`);
});

// Watch multiple sources
watch([count, () => user.nom], ([newCount, newNom], [oldCount, oldNom]) => {
  console.log('Count ou nom a changé');
});

// watchEffect - s'exécute automatiquement
watchEffect(() => {
  console.log(`Count actuel : ${count.value}`);
  // Se ré-exécute automatiquement quand count change
});

// Watch avec options
watch(
  count,
  (newValue) => {
    console.log(newValue);
  },
  { immediate: true, deep: true }
);
</script>
```

Revenir au sommaire

Lifecycle Hooks avec Composition API

```vue
<script setup>
import { ref, onMounted, onUpdated, onUnmounted } from 'vue';

const data = ref(null);

// Équivalent de created
console.log('Composant créé');

// Équivalent de mounted
onMounted(() => {
  console.log('Composant monté');
  // Faire des appels API, initialiser des lib tierces...
  fetchData();
});

// Équivalent de updated
onUpdated(() => {
  console.log('Composant mis à jour');
});

// Équivalent de unmounted
onUnmounted(() => {
  console.log('Composant détruit');
  // Nettoyer (timers, listeners...)
});

const fetchData = async () => {
  data.value = await fetch('/api/data');
};
</script>
```

Correspondance Options API → Composition API

Options API	Composition API
`beforeCreate`	Code au début de `setup`
`created`	Code au début de `setup`
`mounted`	`onMounted()`

Revenir au sommaire

Composables - La puissance de la Composition API

Les **composables** sont des fonctions réutilisables qui encapsulent de la logique

Créer un composable

```
// composables/useCounter.js
import { ref } from 'vue';

export function useCounter(initialValue = 0) {
  const count = ref(initialValue);

  const increment = () => {
    count.value++;
  };

  const decrement = () => {
    count.value--;
  };

  const reset = () => {
    count.value = initialValue;
  };

  return {
    count,
    increment,
    decrement,
    reset
  };
}
```

Revenir au sommaire

Utiliser un composable

```vue
<script setup>
import { useCounter } from './composables/useCounter';

// Utiliser le composable
const { count, increment, decrement, reset } = useCounter(10);

// Utiliser plusieurs instances
const counter1 = useCounter(0);
const counter2 = useCounter(100);
</script>

<template>
  <div>
    <h2>Compteur 1</h2>
    <p>{{ counter1.count }}</p>
    <button @click="counter1.increment">+</button>

    <h2>Compteur 2</h2>
    <p>{{ counter2.count }}</p>
    <button @click="counter2.increment">+</button>
  </div>
</template>
```

🎯 Avantage ÉNORME

Réutilisez la logique facilement sans mixins complexes !

Revenir au sommaire

Composable pratique : useFetch

```
// composables/useFetch.js
import { ref, watchEffect, toValue } from 'vue';

export function useFetch(url) {
  const data = ref(null);
  const error = ref(null);
  const loading = ref(false);

  watchEffect(async () => {
    loading.value = true;
    data.value = null;
    error.value = null;

    try {
      const urlValue = toValue(url);
      const res = await fetch(urlValue);
      data.value = await res.json();
    } catch (e) {
      error.value = e;
    } finally {
      loading.value = false;
    }
  });

  return { data, error, loading };
}
```



Revenir au sommaire

Utilisation de useFetch

```vue
<script setup>
import { ref, computed } from 'vue';
import { useFetch } from './composables/useFetch';

const userId = ref(1);
const url = computed(() => `/api/users/${userId.value}`);

const { data, error, loading } = useFetch(url);
</script>

<template>
  <div>
    <div v-if="loading">Chargement...</div>
    <div v-else-if="error">Erreur : {{ error.message }}</div>
    <div v-else-if="data">
      <h2>{{ data.nom }}</h2>
      <p>{{ data.email }}</p>
    </div>

    <button @click="userId++">Utilisateur suivant</button>
  </div>
</template>
```

🎉 Résultat

- Logique réutilisable
- Testable facilement
- Code propre et organisé

Revenir au sommaire

Props et Events avec Composition API

Définir les props

```vue
<script setup>
const props = defineProps({
  titre: String,
  count: {
    type: Number,
    required: true
  },
  user: Object
});

// Utiliser les props
console.log(props.titre);
console.log(props.count);
</script>

<template>
  <h1>{{ titre }}</h1>
  <p>{{ count }}</p>
</template>
```

Émettre des événements

```vue
<script setup>
const emit = defineEmits(['update', 'delete']);

const handleClick = () => {
  emit('update', { id: 1, value: 'new' });
};

const handleDelete = () => {
  emit('delete', props.count);
};
</script>

<template>
  <button @click="handleClick">Update</button>
  <button @click="handleDelete">Delete</button>
</template>
```

Revenir au sommaire

Provide / Inject avec Composition API

Fournir des données (composant parent)

```vue
<script setup>
import { ref, provide } from 'vue';

const theme = ref('dark');
const updateTheme = (newTheme) => {
  theme.value = newTheme;
};

// Provide
provide('theme', theme);
provide('updateTheme', updateTheme);
</script>
```

Injecter des données (composant enfant)

```vue
<script setup>
import { inject } from 'vue';

// Inject
const theme = inject('theme');
const updateTheme = inject('updateTheme');

const toggleTheme = () => {
  updateTheme(theme.value === 'dark' ? 'light' : 'dark');
};
</script>

<template>
  <button @click="toggleTheme">
    Thème actuel : {{ theme }}
  </button>
</template>
```

Revenir au sommaire

Avantages de la Composition API

🎯 Organisation par fonctionnalité

```
<!-- Options API : Logique dispersée -->
data() { return { userX, cartY, themeZ } }
methods { userMethod(), cartMethod(), themeMethod() }
computed { userComputed, cartComputed }
```

```
<!-- Composition API : Logique groupée -->
// Feature User
const { user, updateUser } = useUser();

// Feature Cart
const { cart, addToCart } = useCart();

// Feature Theme
const { theme, toggleTheme } = useTheme();
```

✨ Réutilisabilité

Options API : Mixins (conflits possibles, origine floue) **Composition API** : Composables (clair, explicite, pas de conflit)

🔥 TypeScript

Support TypeScript EXCELLENT (types automatiquement inférés)

Revenir au sommaire

Migration Options API → Composition API

Étape 1 : Convertir data()

```
<!-- Options API -->
data() {
  return {
    count: 0,
    message: 'Hello'
  }
}

<!-- Composition API -->
import { ref } from 'vue';

const count = ref(0);
const message = ref('Hello');
```

Étape 2 : Convertir methods

```
<!-- Options API -->
methods: {
  increment() {
    this.count++;
  }
}

<!-- Composition API -->
const increment = () => {
  count.value++;
};
```

Revenir au sommaire

Migration (suite)

Étape 3 : Convertir computed

```
<!-- Options API -->
computed: {
  double() {
    return this.count * 2;
  }
}

<!-- Composition API -->
import { computed } from 'vue';

const double = computed(() => count.value * 2);
```

Étape 4 : Convertir lifecycle

```
<!-- Options API -->
mounted() {
  this.fetchData();
}

<!-- Composition API -->
import { onMounted } from 'vue';

onMounted(() => {
  fetchData();
});
```



Revenir au sommaire

Exemple complet : Options API → Composition API

Options API (avant)

```
<script>
export default {
  data() {
    return {
      count: 0,
      message: 'Hello'
    }
  },
  computed: {
    double() {
      return this.count * 2;
    }
  },
  methods: {
    increment() {
      this.count++;
    }
  },
  mounted() {
    console.log('Component mounted');
  }
}
</script>

<template>
  <div>
    <p>{{ count }} (double: {{ double }})</p>
    <button @click="increment">+</button>
  </div>
</template>
```

Revenir au sommaire

Exemple complet : Composition API (après)

Composition API (après)

```vue
<script setup>
import { ref, computed, onMounted } from 'vue';

const count = ref(0);
const message = ref('Hello');

const double = computed(() => count.value * 2);

const increment = () => {
  count.value++;
};

onMounted(() => {
  console.log('Component mounted');
});
</script>

<template>
  <div>
    <p>{{ count }} (double: {{ double }})</p>
    <button @click="increment">+</button>
  </div>
</template>
```

📊 Résultat

- Moins de code
- Plus lisible
- Prêt pour être extrait en composable

Revenir au sommaire

Bonnes pratiques Composition API

1. Nommage des composables

✅ **Bon :** `useCounter` , `useFetch` , `useAuth` ❌ **Mauvais :** `counter` , `fetch` , `auth`

2. Retourner un objet depuis les composables

```
// ✅ Bon
export function useCounter() {
  const count = ref(0);
  const increment = () => count.value++;

  return { count, increment };
}

// ❌ Mauvais
export function useCounter() {
  const count = ref(0);
  return count; // Pas extensible
}
```

3. Éviter les refs inutiles

```
// ✅ Bon
const double = computed(() => count.value * 2);

// ❌ Mauvais
const double = ref(count.value * 2); // Ne se met pas à jour !
```

Revenir au sommaire

Exercice pratique : Créer un composable useLocalStorage

Objectif

Créer un composable qui synchronise une valeur avec le localStorage

Fonctionnalités

- Lire la valeur du localStorage au chargement
- Sauvegarder automatiquement à chaque changement
- Support d'une valeur par défaut

Solution attendue

```vue
<script setup>
import { useLocalStorage } from './composables/useLocalStorage';

const theme = useLocalStorage('theme', 'light');
const username = useLocalStorage('username', 'Anonymous');
</script>

<template>
  <div>
    <p>Thème: {{ theme }}</p>
    <button @click="theme = theme === 'dark' ? 'light' : 'dark'">
      Toggle theme
    </button>
  </div>
</template>
```

Revenir au sommaire

Solution : useLocalStorage

```
// composables/useLocalStorage.js
import { ref, watch } from 'vue';

export function useLocalStorage(key, defaultValue) {
  // Lire la valeur initiale du localStorage
  const storedValue = localStorage.getItem(key);
  const value = ref(storedValue ? JSON.parse(storedValue) : defaultValue);

  // Sauvegarder dans le localStorage à chaque changement
  watch(value, (newValue) => {
    localStorage.setItem(key, JSON.stringify(newValue));
  }, { deep: true });

  return value;
}
```

💡 Améliorations possibles

- Gérer les erreurs de parsing JSON
- Support de la synchronisation entre onglets
- Option pour désactiver la sauvegarde automatique



Revenir au sommaire

Ressources et documentation

📚 Documentation officielle

- Vue 3 Composition API
- Script Setup
- Reactivity API

🎓 Tutoriels recommandés

- VueSchool - Composition API
- VueMastery - Composition API

🛠️ Outils

- VueUse - Collection de composables utiles
- Vue DevTools - Support Composition API

Revenir au sommaire

Récapitulatif

Ce que vous avez appris

- ✅ Options API vs Composition API
- ✅ `ref()` et `reactive()`
- ✅ `computed()` et watchers
- ✅ Lifecycle hooks en Composition API
- ✅ Composables et réutilisabilité
- ✅ Props, events, provide/inject

🎯 Quand utiliser la Composition API ?

- Projets moyens à grands
- Logique complexe à réutiliser
- Avec TypeScript
- Code organisé par fonctionnalité

💭 Réflexion finale

La Composition API n'est PAS obligatoire. Options API reste valide et maintenue. **Choisissez l'outil qui convient le mieux à votre projet et votre équipe !**



Revenir au sommaire

🎉 Félicitations !

Vous maîtrisez maintenant **les deux approches** pour développer avec Vue.js !

🚀 Prochaines étapes

1. Expérimentez avec la Composition API dans vos projets
2. Créez vos propres composables
3. Explorez VueUse pour des composables prêts à l'emploi
4. Migrez progressivement si vous avez un projet existant

💡 Conseil final

Ne migrez pas un projet entier d'un coup ! Options API et Composition API peuvent coexister dans le même projet.

Migrez composant par composant, feature par feature.

Revenir au sommaire

Questions ?

N'hésitez pas à poser vos questions sur la Composition API !

Bon code avec Vue.js ! 🎊

Revenir au sommaire

Module 7

CI/CD avec GitLab 🚀

Intégration Continue et Déploiement Continu

Revenir au sommaire

LIENS UTILES

💪 Exercices Pratiques CI/CD

📝 QCM CI/CD

Revenir au sommaire

Qu'est-ce que le CI/CD ? 🔄

CI/CD = Continuous Integration / Continuous Deployment

Revenir au sommaire

Continuous Integration (CI) 🔨

- **Intégration automatique** du code dans la branche principale
- **Tests automatiques** à chaque commit/merge
- **Détection précoce** des bugs et conflits
- **Build automatique** de l'application

Revenir au sommaire

Continuous Deployment (CD) 📦

- **Déploiement automatique** vers différents environnements
- **Livraison continue** des fonctionnalités
- **Réduction des erreurs** humaines
- **Mises en production rapides**

Revenir au sommaire

Avantages du CI/CD 💡

- ✅ **Qualité du code** : Tests automatiques à chaque changement
- ✅ **Rapidité** : Déploiements plus fréquents et rapides
- ✅ **Fiabilité** : Processus reproductible et standardisé
- ✅ **Traçabilité** : Historique complet des déploiements
- ✅ **Feedback rapide** : Détection immédiate des problèmes
- ✅ **Collaboration** : Facilite le travail en équipe

Revenir au sommaire

GitLab CI/CD 🦊

GitLab CI/CD est intégré nativement dans GitLab

Revenir au sommaire

Caractéristiques principales

- Configuration via fichier `.gitlab-ci.yml`
- Pipelines visuels dans l'interface
- Runners pour exécuter les jobs
- Variables d'environnement sécurisées
- Intégration avec Docker
- Déploiement vers multiples environnements

Revenir au sommaire

Architecture GitLab CI/CD 🏗️



Revenir au sommaire

Concepts clés 📚

Pipeline

Une **pipeline** est une séquence de stages qui s'exécutent automatiquement

```
stages:
  - build
  - test
  - deploy
```

Revenir au sommaire

Job

Un **job** est une tâche unitaire à exécuter

```yaml
build-job:
  stage: build
  script:
    - npm install
    - npm run build
```

Revenir au sommaire

Stage

Un **stage** regroupe plusieurs jobs qui s'exécutent en parallèle

```
test:unit:
  stage: test
  script:
    - npm run test:unit

test:e2e:
  stage: test
  script:
    - npm run test:e2e
```

Revenir au sommaire

Runner

Un **runner** est un agent qui exécute les jobs de la pipeline

Types de runners :

- **Shared runners** : Fournis par GitLab
- **Specific runners** : Dédiés à un projet
- **Group runners** : Partagés dans un groupe de projets

Revenir au sommaire

Fichier .gitlab-ci.yml 

Le fichier `.gitlab-ci.yml` à la racine du projet définit la pipeline

Revenir au sommaire

Structure de base

```yaml
image: node:20

stages:
  - build
  - test
  - deploy

before_script:
  - echo "Début du job"

after_script:
  - echo "Fin du job"
```

Revenir au sommaire

Variables globales

```yaml
variables:
  NODE_ENV: "production"
  APP_NAME: "taskflow"
  DEPLOY_SERVER: "prod.example.com"
```

Revenir au sommaire

Cache et Artifacts

```yaml
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .npm/
```



Revenir au sommaire

Revenir au sommaire

Pipeline pour Vue.js 🎯

Configuration complète - Partie 1

```yaml
image: node:20

stages:
  - install
  - lint
  - test
  - build
  - deploy
```

Revenir au sommaire

Configuration complète - Partie 2

```yaml
variables:
  NODE_ENV: "production"
  npm_config_cache: "$CI_PROJECT_DIR/.npm"
  CYPRESS_CACHE_FOLDER: "$CI_PROJECT_DIR/.cypress"

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .npm/
    - .cypress/
```



Revenir au sommaire

Job : Installation des dépendances

```yaml
install_dependencies:
  stage: install
  script:
    - npm ci --prefer-offline
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day
  only:
    - merge_requests
    - main
    - develop
```



Revenir au sommaire

Job : Linting du code

```yaml
lint:code:
  stage: lint
  dependencies:
    - install_dependencies
  script:
    - npm run lint
  only:
    - merge_requests
    - main
    - develop
```

Revenir au sommaire

Job : Tests unitaires

```yaml
test:unit:
  stage: test
  dependencies:
    - install_dependencies
  script:
    - npm run test:unit -- --coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
    paths:
      - coverage/
    expire_in: 30 days
```



Revenir au sommaire

Job : Tests E2E

```yaml
test:e2e:
  stage: test
  dependencies:
    - install_dependencies
  script:
    - npm run test:e2e:ci
  artifacts:
    when: on_failure
    paths:
      - cypress/screenshots/
      - cypress/videos/
    expire_in: 7 days
  only:
    - merge_requests
    - main
```



Revenir au sommaire

Job : Build de l'application

```yaml
build:app:
  stage: build
  dependencies:
    - install_dependencies
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
  only:
    - main
    - develop
    - tags
```

Revenir au sommaire

Job : Déploiement en staging

```yaml
deploy:staging:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: staging
    url: https://staging.taskflow.com
  script:
    - echo "Déploiement sur staging"
    - npm run deploy:staging
  only:
    - develop
```

Revenir au sommaire

Job : Déploiement en production

```yaml
deploy:production:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: production
    url: https://taskflow.com
  script:
    - echo "Déploiement en production"
    - npm run deploy:prod
  only:
    - tags
  when: manual
```

Revenir au sommaire

Variables d'environnement 🔐

Types de variables

GitLab CI/CD supporte plusieurs types de variables :

- **Variables de projet** : Spécifiques à un projet
- **Variables de groupe** : Partagées dans un groupe
- **Variables d'instance** : Globales à l'instance GitLab
- **Variables prédéfinies** : Fournies par GitLab

Revenir au sommaire

Variables prédéfinies utiles

```yaml
deploy:
  script:
    - echo "Branch: $CI_COMMIT_BRANCH"
    - echo "Commit: $CI_COMMIT_SHA"
    - echo "Job ID: $CI_JOB_ID"
    - echo "Pipeline ID: $CI_PIPELINE_ID"
    - echo "Project: $CI_PROJECT_NAME"
    - echo "URL: $CI_PROJECT_URL"
```



Revenir au sommaire

Ajouter des variables protégées

Dans GitLab :

1. **Settings** → **CI/CD** → **Variables**
2. Cliquer sur **Add variable**
3. Définir la clé et la valeur
4. Cocher **Protected** pour les branches protégées
5. Cocher **Masked** pour masquer dans les logs

Revenir au sommaire

Utilisation des variables

```
deploy:production:
  script:
    - echo "Deploying to $DEPLOY_SERVER"
    - scp -i $SSH_PRIVATE_KEY dist/* user@$DEPLOY_SERVER:/var/www/
  variables:
    DEPLOY_SERVER: "prod.example.com"
  only:
    - tags
```



Revenir au sommaire

Stratégies de déploiement 🎯

1. Déploiement manuel

```yaml
deploy:production:
  stage: deploy
  script:
    - npm run deploy:prod
  when: manual
  only:
    - main
```

Le job nécessite une action manuelle pour s'exécuter

Revenir au sommaire

2. Déploiement automatique

```yaml
deploy:staging:
  stage: deploy
  script:
    - npm run deploy:staging
  only:
    - develop
```

Le job s'exécute automatiquement

Revenir au sommaire

3. Déploiement par tags

```yaml
deploy:production:
  stage: deploy
  script:
    - npm run deploy:prod
  only:
    - tags
  except:
    - branches
```

Déploiement uniquement lors de la création d'un tag

Revenir au sommaire

4. Environnements multiples - Template

```
# Définition du template de déploiement
.deploy_template:
  stage: deploy
  script:
    - npm run build
    - echo "Deploying to $ENVIRONMENT"
```

Revenir au sommaire

4. Environnements multiples - Development

```yaml
# Utilisation du template pour dev
deploy:dev:
  extends: .deploy_template
  variables:
    ENVIRONMENT: "development"
  environment:
    name: development
  only:
    - develop
```

Revenir au sommaire

4. Environnements multiples - Staging

```yaml
# Utilisation du template pour staging
deploy:staging:
  extends: .deploy_template
  variables:
    ENVIRONMENT: "staging"
  environment:
    name: staging
  only:
    - main
```

Revenir au sommaire

4. Environnements multiples - Production

```yaml
# Utilisation du template pour production
deploy:prod:
  extends: .deploy_template
  variables:
    ENVIRONMENT: "production"
  environment:
    name: production
  when: manual
  only:
    - tags
```

Revenir au sommaire

Docker et GitLab CI/CD 🐳

Utiliser une image Docker personnalisée

```yaml
image: node:20-alpine

variables:
  NODE_ENV: production

build:
  script:
    - apk add --no-cache git
    - npm ci
    - npm run build
```

Revenir au sommaire

Build d'une image Docker

```yaml
build:docker:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG
```



Revenir au sommaire

Dockerfile pour Vue.js - Partie 1

```
# Stage 1: Build
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production
```

Revenir au sommaire

Dockerfile pour Vue.js - Partie 2

```
COPY . .
RUN npm run build

# Stage 2: Production
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```



Revenir au sommaire

Tests automatisés 🧪

Configuration Vitest pour CI

```yaml
test:unit:
  stage: test
  script:
    - npm run test:unit -- --run --coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    reports:
      junit: coverage/junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```



Revenir au sommaire

Configuration package.json

```
{
  "scripts": {
    "test:unit": "vitest",
    "test:unit:ci": "vitest run --coverage --reporter=junit --reporter=default",
    "test:e2e": "cypress open",
    "test:e2e:ci": "cypress run"
  }
}
```



Revenir au sommaire

Tests E2E avec Cypress

```yaml
test:e2e:
  stage: test
  image: cypress/browsers:node20.11.0-chrome121.0.6167.85-1
  script:
    - npm ci
    - npm run build
    - npm run test:e2e:ci
  artifacts:
    when: on_failure
    paths:
      - cypress/screenshots/
      - cypress/videos/
    expire_in: 7 days
```

Revenir au sommaire

Optimisations des pipelines ⚡

1. Utiliser le cache efficacement

```yaml
cache:
  key:
    files:
      - package-lock.json
  paths:
    - node_modules/
    - .npm/
  policy: pull-push

build:
  cache:
    policy: pull
  script:
    - npm run build
```

Revenir au sommaire

2. Paralléliser les jobs

```yaml
test:unit:
  stage: test
  script:
    - npm run test:unit

test:e2e:
  stage: test
  script:
    - npm run test:e2e

lint:code:
  stage: test
  script:
    - npm run lint
```

Les 3 jobs s'exécutent en parallèle

Revenir au sommaire

3. Utiliser des règles intelligentes

```yaml
build:
  script:
    - npm run build
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - when: never
```

Revenir au sommaire

4. Limiter l'exécution des jobs

```yaml
test:expensive:
  script:
    - npm run test:e2e
  only:
    changes:
      - src/**/*.vue
      - src/**/*.js
      - tests/**/*
  except:
    - README.md
    - docs/**/*
```

Revenir au sommaire

5. Utiliser des templates - Définition

```yaml
# Définition du template
.test_template:
  stage: test
  image: node:20
  before_script:
    - npm ci
  only:
    - merge_requests
    - main
```

Revenir au sommaire

5. Utiliser des templates - Utilisation

```yaml
# Utilisation du template
test:unit:
  extends: .test_template
  script:
    - npm run test:unit

test:integration:
  extends: .test_template
  script:
    - npm run test:integration
```

Revenir au sommaire

Déploiement sur différentes plateformes 🌐

Déploiement sur Netlify

```yaml
deploy:netlify:
  stage: deploy
  image: node:20
  script:
    - npm install -g netlify-cli
    - npm run build
    - netlify deploy --prod --dir=dist --auth=$NETLIFY_AUTH_TOKEN --site=$NETLIFY_SITE_ID
  only:
    - main
  environment:
    name: production
    url: https://taskflow.netlify.app
```

Revenir au sommaire

Déploiement sur Vercel

```yaml
deploy:vercel:
  stage: deploy
  image: node:20
  script:
    - npm install -g vercel
    - vercel --prod --token=$VERCEL_TOKEN --confirm
  only:
    - main
  environment:
    name: production
    url: https://taskflow.vercel.app
```

Revenir au sommaire

Déploiement sur serveur SSH

```yaml
deploy:ssh:
  stage: deploy
  before_script:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client -y )'
    - eval $(ssh-agent -s)
    - echo "$SSH_PRIVATE_KEY" | tr -d '\r' | ssh-add -
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh -o StrictHostKeyChecking=no user@$DEPLOY_SERVER "cd /var/www && git pull"
    - ssh user@$DEPLOY_SERVER "cd /var/www && npm install && npm run build"
  only:
    - main
```



Revenir au sommaire

Déploiement sur AWS S3

```yaml
deploy:aws:
  stage: deploy
  image: python:3.12
  before_script:
    - pip install awscli
  script:
    - npm run build
    - aws s3 sync dist/ s3://$S3_BUCKET --delete
    - aws cloudfront create-invalidation --distribution-id $CLOUDFRONT_ID --paths "/*"
  only:
    - main
  environment:
    name: production
    url: https://taskflow.com
```

Revenir au sommaire

Monitoring et notifications 📊

Notifications Slack

```yaml
notify:slack:
  stage: .post
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{\"text\":\"✅ Pipeline réussie pour $CI_PROJECT_NAME ($CI_COMMIT_BRANCH)\"}" \
      $SLACK_WEBHOOK_URL
  when: on_success
  only:
    - main

notify:slack:failure:
  stage: .post
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{\"text\":\"❌ Pipeline échouée pour $CI_PROJECT_NAME ($CI_COMMIT_BRANCH)\"}" \
      $SLACK_WEBHOOK_URL
  when: on_failure
```



Revenir au sommaire

Notifications par email

Dans GitLab : **Settings** → **Integrations** → **Emails on push**

Configuration :

- Recipients : emails des destinataires
- Disable code diffs : masquer les diffs
- Send from committer : envoyer depuis l'auteur du commit
- Push events : notifications sur push
- Tag push events : notifications sur tags

Revenir au sommaire

Métriques de pipeline

GitLab fournit automatiquement :

- **Durée d'exécution** de la pipeline
- **Taux de succès** des pipelines
- **Temps d'exécution** par job
- **Historique** des déploiements
- **Couverture de code**

Accessible via : **CI/CD** → **Pipelines** → **Charts**

Revenir au sommaire

Sécurité dans les pipelines 🔒

1. Scan de dépendances

```yaml
security:dependencies:
  stage: test
  script:
    - npm audit --audit-level=high
  allow_failure: true
```

Revenir au sommaire

2. Scan de vulnérabilités

```yaml
include:
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml

dependency_scanning:
  stage: test

sast:
  stage: test

secret_detection:
  stage: test
```



Revenir au sommaire

3. Analyse de code statique

```yaml
code_quality:
  stage: test
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_DRIVER: overlay2
  script:
    - docker run --rm --volume "$PWD":/code codeclimate/codeclimate analyze
  artifacts:
    reports:
      codequality: gl-code-quality-report.json
```



Revenir au sommaire

4. Vérification des secrets

```yaml
secrets:check:
  stage: test
  image: trufflesecurity/trufflehog:latest
  script:
    - trufflehog filesystem . --json --fail
  allow_failure: false
```

Revenir au sommaire

Bonnes pratiques CI/CD 💎

1. Organisation des stages

✅ **Ordre logique :**

```yaml
stages:
  - install      # Installation des dépendances
  - lint         # Vérification du code
  - test         # Tests automatisés
  - build        # Build de l'application
  - security     # Scans de sécurité
  - deploy       # Déploiement
```

Revenir au sommaire

2. Nommage des jobs

✅ **Noms explicites :**

```yaml
install:dependencies:     # ✅ Clair
lint:eslint:              # ✅ Précis
test:unit:vitest:         # ✅ Détaillé
build:production:         # ✅ Contexte

install:                  # ❌ Trop vague
check:                    # ❌ Ambigu
run:                      # ❌ Pas informatif
```

Revenir au sommaire

3. Gestion des artifacts

✅ **Conserver uniquement le nécessaire** :

```yaml
build:
  artifacts:
    paths:
      - dist/              # ✅ Résultat du build
    expire_in: 1 week      # ✅ Nettoyage auto

test:
  artifacts:
    paths:
      - coverage/          # ✅ Rapport de couverture
      - node_modules/      # ❌ Trop lourd, utiliser cache
    expire_in: 30 days     # ❌ Trop long pour des tests
```

Revenir au sommaire

4. Utilisation du cache

✅ Cache intelligent :

```yaml
cache:
  key:
    files:
      - package-lock.json  # ✅ Clé basée sur les dépendances
  paths:
    - node_modules/        # ✅ Cache des modules
    - .npm/                # ✅ Cache npm
  policy: pull-push        # ✅ Mise à jour si nécessaire
```

Revenir au sommaire

5. Variables sensibles

✅ **Sécurité des secrets** :

- Utiliser les variables GitLab (masked + protected)
- Ne jamais commiter de secrets dans le code
- Utiliser des outils comme HashiCorp Vault
- Limiter l'accès aux variables sensibles

❌ **À éviter** :

```
deploy:
  script:
    - echo "API_KEY=abc123"  # ❌ Visible dans les logs
    - export PASSWORD=secret # ❌ Exposé
```

Revenir au sommaire

6. Feedback rapide

✅ **Pipeline efficace :**

```yaml
stages:
  - quick-checks  # Lint, format (30s)
  - test          # Tests unitaires (2min)
  - build         # Build (1min)
  - slow-tests    # Tests E2E (10min)
  - deploy        # Déploiement
```

Placer les tests rapides en premier pour un feedback immédiat

Revenir au sommaire

7. Documentation

✅ Commenter le .gitlab-ci.yml :

```yaml
# Installation des dépendances
# Cache les node_modules pour accélérer les builds suivants
install:dependencies:
  stage: install
  script:
    - npm ci --prefer-offline
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths:
      - node_modules/
```

Revenir au sommaire

8. Gestion des erreurs

✅ Anticiper les échecs :

```yaml
test:flaky:
  script:
    - npm run test:e2e
  retry:
    max: 2                          # ✅ Retry en cas d'échec
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
  allow_failure: true               # ✅ N'empêche pas la suite
```

Revenir au sommaire

Exemple complet : Pipeline TaskFlow 🎯

Configuration complète - Partie 1

```yaml
# Pipeline CI/CD pour TaskFlow (Application Vue.js 3)
# Auteur: Formation Vue.js 2025
# Description: Pipeline complète avec tests, build et déploiement multi-environnements

image: node:20-alpine

stages:
  - install
  - lint
  - test
  - security
  - build
  - deploy
  - notify
```

Revenir au sommaire

Configuration complète - Partie 2

```yaml
# Variables globales
variables:
  NODE_ENV: "production"
  npm_config_cache: "$CI_PROJECT_DIR/.npm"
  CYPRESS_CACHE_FOLDER: "$CI_PROJECT_DIR/.cypress"
  FF_USE_FASTZIP: "true"
  ARTIFACT_COMPRESSION_LEVEL: "fast"
  CACHE_COMPRESSION_LEVEL: "fast"
```



Revenir au sommaire

Configuration complète - Partie 3

```yaml
# Cache global
cache:
  key:
    files:
      - package-lock.json
  paths:
    - node_modules/
    - .npm/
    - .cypress/
  policy: pull-push
```

Revenir au sommaire

Configuration complète - Partie 4

```yaml
# Template pour les jobs de test
.test_template:
  stage: test
  dependencies:
    - install:dependencies
  only:
    - merge_requests
    - main
    - develop
  except:
    changes:
      - "*.md"
      - docs/**/*
```

Revenir au sommaire

Job : Installation

```yaml
install:dependencies:
  stage: install
  script:
    - echo "📦 Installation des dépendances..."
    - npm ci --prefer-offline --no-audit
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day
  cache:
    policy: pull-push
  only:
    - merge_requests
    - main
    - develop
```



Revenir au sommaire

Job : Lint ESLint

```yaml
lint:eslint:
  stage: lint
  dependencies:
    - install:dependencies
  script:
    - echo "🔍 Analyse ESLint..."
    - npm run lint
  cache:
    policy: pull
  only:
    - merge_requests
    - main
    - develop
```

Revenir au sommaire

Job : Lint Prettier

```yaml
lint:prettier:
  stage: lint
  dependencies:
    - install:dependencies
  script:
    - echo "💅 Vérification du formatage..."
    - npx prettier --check "src/**/*.{js,vue,css,json}"
  cache:
    policy: pull
  only:
    - merge_requests
    - main
    - develop
```

Revenir au sommaire

Job : Tests unitaires

```yaml
test:unit:
  extends: .test_template
  script:
    - echo "🧪 Exécution des tests unitaires..."
    - npm run test:unit -- --run --coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    reports:
      junit: coverage/junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
    paths:
      - coverage/
    expire_in: 30 days
```

Revenir au sommaire

Job : Tests E2E

```yaml
test:e2e:
  extends: .test_template
  image: cypress/browsers:node20.11.0-chrome121.0.6167.85-1
  script:
    - echo "🚀 Exécution des tests E2E..."
    - npm run build
    - npm run test:e2e:ci
  artifacts:
    when: on_failure
    paths:
      - cypress/screenshots/
      - cypress/videos/
    expire_in: 7 days
  allow_failure: true
```

Revenir au sommaire

Job : Audit de sécurité

```yaml
security:audit:
  stage: security
  script:
    - echo "🔒 Audit de sécurité des dépendances..."
    - npm audit --audit-level=moderate
  allow_failure: true
  only:
    - merge_requests
    - main
```



Revenir au sommaire

Job : Scan de secrets

```yaml
security:secrets:
  stage: security
  image: trufflesecurity/trufflehog:latest
  script:
    - echo "🕵️ Recherche de secrets exposés..."
    - trufflehog filesystem . --json
  allow_failure: true
  only:
    - merge_requests
    - main
```

Revenir au sommaire

Job : Build

```yaml
build:app:
  stage: build
  dependencies:
    - install:dependencies
  script:
    - echo "🏗️ Build de l'application..."
    - npm run build
    - echo "📊 Taille du build:"
    - du -sh dist/
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
  cache:
    policy: pull
  only:
    - main
    - develop
    - tags
```

Revenir au sommaire

Job : Déploiement Development

```yaml
deploy:development:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: development
    url: https://dev.taskflow.com
    on_stop: stop:development
  script:
    - echo "🚀 Déploiement sur Development..."
    - npm run deploy:dev
  only:
    - develop
```

Revenir au sommaire

Job : Déploiement Staging

```yaml
deploy:staging:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: staging
    url: https://staging.taskflow.com
    on_stop: stop:staging
  script:
    - echo "🚀 Déploiement sur Staging..."
    - npm run deploy:staging
  only:
    - main
```



Revenir au sommaire

Job : Déploiement Production

```yaml
deploy:production:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: production
    url: https://taskflow.com
  script:
    - echo "🚀 Déploiement en Production..."
    - npm run deploy:prod
    - echo "✅ Application déployée avec succès!"
  only:
    - tags
  when: manual
```

Revenir au sommaire

Job : Arrêt environnement Dev

```yaml
stop:development:
  stage: deploy
  environment:
    name: development
    action: stop
  script:
    - echo "🛑 Arrêt de l'environnement Development..."
    - npm run stop:dev
  when: manual
  only:
    - develop
```

Revenir au sommaire

Job : Arrêt environnement Staging

```yaml
stop:staging:
  stage: deploy
  environment:
    name: staging
    action: stop
  script:
    - echo "🔴 Arrêt de l'environnement Staging..."
    - npm run stop:staging
  when: manual
  only:
    - main
```

Revenir au sommaire

Job : Notification succès

```yaml
notify:success:
  stage: notify
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{\"text\":\"✅ Pipeline réussie pour TaskFlow\n• Branch: $CI_COMMIT_BRANCH\n• Commit: $CI_COMMIT_SHORT_SHA\n• Auteur: $CI_COMMIT_AUTHOR\"}" \
      $SLACK_WEBHOOK_URL
  when: on_success
  only:
    - main
    - develop
```

Revenir au sommaire

Job : Notification échec

```yaml
notify:failure:
  stage: notify
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{\"text\":\"❌ Pipeline échouée pour TaskFlow\n• Branch: $CI_COMMIT_BRANCH\n• Commit: $CI_COMMIT_SHORT_SHA\n• Job: $CI_JOB_NAME\"}" \
      $SLACK_WEBHOOK_URL
  when: on_failure
  only:
    - main
    - develop
```

Revenir au sommaire

Workflows avancés 🔀

Workflow 1 : Feature Branch

```yaml
workflow:
  rules:
    # Pipelines pour merge requests
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      variables:
        ENVIRONMENT: "review"
    # Pipelines pour develop
    - if: '$CI_COMMIT_BRANCH == "develop"'
      variables:
        ENVIRONMENT: "development"
    # Pipelines pour main
    - if: '$CI_COMMIT_BRANCH == "main"'
      variables:
        ENVIRONMENT: "staging"
```

Revenir au sommaire

Workflow 2 : GitFlow

```yaml
# Développement
develop:
  only:
    - develop
  script:
    - npm run deploy:dev
  environment:
    name: development

# Release candidates
release:
  only:
    - /^release\/.*$/
  script:
    - npm run deploy:staging
  environment:
    name: staging
```



Revenir au sommaire

Revenir au sommaire

Workflow 3 : Trunk-Based Development

```yaml
workflow:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

# Tous les commits sur main déclenchent:
# 1. Tests complets
# 2. Build
# 3. Déploiement automatique en staging
# 4. Déploiement manuel en production

deploy:staging:
  when: on_success  # Automatique

deploy:production:
  when: manual      # Manuel après validation
```



Revenir au sommaire

Review Apps 🔍

Qu'est-ce qu'une Review App ?

Une **Review App** est un environnement éphémère créé automatiquement pour chaque merge request

Avantages :

- Tester les changements avant le merge
- Partager facilement avec l'équipe
- Détection précoce des problèmes
- Feedback visuel immédiat



Revenir au sommaire

Configuration Review App

```yaml
review:
  stage: deploy
  script:
    - echo "🔍 Création de la Review App..."
    - npm run build
    - npm run deploy:review
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_COMMIT_REF_SLUG.review.taskflow.com
    on_stop: stop_review
    auto_stop_in: 1 week
  only:
    - merge_requests
```



Revenir au sommaire

Arrêt automatique Review App

```yaml
stop_review:
  stage: deploy
  script:
    - echo "🛑 Suppression de la Review App..."
    - npm run destroy:review
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
  only:
    - merge_requests
```



Revenir au sommaire

Déploiement progressif 📈

Blue-Green Deployment

```yaml
deploy:blue:
  stage: deploy
  script:
    - npm run deploy:blue
  environment:
    name: production-blue
    url: https://blue.taskflow.com

deploy:green:
  stage: deploy
  script:
    - npm run deploy:green
  environment:
    name: production-green
    url: https://green.taskflow.com
  when: manual
```

Revenir au sommaire

Revenir au sommaire

Canary Deployment

```yaml
deploy:canary:
  stage: deploy
  script:
    - echo "🐤 Déploiement Canary (10% du trafic)..."
    - npm run deploy:canary --percentage=10
  environment:
    name: production-canary
  when: manual

deploy:full:
  stage: deploy
  script:
    - echo "🚀 Déploiement complet (100%)..."
    - npm run deploy:full
  environment:
    name: production
  when: manual
  needs:
    - deploy:canary
```

Revenir au sommaire

Rollback et gestion d'incidents

Rollback automatique

```
deploy:production:
  stage: deploy
  script:
    - npm run deploy:prod
    - npm run health:check || npm run rollback:prod
  environment:
    name: production
  only:
    - tags
```

Revenir au sommaire

Rollback manuel

```yaml
rollback:production:
  stage: deploy
  script:
    - echo "⏮ Rollback vers la version précédente..."
    - git checkout HEAD~1
    - npm run build
    - npm run deploy:prod
  environment:
    name: production
  when: manual
  only:
    - main
```

Revenir au sommaire

Gestion des incidents

```yaml
hotfix:emergency:
  stage: deploy
  script:
    - echo "🚨 Déploiement d'urgence..."
    - npm run build
    - npm run deploy:prod --force
  environment:
    name: production
  when: manual
  only:
    - hotfix/*
```



Revenir au sommaire

Exercice pratique : Pipeline Vue.js 🎯

Objectif

Créer une pipeline CI/CD complète pour l'application TaskFlow avec :

1. Installation et cache des dépendances
2. Lint du code (ESLint)
3. Tests unitaires avec couverture
4. Build de l'application
5. Déploiement sur Netlify (staging et production)

Revenir au sommaire

Étape 1 : Créer le fichier .gitlab-ci.yml

À la racine du projet TaskFlow, créer `.gitlab-ci.yml`

Revenir au sommaire

Étape 2 : Configuration de base

```yaml
image: node:20

stages:
  - install
  - lint
  - test
  - build
  - deploy

variables:
  npm_config_cache: "$CI_PROJECT_DIR/.npm"

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .npm/
```

Revenir au sommaire

Étape 3 : Job d'installation

```yaml
install:dependencies:
  stage: install
  script:
    - npm ci --prefer-offline
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day
```



Revenir au sommaire

Étape 4 : Job de lint

```yaml
lint:code:
  stage: lint
  dependencies:
    - install:dependencies
  script:
    - npm run lint
```

Revenir au sommaire

Étape 5 : Job de tests

```yaml
test:unit:
  stage: test
  dependencies:
    - install:dependencies
  script:
    - npm run test:unit -- --run --coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    paths:
      - coverage/
    expire_in: 30 days
```

Revenir au sommaire

Étape 6 : Job de build

```yaml
build:app:
  stage: build
  dependencies:
    - install:dependencies
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
  only:
    - main
    - develop
```



Revenir au sommaire

Étape 7 : Déploiement Staging

```yaml
deploy:staging:
  stage: deploy
  dependencies:
    - build:app
  script:
    - npm install -g netlify-cli
    - netlify deploy --dir=dist --auth=$NETLIFY_AUTH_TOKEN --site=$NETLIFY_SITE_ID
  environment:
    name: staging
    url: https://staging--taskflow.netlify.app
  only:
    - develop
```

Revenir au sommaire

Étape 8 : Déploiement Production

```yaml
deploy:production:
  stage: deploy
  dependencies:
    - build:app
  script:
    - npm install -g netlify-cli
    - netlify deploy --prod --dir=dist --auth=$NETLIFY_AUTH_TOKEN --site=$NETLIFY_SITE_ID
  environment:
    name: production
    url: https://taskflow.netlify.app
  only:
    - main
  when: manual
```



Revenir au sommaire

Étape 9 : Configuration des variables

Dans GitLab :

1. Aller dans **Settings** → **CI/CD** → **Variables**
2. Ajouter les variables :
   - `NETLIFY_AUTH_TOKEN` (masked)
   - `NETLIFY_SITE_ID` (masked)

Revenir au sommaire

Étape 10 : Test de la pipeline

1. Commit et push du fichier `.gitlab-ci.yml`
2. Vérifier l'exécution dans **CI/CD** → **Pipelines**
3. Observer les logs de chaque job
4. Vérifier le déploiement sur Netlify

Revenir au sommaire

Ressources et documentation 📚

Revenir au sommaire

Documentation officielle

- **GitLab CI/CD** : https://docs.gitlab.com/ee/ci/
- **GitLab Runner** : https://docs.gitlab.com/runner/
- **Variables CI/CD** : https://docs.gitlab.com/ee/ci/variables/
- **YAML Syntax** : https://docs.gitlab.com/ee/ci/yaml/



Revenir au sommaire

Templates GitLab CI/CD

GitLab fournit des templates prêts à l'emploi :

```yaml
include:
  - template: Jobs/Code-Quality.gitlab-ci.yml
  - template: Jobs/SAST.gitlab-ci.yml
  - template: Jobs/Dependency-Scanning.gitlab-ci.yml
  - template: Jobs/Secret-Detection.gitlab-ci.yml
```

Disponibles sur : https://gitlab.com/gitlab-org/gitlab/-/tree/master/lib/gitlab/ci/templates

Revenir au sommaire

Outils complémentaires

- **GitLab Runner** : https://docs.gitlab.com/runner/install/
- **Docker** : https://docs.docker.com/
- **Kubernetes** : https://kubernetes.io/docs/
- **Helm** : https://helm.sh/docs/
- **Terraform** : https://www.terraform.io/docs/



Revenir au sommaire

Communauté et support

- **GitLab Forum** : https://forum.gitlab.com/
- **GitLab Discord** : https://discord.gg/gitlab
- **Stack Overflow** : Tag `gitlab-ci`
- **GitLab Issues** : https://gitlab.com/gitlab-org/gitlab/-/issues



Revenir au sommaire

Points clés à retenir 🎯

Revenir au sommaire

CI/CD avec GitLab

- ✅ Configuration via .gitlab-ci.yml
- ✅ Stages pour organiser la pipeline
- ✅ Jobs pour les tâches unitaires
- ✅ Cache pour optimiser les builds
- ✅ Artifacts pour partager entre jobs

Revenir au sommaire

Bonnes pratiques

- ✅ Tests automatiques à chaque commit
- ✅ Déploiements progressifs
- ✅ Variables sécurisées pour les secrets
- ✅ Review Apps pour les merge requests
- ✅ Monitoring et notifications

Revenir au sommaire

Sécurité

- ✅ Scan de dépendances
- ✅ Détection de secrets
- ✅ Analyse de code statique
- ✅ Variables masked et protected

Revenir au sommaire

Conclusion 🎉

Ce que vous avez appris

- ✅ Concepts du CI/CD
- ✅ Configuration GitLab CI/CD
- ✅ Création de pipelines complètes
- ✅ Déploiement multi-environnements
- ✅ Optimisation et bonnes pratiques
- ✅ Sécurité dans les pipelines



Revenir au sommaire

Prochaines étapes

1. 🎯 **Pratiquer** : Créer des pipelines pour vos projets
2. 📖 **Approfondir** : Explorer les fonctionnalités avancées
3. 🔒 **Sécuriser** : Intégrer les scans de sécurité
4. 📊 **Monitorer** : Suivre les métriques des pipelines
5. 🚀 **Automatiser** : Aller plus loin dans l'automatisation

Revenir au sommaire

Ressources finales

- 📚 Documentation GitLab CI/CD
- 🎥 Tutoriels vidéo
- 💬 Communauté GitLab
- 🛠️ Templates officiels
- 📝 Exemples de projets

Revenir au sommaire

Questions ? 🤔

Merci d'avoir suivi ce module !

N'hésitez pas à poser vos questions sur :

- La configuration des pipelines
- Les déploiements
- Les bonnes pratiques
- Les cas d'usage spécifiques



Revenir au sommaire

À vous de jouer ! 🚀

Passez maintenant à l'exercice pratique !

Créez votre première pipeline CI/CD pour TaskFlow et déployez automatiquement votre application.

Bon courage ! 💪

Revenir au sommaire

QCM Module Bonus

CI/CD avec GitLab ✅

Validez vos connaissances !

Revenir au sommaire

Question 1

Que signifie CI/CD ?

A) Configuration Integration / Code Deployment

B) Continuous Integration / Continuous Deployment

C) Code Integration / Continuous Delivery

D) Central Integration / Cloud Deployment

Revenir au sommaire

Réponse : B

Continuous Integration / Continuous Deployment

CI/CD est l'ensemble des pratiques permettant :

- **CI** : Intégrer automatiquement le code et le tester
- **CD** : Déployer automatiquement vers les environnements

Revenir au sommaire

Question 2

Quel fichier configure la pipeline GitLab CI/CD ?

A) `gitlab-pipeline.yml`

B) `.gitlab-ci.yml`

C) `ci-config.yml`

D) `.gitlab-pipeline.yaml`

Revenir au sommaire

Réponse : B

`.gitlab-ci.yml`

C'est le fichier de configuration standard qui doit être placé à la racine du projet.

Revenir au sommaire

Question 3

Qu’est-ce qu’un Runner GitLab ?

A) Un développeur qui gère les déploiements

B) Un script qui s’exécute dans la pipeline

C) Un agent qui exécute les jobs de la pipeline

D) Un serveur de déploiement

Revenir au sommaire

Réponse : C

Un agent qui exécute les jobs de la pipeline

Les Runners sont des agents (machines physiques ou virtuelles) qui exécutent les jobs définis dans `.gitlab-ci.yml` .

Revenir au sommaire

Question 4

Quelle est la différence entre un stage et un job ?

A) Un stage est une tâche, un job est un groupe de tâches

B) Un stage regroupe des jobs qui s'exécutent en parallèle

C) Il n'y a pas de différence

D) Un job contient plusieurs stages

Revenir au sommaire

Réponse : B

Un stage regroupe des jobs qui s'exécutent en parallèle

```yaml
stages:
  - test          # Stage

test:unit:        # Job dans le stage test
  stage: test

test:e2e:         # Job dans le stage test (parallèle)
  stage: test
```

Revenir au sommaire

Question 5

À quoi sert le cache dans GitLab CI/CD ?

A) À stocker les logs de la pipeline

B) À accélérer les builds en conservant les dépendances

C) À sauvegarder les résultats des tests

D) À créer des backups du code

Revenir au sommaire

Réponse : B

À accélérer les builds en conservant les dépendances

Le cache permet de réutiliser `node_modules/` entre les pipelines pour éviter de tout réinstaller.

Revenir au sommaire

Question 6

Quelle est la différence entre cache et artifacts ?

A) Il n'y a pas de différence

B) Le cache est pour les dépendances, les artifacts pour les résultats

C) Les artifacts sont plus rapides

D) Le cache est automatique, les artifacts sont manuels

Revenir au sommaire

Réponse : B

Le cache est pour les dépendances, les artifacts pour les résultats

- **Cache** : Optimisation (node_modules, .npm)
- **Artifacts** : Résultats à partager entre jobs (dist/, coverage/)

Revenir au sommaire

Question 7

Comment limiter l'exécution d'un job à la branche main ?

A) `branch: main`

B) `only: main`

C) `when: main`

D) `if: main`

Revenir au sommaire

Réponse : B

only: main

```
deploy:
  script:
    - npm run deploy
  only:
    - main
```

Revenir au sommaire

Question 8

Que fait le mot-clé `when: manual` **?**

A) Le job s'exécute automatiquement

B) Le job nécessite une action manuelle pour démarrer

C) Le job s'exécute seulement en cas d'erreur

D) Le job est désactivé

Revenir au sommaire

Réponse : B

Le job nécessite une action manuelle pour démarrer

Utile pour les déploiements en production :

```
deploy:production:
  when: manual
```

Revenir au sommaire

Question 9

Quelle est l'utilité des Review Apps ?

A) Réviser le code avant merge

B) Créer un environnement éphémère pour tester les MR

C) Faire des revues de performance

D) Analyser les statistiques de l'application

Revenir au sommaire

Réponse : B

Créer un environnement éphémère pour tester les MR

Chaque merge request obtient son propre environnement de test accessible via URL.

Revenir au sommaire

Question 10

Comment définir une variable d'environnement dans GitLab CI ?

A) `env: NODE_ENV=production`

B) `variables: NODE_ENV: "production"`

C) `set: NODE_ENV=production`

D) `export NODE_ENV=production`

Revenir au sommaire

Réponse : B

```
variables: NODE_ENV: "production"
```

```yaml
variables:
  NODE_ENV: "production"
  APP_NAME: "taskflow"
```

Revenir au sommaire

Question 11

Quel type de variable doit-on utiliser pour les secrets ?

A) Variables publiques

B) Variables masked et protected

C) Variables cachées

D) Variables temporaires

Revenir au sommaire

Réponse : B

Variables masked et protected

- **Masked** : Masquée dans les logs
- **Protected** : Accessible uniquement sur branches protégées

Revenir au sommaire

Question 12

Que signifie le mot-clé `dependencies` dans un job ?

A) Liste les packages npm nécessaires

B) Spécifie les jobs dont on récupère les artifacts

C) Définit les jobs qui doivent s'exécuter avant

D) Liste les services Docker requis

Revenir au sommaire

Réponse : B

Spécifie les jobs dont on récupère les artifacts

```yaml
build:
  artifacts:
    paths:
      - dist/

deploy:
  dependencies:
    - build  # Récupère dist/ du job build
```

Revenir au sommaire

Question 13

À quoi sert le stage `.post` **?**

A) C'est un stage qui s'exécute toujours en dernier

B) C'est un stage pour envoyer des emails

C) C'est un stage de nettoyage

D) Il n'existe pas

Revenir au sommaire

Réponse : A

C'est un stage qui s'exécute toujours en dernier

Utile pour les notifications :

```
notify:
  stage: .post
  when: on_success
```

Revenir au sommaire

Question 14

Quelle syntaxe permet d'extraire la couverture de code ?

A) `code_coverage: '/(\d+)%/'`

B) `coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'`

C) `test_coverage: true`

D) `extract_coverage: yes`

Revenir au sommaire

Réponse : B

```
coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
```

Expression régulière pour extraire le pourcentage de couverture des logs.

Revenir au sommaire

Question 15

Comment créer un environnement dynamique pour les Review Apps ?

A) `environment: review`

B) `environment: $CI_COMMIT_REF_SLUG`

C) `environment: name: review/$CI_COMMIT_REF_SLUG`

D) `dynamic_environment: true`

Revenir au sommaire

Réponse : C

`environment: name: review/$CI_COMMIT_REF_SLUG`

```yaml
environment:
  name: review/$CI_COMMIT_REF_SLUG
  url: https://$CI_COMMIT_REF_SLUG.review.app.com
```

Revenir au sommaire

Question 16

Quelle image Docker utiliser pour les tests Cypress ?

A) `node:20`

B) `cypress/base:20`

C) `cypress/browsers:node20.11.0-chrome121`

D) `docker:24`

Revenir au sommaire

Réponse : C

```
cypress/browsers:node20.11.0-chrome121
```

Image officielle Cypress avec navigateurs pour tests E2E.

Revenir au sommaire

Question 17

Comment paralléliser des jobs dans le même stage ?

A) Utiliser `parallel: true`

B) Les jobs du même stage s'exécutent automatiquement en parallèle

C) Utiliser `concurrent: true`

D) Ce n'est pas possible

Revenir au sommaire

Réponse : B

Les jobs du même stage s'exécutent automatiquement en parallèle

```yaml
stages:
  - test

test:unit:     # S'exécutent
  stage: test # en

test:e2e:      # parallèle
  stage: test
```

Revenir au sommaire

Question 18

Que fait `allow_failure: true` **?**

A) Empêche le job de s'exécuter

B) Permet à la pipeline de continuer même si le job échoue

C) Relance automatiquement le job en cas d'échec

D) Cache les erreurs dans les logs

Revenir au sommaire

Réponse : B

Permet à la pipeline de continuer même si le job échoue

Utile pour les tests E2E flaky ou les scans de sécurité non bloquants.

Revenir au sommaire

Question 19

Comment configurer un retry automatique ?

A) `retry: 3`

B) `auto_retry: true`

C) `retry: { max: 2, when: runner_system_failure }`

D) `on_failure: retry`

Revenir au sommaire

Réponse : C

`retry: { max: 2, when: runner_system_failure }`

```yaml
test:e2e:
  script:
    - npm run test:e2e
  retry:
    max: 2
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
```

Revenir au sommaire

Question 20

Quelle est la meilleure pratique pour la clé de cache ?

A) Utiliser une clé fixe : `key: cache`

B) Utiliser la branche : `key: ${CI_COMMIT_REF_SLUG}`

C) Utiliser le hash du fichier de dépendances : `key: {files: ['package-lock.json']}`

D) Ne pas définir de clé

Revenir au sommaire

Réponse : C

Utiliser le hash du fichier de dépendances

Le cache est invalidé uniquement quand `package-lock.json` change.

```yaml
cache:
  key:
    files:
      - package-lock.json
```

Revenir au sommaire

Question 21

Comment inclure des templates GitLab officiels ?

A) `import: template:Security/SAST.gitlab-ci.yml`

B) `include: - template: Security/SAST.gitlab-ci.yml`

C) `use: Security/SAST`

D) `template: SAST`

Revenir au sommaire

Réponse : B

```
include: - template: Security/SAST.gitlab-ci.yml
```

```yaml
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
```

Revenir au sommaire

Question 22

Quelle variable contient le SHA du commit actuel ?

A) `$CI_COMMIT_ID`

B) `$CI_COMMIT_SHA`

C) `$GIT_COMMIT`

D) `$COMMIT_HASH`

Revenir au sommaire

Réponse : B

```
$CI_COMMIT_SHA
```

Variables prédéfinies utiles :

- `$CI_COMMIT_SHA` : SHA complet
- `$CI_COMMIT_SHORT_SHA` : SHA court
- `$CI_COMMIT_BRANCH` : Nom de la branche

Revenir au sommaire

Question 23

Comment exécuter un job uniquement sur les tags ?

A) `only: - tags`

B) `when: tag`

C) `if: $CI_COMMIT_TAG`

D) A et C sont correctes

Revenir au sommaire

Réponse : D

A et C sont correctes

Deux syntaxes possibles :

```yaml
# Syntaxe classique
deploy:
  only:
    - tags

# Syntaxe rules (plus récente)
deploy:
  rules:
    - if: $CI_COMMIT_TAG
```

Revenir au sommaire

Question 24

Que fait `before_script` **dans un job ?**

A) S'exécute avant tous les jobs de la pipeline

B) S'exécute avant le script principal du job

C) Définit les prérequis du job

D) Configure l'environnement

Revenir au sommaire

Réponse : B

S'exécute avant le script principal du job

```yaml
deploy:
  before_script:
    - echo "Préparation..."
  script:
    - npm run deploy
  after_script:
    - echo "Nettoyage..."
```

Revenir au sommaire

Question 25

Quelle est la meilleure stratégie de déploiement pour minimiser le downtime ?

A) Direct deployment

B) Blue-Green deployment

C) Big bang deployment

D) Manual deployment

Revenir au sommaire

Réponse : B

Blue-Green deployment

Deux environnements identiques :

- **Blue** : Version actuelle en production
- **Green** : Nouvelle version testée
- Basculement instantané du trafic

Revenir au sommaire

Question 26

Comment sauvegarder des screenshots Cypress en cas d'échec ?

A) `artifacts: paths: - cypress/screenshots/`

B) `artifacts: when: on_failure, paths: - cypress/screenshots/`

C) `save_screenshots: true`

D) C'est automatique

Revenir au sommaire

Réponse : B

artifacts: when: on_failure, paths: - cypress/screenshots/

```
test:e2e:
  artifacts:
    when: on_failure
    paths:
      - cypress/screenshots/
      - cypress/videos/
```

Revenir au sommaire

Question 27

Quel est l'ordre correct des stages pour une pipeline Vue.js ?

A) build → test → lint → deploy

B) lint → test → build → deploy

C) test → lint → build → deploy

D) deploy → test → build → lint

Revenir au sommaire

Réponse : B

lint → test → build → deploy

Ordre logique :

1. Vérifier le code (lint)
2. Tester (test)
3. Builder (build)
4. Déployer (deploy)

Revenir au sommaire

Question 28

Comment limiter un job aux changements sur certains fichiers ?

A) `files: - src/**/*`

B) `only: changes: - src/**/*`

C) `watch: - src/**/*`

D) `on_change: src/`

Revenir au sommaire

Réponse : B

only: changes: - src/**/*

```
test:expensive:
  script:
    - npm run test:e2e
  only:
    changes:
      - src/**/*.vue
      - src/**/*.js
```

Revenir au sommaire

Question 29

Que signifie DIND dans GitLab CI ?

A) Docker In Docker

B) Deploy In New Directory

C) Dynamic Instance Network Deployment

D) Docker Installation Node Dependency

Revenir au sommaire

Réponse : A

Docker In Docker

Permet d'utiliser Docker à l'intérieur d'un conteneur Docker :

```
build:docker:
  image: docker:24
  services:
    - docker:24-dind
```

Revenir au sommaire

Question 30

Comment arrêter automatiquement une Review App après 1 semaine ?

A) `timeout: 1 week`

B) `auto_stop: 7 days`

C) `auto_stop_in: 1 week`

D) `expires_in: 1 week`

Revenir au sommaire

Réponse : C

```
auto_stop_in: 1 week
```

```yaml
review:
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    auto_stop_in: 1 week
```

Revenir au sommaire

Résultats 🎯

Calculez votre score !

- **25-30** : Excellent ! 🏆 Vous maîtrisez GitLab CI/CD
- **20-24** : Très bien ! 🎉 Quelques révisions mineures
- **15-19** : Bien ! 👍 Revoyez certains concepts
- **10-14** : Moyen 📚 Relisez le cours
- **< 10** : 💪 Continuez à pratiquer !

Revenir au sommaire

Points clés à retenir 

Revenir au sommaire

Configuration

- ✅ Fichier `.gitlab-ci.yml` à la racine
- ✅ Structure : stages → jobs → script
- ✅ Runners pour exécuter les jobs
- ✅ Variables pour la configuration

Revenir au sommaire

Optimisation

- ✅ Cache pour les dépendances
- ✅ Artifacts pour les résultats
- ✅ Parallélisation des jobs
- ✅ Conditions d’exécution intelligentes



Revenir au sommaire

Déploiement

- ✅ Environnements multiples
- ✅ Déploiements manuels/automatiques
- ✅ Review Apps pour les MR
- ✅ Stratégies Blue-Green, Canary

Revenir au sommaire

Sécurité

- ✅ Variables masked et protected
- ✅ Scans de sécurité
- ✅ Détection de secrets
- ✅ Audit des dépendances

Revenir au sommaire

Bravo ! 🎉

Vous avez terminé le QCM du Module Bonus CI/CD !

Vous êtes maintenant prêt à :

- ✅ Créer des pipelines CI/CD complètes
- ✅ Automatiser vos déploiements
- ✅ Optimiser vos workflows
- ✅ Sécuriser vos applications

Passez aux exercices pratiques pour consolider vos acquis ! 💪

Revenir au sommaire

Exercices pratiques

Module Bonus - CI/CD avec GitLab 🎯

Mettons en pratique !

Revenir au sommaire

Exercice 1 : Pipeline basique 🚀

Revenir au sommaire

Objectif

Créer une pipeline CI/CD simple pour TaskFlow avec installation, lint et tests.

Revenir au sommaire

Étapes

1. À la racine de votre projet, créer `.gitlab-ci.yml`
2. Configurer 3 stages : `install` , `lint` , `test`
3. Créer un job pour installer les dépendances
4. Créer un job pour linter le code
5. Créer un job pour exécuter les tests unitaires

Revenir au sommaire

Code de départ

```yaml
image: node:20

stages:
  # TODO: Définir les stages

# TODO: Configurer le cache

# TODO: Job d'installation

# TODO: Job de lint

# TODO: Job de tests
```

Revenir au sommaire

Solution - Configuration

```yaml
image: node:20

stages:
  - install
  - lint
  - test

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .npm/
```

Revenir au sommaire

Solution - Job d'installation

```yaml
install:dependencies:
  stage: install
  script:
    - npm ci --prefer-offline
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day
```

Revenir au sommaire

Solution - Job de lint

```yaml
lint:eslint:
  stage: lint
  dependencies:
    - install:dependencies
  script:
    - npm run lint
```

Revenir au sommaire

Solution - Job de tests

```yaml
test:unit:
  stage: test
  dependencies:
    - install:dependencies
  script:
    - npm run test:unit -- --run
```

Revenir au sommaire

Exercice 2 : Build et artifacts 📦

Revenir au sommaire

Objectif

Ajouter un stage de build et gérer les artifacts correctement.

Revenir au sommaire

Étapes

1. Ajouter un stage `build` après `test`
2. Créer un job qui build l'application
3. Configurer les artifacts pour sauvegarder le dossier `dist/`
4. Définir une expiration de 1 semaine
5. Le job ne doit s'exécuter que sur `main` et `develop`

Revenir au sommaire

Solution

```yaml
build:app:
  stage: build
  dependencies:
    - install:dependencies
  script:
    - echo "Building application..."
    - npm run build
    - echo "Build size:"
    - du -sh dist/
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
  only:
    - main
    - develop
```

Revenir au sommaire

Exercice 3 : Variables d'environnement 🔐

Revenir au sommaire

Objectif

Utiliser des variables pour différencier les environnements.

Revenir au sommaire

Étapes

1. Définir des variables globales (`NODE_ENV` , `APP_NAME`)
2. Créer un job qui affiche les variables GitLab CI/CD
3. Utiliser des variables spécifiques à un job
4. Configurer des variables dans l’interface GitLab

Revenir au sommaire

Solution - Variables globales

```yaml
variables:
  NODE_ENV: "production"
  APP_NAME: "taskflow"
  npm_config_cache: "$CI_PROJECT_DIR/.npm"

show:variables:
  stage: test
  script:
    - echo "Node Environment: $NODE_ENV"
    - echo "App Name: $APP_NAME"
    - echo "Branch: $CI_COMMIT_BRANCH"
    - echo "Commit: $CI_COMMIT_SHA"
    - echo "Pipeline ID: $CI_PIPELINE_ID"
```



Revenir au sommaire

Solution - Variables spécifiques

```yaml
deploy:staging:
  stage: deploy
  variables:
    DEPLOY_ENV: "staging"
    API_URL: "https://api-staging.taskflow.com"
  script:
    - echo "Deploying to $DEPLOY_ENV"
    - echo "API URL: $API_URL"
  only:
    - develop
```



Revenir au sommaire

Exercice 4 : Déploiement multi-environnements 🌐

Revenir au sommaire

Objectif

Configurer des déploiements vers différents environnements.

Revenir au sommaire

Étapes

1. Ajouter un stage `deploy`
2. Créer 3 jobs de déploiement :
   - `deploy:development` (automatique sur `develop`)
   - `deploy:staging` (automatique sur `main`)
   - `deploy:production` (manuel sur `tags`)
3. Configurer les environnements GitLab
4. Définir les URLs des environnements



Revenir au sommaire

Solution - Deploy Development

```yaml
deploy:development:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: development
    url: https://dev.taskflow.com
  script:
    - echo "Deploying to Development..."
    - npm run deploy:dev
  only:
    - develop
```

Revenir au sommaire

Solution - Deploy Staging

```yaml
deploy:staging:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: staging
    url: https://staging.taskflow.com
  script:
    - echo "Deploying to Staging..."
    - npm run deploy:staging
  only:
    - main
```

Revenir au sommaire

Solution - Deploy Production

```yaml
deploy:production:
  stage: deploy
  dependencies:
    - build:app
  environment:
    name: production
    url: https://taskflow.com
  script:
    - echo "Deploying to Production..."
    - npm run deploy:prod
  only:
    - tags
  when: manual
```

Revenir au sommaire

Exercice 5 : Tests avec couverture 🧪

Revenir au sommaire

Objectif

Configurer les tests unitaires avec rapport de couverture.

Revenir au sommaire

Étapes

1. Modifier le job `test:unit` pour générer la couverture
2. Extraire le pourcentage de couverture
3. Générer un rapport Cobertura
4. Configurer les artifacts pour les rapports
5. Afficher la couverture dans GitLab

Revenir au sommaire

Solution

```yaml
test:unit:
  stage: test
  dependencies:
    - install:dependencies
  script:
    - npm run test:unit -- --run --coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    reports:
      junit: coverage/junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
    paths:
      - coverage/
    expire_in: 30 days
```



Revenir au sommaire

Exercice 6 : Optimisation du cache 💨

Revenir au sommaire

Objectif

Optimiser les temps de build avec un cache intelligent.

Revenir au sommaire

Étapes

1. Configurer le cache basé sur `package-lock.json`
2. Utiliser différentes politiques de cache
3. Optimiser le cache pour les jobs de build
4. Mesurer l'amélioration des temps d'exécution



Revenir au sommaire

Solution - Cache global

```yaml
cache:
  key:
    files:
      - package-lock.json
  paths:
    - node_modules/
    - .npm/
  policy: pull-push
```

Revenir au sommaire

Solution - Optimisation par job

```yaml
install:dependencies:
  stage: install
  cache:
    policy: pull-push
  script:
    - npm ci --prefer-offline

build:app:
  stage: build
  cache:
    policy: pull
  script:
    - npm run build
```



Revenir au sommaire

Exercice 7 : Review Apps 🔍

Revenir au sommaire

Objectif

Créer des Review Apps automatiques pour les merge requests.

Revenir au sommaire

Étapes

1. Créer un job `review` qui s'exécute sur les merge requests
2. Configurer un environnement dynamique
3. Ajouter un job pour arrêter la Review App
4. Configurer l'auto-stop après 1 semaine

Revenir au sommaire

Solution - Review App

```yaml
review:
  stage: deploy
  dependencies:
    - build:app
  script:
    - echo "Creating Review App for MR..."
    - npm run deploy:review
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_COMMIT_REF_SLUG.review.taskflow.com
    on_stop: stop_review
    auto_stop_in: 1 week
  only:
    - merge_requests
```



Revenir au sommaire

Solution - Stop Review App

```yaml
stop_review:
  stage: deploy
  script:
    - echo "Stopping Review App..."
    - npm run destroy:review
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
  only:
    - merge_requests
```



Revenir au sommaire

Exercice 8 : Tests E2E 🎭

Revenir au sommaire

Objectif

Ajouter des tests end-to-end avec Cypress.

Revenir au sommaire

Étapes

1. Créer un job `test:e2e` dans le stage `test`
2. Utiliser l'image Docker Cypress
3. Sauvegarder les screenshots et vidéos en cas d'échec
4. Configurer le job pour qu'il puisse échouer sans bloquer

Revenir au sommaire

Solution

```yaml
test:e2e:
  stage: test
  image: cypress/browsers:node20.11.0-chrome121.0.6167.85-1
  dependencies:
    - install:dependencies
  script:
    - npm run build
    - npm run test:e2e:ci
  artifacts:
    when: on_failure
    paths:
      - cypress/screenshots/
      - cypress/videos/
    expire_in: 7 days
  allow_failure: true
  only:
    - merge_requests
    - main
```



Revenir au sommaire

Exercice 9 : Sécurité 🔒

Revenir au sommaire

Objectif

Ajouter des scans de sécurité à la pipeline.

Revenir au sommaire

Étapes

1. Ajouter un stage `security`
2. Créer un job pour l'audit npm
3. Ajouter un scan de dépendances GitLab
4. Configurer la détection de secrets
5. Les jobs de sécurité peuvent échouer sans bloquer

Revenir au sommaire

Solution - Audit npm

```yaml
security:audit:
  stage: security
  script:
    - echo "Running security audit..."
    - npm audit --audit-level=moderate
  allow_failure: true
  only:
    - merge_requests
    - main
```



Revenir au sommaire

Solution - GitLab Security Scans

```yaml
include:
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml

dependency_scanning:
  stage: security
  allow_failure: true

sast:
  stage: security
  allow_failure: true

secret_detection:
  stage: security
  allow_failure: false
```



Revenir au sommaire

Exercice 10 : Notifications 📣

Revenir au sommaire

Objectif

Ajouter des notifications Slack en cas de succès ou d'échec.

Revenir au sommaire

Étapes

1. Ajouter un stage `.post` (stage spécial qui s'exécute toujours)
2. Créer un job `notify:success`
3. Créer un job `notify:failure`
4. Configurer les conditions d'exécution
5. Ajouter une variable `SLACK_WEBHOOK_URL` dans GitLab

Revenir au sommaire

Solution - Notification succès

```yaml
notify:success:
  stage: .post
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{
        \"text\":\"✅ Pipeline réussie pour TaskFlow\",
        \"blocks\":[{
          \"type\":\"section\",
          \"text\":{
            \"type\":\"mrkdwn\",
            \"text\":\"*Pipeline réussie*\n• Branch: $CI_COMMIT_BRANCH\n• Commit: $CI_COMMIT_SHORT_SHA\n• Auteur: $CI_COMMIT_AUTHOR\"
          }
        }]
      }" \
      $SLACK_WEBHOOK_URL
  when: on_success
  only:
    - main
    - develop
```

Revenir au sommaire

Solution - Notification échec

```yaml
notify:failure:
  stage: .post
  script:
    - |
      curl -X POST -H 'Content-type: application/json' \
      --data "{
        \"text\":\"❌ Pipeline échouée pour TaskFlow\",
        \"blocks\":[{
          \"type\":\"section\",
          \"text\":{
            \"type\":\"mrkdwn\",
            \"text\":\"*Pipeline échouée*\n• Branch: $CI_COMMIT_BRANCH\n• Job: $CI_JOB_NAME\n• Pipeline: $CI_PIPELINE_URL\"
          }
        }]
      }" \
      $SLACK_WEBHOOK_URL
  when: on_failure
  only:
    - main
    - develop
```

Revenir au sommaire

Exercice 11 : Docker Build 🐳

Revenir au sommaire

Objectif

Créer une image Docker de l'application Vue.js.

Revenir au sommaire

Étapes

1. Créer un `Dockerfile` multi-stage
2. Ajouter un job `build:docker` qui build l'image
3. Pusher l'image sur le registry GitLab
4. Utiliser les variables GitLab pour l'authentification

Revenir au sommaire

Solution - Dockerfile Partie 1

```
# Stage 1: Build
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci --only=production

# Copy source code
COPY . .
```

Revenir au sommaire

Solution - Dockerfile Partie 2

```
# Build application
RUN npm run build

# Stage 2: Production
FROM nginx:alpine

# Copy built files
COPY --from=builder /app/dist /usr/share/nginx/html

# Copy nginx config
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Revenir au sommaire

Solution - Job Docker Build

```yaml
build:docker:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG .
    - docker tag $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG $CI_REGISTRY_IMAGE:latest
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG
    - docker push $CI_REGISTRY_IMAGE:latest
  only:
    - main
    - develop
```



Revenir au sommaire

Exercice 12 : Pipeline complète TaskFlow 🎯

Revenir au sommaire

Objectif

Assembler tous les éléments pour créer la pipeline complète de TaskFlow.

Revenir au sommaire

Fonctionnalités requises

1. ✅ Installation avec cache optimisé
2. ✅ Lint (ESLint + Prettier)
3. ✅ Tests unitaires avec couverture
4. ✅ Tests E2E
5. ✅ Scans de sécurité
6. ✅ Build de l'application
7. ✅ Déploiement multi-environnements
8. ✅ Review Apps
9. ✅ Notifications

Revenir au sommaire

Solution - Structure Partie 1

```yaml
image: node:20-alpine

stages:
  - install
  - lint
  - test
  - security
  - build
  - deploy
  - notify

variables:
  NODE_ENV: "production"
  npm_config_cache: "$CI_PROJECT_DIR/.npm"
  CYPRESS_CACHE_FOLDER: "$CI_PROJECT_DIR/.cypress"
```



Revenir au sommaire

Solution - Structure Partie 2

```yaml
cache:
  key:
    files:
      - package-lock.json
  paths:
    - node_modules/
    - .npm/
    - .cypress/
  policy: pull-push

# Template pour les jobs de test
.test_template:
  stage: test
  dependencies:
    - install:dependencies
  only:
    - merge_requests
    - main
    - develop
```



Revenir au sommaire

Conseil final 💡

Testez votre pipeline progressivement !

1. Commencer simple (install + lint)
2. Ajouter les tests
3. Intégrer le build
4. Configurer les déploiements
5. Optimiser et ajouter les fonctionnalités avancées

Revenir au sommaire

Projet fil rouge : TaskFlow 🎯

Revenir au sommaire

Défi final

Créer une pipeline CI/CD complète pour TaskFlow avec :

1. **Branches** :
   - `develop` → Déploiement automatique sur Development
   - `main` → Déploiement automatique sur Staging
   - `tags` → Déploiement manuel sur Production

Revenir au sommaire

2. **Tests** :
   - Tests unitaires (min 80% de couverture)
   - Tests E2E Cypress
   - Lint du code

Revenir au sommaire

3. **Sécurité** :
   - Audit npm
   - Scan de dépendances
   - Détection de secrets

Revenir au sommaire

4. **Review Apps** :
    - Environnement éphémère par merge request
    - Auto-stop après 1 semaine

Revenir au sommaire

5. **Notifications** :
    - Slack sur succès/échec
    - Email pour les déploiements en production

Revenir au sommaire

6. **Docker** (bonus) :
   - Build d'une image Docker
   - Push sur le registry GitLab
   - Déploiement via Docker Compose

Revenir au sommaire

Critères de réussite

- ✅ Pipeline verte sur `main`
- ✅ Déploiement automatique fonctionnel
- ✅ Tests passants avec bonne couverture
- ✅ Review Apps créées sur MR
- ✅ Notifications configurées
- ✅ Documentation du workflow

Revenir au sommaire

Ressources supplémentaires 📚

Revenir au sommaire

Documentation

- **GitLab CI/CD** : https://docs.gitlab.com/ee/ci/
- **GitLab Runner** : https://docs.gitlab.com/runner/
- **Docker** : https://docs.docker.com/
- **Cypress CI** : https://docs.cypress.io/guides/continuous-integration



Revenir au sommaire

Templates utiles

```yaml
# Import de templates GitLab
include:
  - template: Jobs/Code-Quality.gitlab-ci.yml
  - template: Jobs/SAST.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Jobs/Browser-Performance-Testing.gitlab-ci.yml
```



Revenir au sommaire

Outils de validation

- **GitLab CI Lint** : https://gitlab.com/ci/lint
- **YAML Validator** : https://www.yamllint.com/
- **Pipeline Editor** : Intégré dans GitLab

Revenir au sommaire

Bravo ! 🎉

Vous avez complété les exercices du Module Bonus CI/CD !

Vous maîtrisez maintenant :

- ✅ La création de pipelines CI/CD
- ✅ Le déploiement multi-environnements
- ✅ Les bonnes pratiques DevOps
- ✅ L'intégration continue avec GitLab

Continuez vers le QCM pour valider vos connaissances ! 📝

Revenir au sommaire

📋 Cheatsheet Vue.js 3

Guide de référence rapide 2025

🎯 Création et installation

Créer un projet

```
npm create vite@latest mon-app -- --template vue
cd mon-app
npm install
npm run dev
```

Packages essentiels

```
npm install vue-router@4
npm install pinia
npm install axios
npm install -D vitest @vue/test-utils
```

🧩 Composant de base

```
<template>
  <div>
    <h1>{{ titre }}</h1>
    <button @click="count++">
      {{ count }}
    </button>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const titre = ref('Hello')
const count = ref(0)
</script>

<style scoped>
div { padding: 20px; }
</style>
```

📝 Directives essentielles

```vue
<template>
  <!-- Conditions -->
  <div v-if="show">Visible</div>
  <div v-else>Caché</div>
  <div v-show="toggle">Toggle CSS</div>

  <!-- Boucles -->
  <ul>
    <li v-for="item in items" :key="item.id">
      {{ item.name }}
    </li>
  </ul>

  <!-- Binding -->
  <img :src="url" :alt="desc">
  <div :class="{ active: isActive }">
  <div :style="{ color: myColor }">
```

```vue
  <!-- Evenements -->
  <button @click="doSomething">
  <input @input="handleInput">
  <form @submit.prevent="onSubmit">

  <!-- Modificateurs -->
  <button @click.stop="...">
  <button @click.once="...">
  <input @keyup.enter="...">

  <!-- v-model -->
  <input v-model="text">
  <input v-model.number="age">
  <input v-model.trim="name">
  <input v-model.lazy="msg">
</template>
```

🔧 Props et Events

```vue
<!-- Composant Enfant -->
<script setup>
const props = defineProps({
  titre: { type: String, required: true },
  count: { type: Number, default: 0 }
})

const emit = defineEmits(['update', 'delete'])

const handleClick = () => {
  emit('update', props.count + 1)
}
</script>

<!-- Composant Parent -->
<template>
  <EnfantComponent
    :titre="monTitre"
    :count="25"
    @update="handleUpdate"
    @delete="handleDelete"
  />
</template>
```

Revenir au sommaire

Slots

```vue
<!-- Composant avec slots -->
<template>
  <div class="card">
    <header><slot name="header">Titre par défaut</slot></header>
    <main><slot>Contenu par défaut</slot></main>
    <footer><slot name="footer" /></footer>
  </div>
</template>

<!-- Utilisation -->
<template>
  <Card>
    <template #header><h1>Mon titre</h1></template>
    <p>Mon contenu</p>
    <template #footer><p>Pied de page</p></template>
  </Card>
</template>
```

Revenir au sommaire

🧮 Computed et Watch

```vue
<script setup>
import { ref, computed, watch } from 'vue'

const firstName = ref('John')
const lastName = ref('Doe')

// Computed (cache automatique)
const fullName = computed(() => {
  return `${firstName.value} ${lastName.value}`
})

// Watch
watch(firstName, (newVal, oldVal) => {
  console.log(`Changement: ${oldVal} -> ${newVal}`)
})

// Watch multiple
watch([firstName, lastName], ([newFirst, newLast]) => {
  console.log('Nom complet changé')
})
</script>
```

Revenir au sommaire

🔄 Lifecycle Hooks

```vue
<script setup>
import {
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted
} from 'vue'

onMounted(() => {
  console.log('Composant monté - Idéal pour les appels API')
})

onUnmounted(() => {
  console.log('Composant détruit - Nettoyer les événements')
})
</script>
```



Revenir au sommaire

🍍 Pinia - Store

```
// stores/counter.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  // State
  const count = ref(0)

  // Getters
  const doubleCount = computed(() => count.value * 2)

  // Actions
  function increment() {
    count.value++
  }

  return { count, doubleCount, increment }
})
```



Revenir au sommaire

🍍 Pinia - Utilisation

```vue
<script setup>
import { useCounterStore } from '@/stores/counter'
import { storeToRefs } from 'pinia'

const store = useCounterStore()

// Destructurer avec réactivité
const { count, doubleCount } = storeToRefs(store)
const { increment } = store
</script>

<template>
  <div>
    <p>Count: {{ count }}</p>
    <p>Double: {{ doubleCount }}</p>
    <button @click="increment">+1</button>
  </div>
</template>
```

Revenir au sommaire

🛤️ Vue Router - Configuration

```
// router/index.js
import { createRouter, createWebHistory } from 'vue-router'

const routes = [
  { path: '/', name: 'Home', component: () => import('@/views/Home.vue') },
  { path: '/about', name: 'About', component: () => import('@/views/About.vue') },
  { path: '/user/:id', name: 'User', component: () => import('@/views/User.vue'), props: true }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```



Revenir au sommaire

🛤️ Vue Router - Navigation

```vue
<template>
  <!-- Liens -->
  <router-link to="/">Home</router-link>
  <router-link :to="{ name: 'User', params: { id: 123 } }">User</router-link>

  <!-- Vue de rendu -->
  <router-view />
</template>

<script setup>
import { useRouter, useRoute } from 'vue-router'

const router = useRouter()
const route = useRoute()

// Navigation programmatique
const goToUser = (id) => router.push({ name: 'User', params: { id } })

// Accès aux paramètres
const userId = route.params.id
</script>
```

Revenir au sommaire

🧪 Tests avec Vitest

```
// MonComposant.spec.js
import { describe, it, expect } from 'vitest'
import { mount } from '@vue/test-utils'
import MonComposant from './MonComposant.vue'

describe('MonComposant', () => {
  it('affiche le bon titre', () => {
    const wrapper = mount(MonComposant, {
      props: { titre: 'Test' }
    })
    expect(wrapper.text()).toContain('Test')
  })

  it('incrémente au clic', async () => {
    const wrapper = mount(MonComposant)
    await wrapper.find('button').trigger('click')
    expect(wrapper.vm.count).toBe(1)
  })
})
```



Revenir au sommaire

🔧 Composables (Custom Hooks)

```
// composables/useMouse.js
import { ref, onMounted, onUnmounted } from 'vue'

export function useMouse() {
  const x = ref(0)
  const y = ref(0)

  function update(event) {
    x.value = event.pageX
    y.value = event.pageY
  }

  onMounted(() => window.addEventListener('mousemove', update))
  onUnmounted(() => window.removeEventListener('mousemove', update))

  return { x, y }
}

// Utilisation
const { x, y } = useMouse()
```



Revenir au sommaire

🎨 CSS et Styles

```vue
<template>
  <div class="container" :class="{ active: isActive }">
    <p :style="{ color: textColor }">Texte</p>
  </div>
</template>

<script setup>
import { ref } from 'vue'
const textColor = ref('red')
const isActive = ref(true)
</script>

<style scoped>
.container {
  padding: 20px;
  color: v-bind(textColor); /* CSS réactive */
}

/* Deep selector */
:deep(.child-class) { color: blue; }
</style>
```

Revenir au sommaire

🎯 Bonnes pratiques 2025

Organisation des fichiers

```
src/
├── components/     # Composants réutilisables
├── composables/    # Hooks personnalisés
├── stores/         # Stores Pinia
├── router/         # Configuration du router
├── views/          # Pages/vues
└── main.ts
```

Nommage

- **Composants**: PascalCase (`UserProfile.vue`)
- **Props**: camelCase (`userId`)
- **Events**: kebab-case (`@update-user`)
- **Composables**: `useCamelCase` (`useMouse`)



Revenir au sommaire

⚡ Astuces et raccourcis

```vue
<script setup>
import { ref, computed, watch } from 'vue'

// Lazy loading
const HeavyComp = defineAsyncComponent(() => import('./Heavy.vue'))

// v-memo (cache conditionnel)
<div v-memo="[valueA, valueB]">...</div>

// v-once (render une seule fois)
<div v-once>{{ staticContent }}</div>

// Provide / Inject
provide('theme', 'dark')
const theme = inject('theme')

// Template refs
const inputRef = ref(null)
onMounted(() => inputRef.value.focus())
</script>

<template>
  <input ref="inputRef" type="text">
</template>
```

Revenir au sommaire

📚 Ressources utiles

Documentation officielle

- **Vue.js**: https://vuejs.org/
- **Vue Router**: https://router.vuejs.org/
- **Pinia**: https://pinia.vuejs.org/
- **Vitest**: https://vitest.dev/
- **Vite**: https://vitejs.dev/

Outils recommandés

- **VS Code Extensions**: Volar, ESLint, Prettier
- **DevTools**: Vue DevTools (Chrome/Firefox)
- **UI Libraries**: Vuetify, PrimeVue, Quasar

Communauté

- **GitHub**: https://github.com/vuejs
- **Discord**: https://discord.com/invite/vue
- **Forum**: https://forum.vuejs.org/



Revenir au sommaire

🎯 Fil Rouge : TaskFlow

Notre Trello-like progressif

🚀 TaskFlow : Votre projet fil rouge

📱 Une application de gestion de tâches complète

Tout au long de la formation, nous allons construire **TaskFlow**, une application moderne de gestion de tâches (type Trello/Notion).

🎨 Fonctionnalités

- 📋 **Liste de tâches centralisée** : Toutes vos tâches au même endroit
- 📝 **Tasks complètes** : Titre, description, priorité
- 🏷️ **Catégories** : Personnel, Travail, Urgent (avec couleurs et icônes)
- 📊 **Filtres** : Toutes, Actives, Terminées
- 🔍 **Recherche** : Trouver rapidement vos tâches
- 📊 **Dashboard** : Vue d'ensemble de votre productivité (admin)
- 🔐 **Authentification** : Gestion des permissions (admin, editor, user)

Revenir au sommaire

🗺️ Progression Module par Module

📦 Module 1 – Les fondations

Composants de base :

- `TaskItem.vue` : Une tâche individuelle
- `TaskList.vue` : Liste de tâches
- `TaskForm.vue` : Formulaire d'ajout/édition
- `TaskFilter.vue` : Filtres de recherche

Concepts utilisés :

- Data binding
- Directives (v-for, v-if, v-model)
- Props & Events
- Computed properties
- Watch pour la recherche

Revenir au sommaire

📦 Module 2 - Composants réutilisables

Avec les Slots :

- `TaskCard.vue` : Carte générique avec slots (header, content, actions)
- `Modal.vue` : Modal réutilisable pour éditer/supprimer
- `BoardColumn.vue` : Colonne avec slot pour les tâches

Concepts utilisés :

- Slots simples
- Slots nommés
- Scoped slots (passer les données de task)
- Composition de composants



Revenir au sommaire

📦 Module 3 - Fonctionnalités avancées

Directives personnalisées :

- `v-auto-focus` : Focus automatique sur les inputs
- `v-click-outside` : Fermer les modals en cliquant dehors
- `v-tooltip` : Afficher des infobulles

Drag & Drop :

- Implémentation native HTML5 dans `BoardColumn.vue`
- Attributs `draggable` et événements `@dragstart` , `@drop`

SFC avancés :

- CSS Scoped pour chaque composant
- Variables CSS réactives pour le thème
- TypeScript pour les interfaces Task, Category, User

Revenir au sommaire

📦 Module 4 - State Management avec Pinia

Stores :

- `useTasksStore()` : Gestion des tâches (CRUD, filtres, persistence)
- `useCategoriesStore()` : Gestion des catégories de tâches
- `useAuthStore()` : Authentification & permissions (admin, editor, user)

Fonctionnalités :

- CRUD complet des tâches
- Liaison tasks ↔ categories
- Filtres intégrés (all, active, done)
- Persistence avec localStorage
- Communication entre stores



Revenir au sommaire

📦 Module 5 - Navigation avec Vue Router

Routes :

```
/                   → HomePage (page d'accueil)
/tasks              → Liste des tâches (TaskListView)
/task/:id           → Détail d'une tâche (TaskDetailView)
/categories         → Gestion des catégories (CategoriesView)
/dashboard          → Dashboard admin (statistiques)
/login              → Page de connexion
/404                → Page non trouvée
```

Features :

- Lazy loading de toutes les vues
- Guards pour protéger `/dashboard` (admin uniquement)
- Guard pour protéger toutes les routes sauf `/` et `/login`
- Navigation programmatique (`$router.push`)
- Redirection après login avec query param `?redirect=`



Revenir au sommaire

📦 Module 6 - Tests avec Vitest

Suite de tests complète :

Tests unitaires :

- Logique du store (ajout/suppression tâches)
- Filtres et recherche
- Calculs (tâches terminées, etc.)

Tests de composants :

- TaskItem affiche correctement les props
- TaskForm émet les événements
- Drag & drop fonctionne

Tests d'intégration :

- Flux complet : créer → modifier → déplacer → supprimer
- Filtres + recherche ensemble
- Navigation entre boards



Revenir au sommaire

🎨 Architecture finale de TaskFlow

```
TaskFlow/
├── src/
│   ├── components/
│   │   ├── TaskItem.vue
│   │   ├── TaskList.vue
│   │   ├── TaskForm.vue
│   │   ├── TaskCard.vue
│   │   ├── BoardColumn.vue
│   │   └── TaskModal.vue
│   ├── views/
│   │   ├── HomePage.vue
│   │   ├── TaskListView.vue
│   │   ├── TaskDetailView.vue
│   │   ├── CategoriesView.vue
│   │   ├── DashboardView.vue
│   │   ├── LoginView.vue
│   │   └── NotFoundView.vue
│   ├── stores/
│   │   ├── tasks.js        # Store principal des tâches
│   │   ├── categories.js   # Catégories avec liaison tasks
│   │   └── auth.js         # Authentification et permissions
│   ├── directives/
│   │   ├── autoFocus.js
│   │   ├── clickOutside.js
│   │   ├── tooltip.js
│   │   └── index.js        # Plugin de directives
│   ├── router/
│   │   └── index.js
│   └── __tests__/
│       ├── components/
│       └── stores/
```

Revenir au sommaire

🧩 Exemple de Task (données)

```ts
interface Task {
  id: number;
  title: string;
  description: string;
  priority: 'low' | 'medium' | 'high' | 'urgent';
  categoryId: number;
  done: boolean;
  createdAt: string;
  updatedAt?: string;
  completedAt?: string;
}

interface Category {
  id: number;
  name: string;
  color: string;
  icon: string;
}

interface User {
  id: number;
  email: string;
  firstName: string;
  lastName: string;
  role: 'admin' | 'editor' | 'user';
}
```


Revenir au sommaire

💡 Pourquoi ce fil rouge ?

🎯 Avantages pédagogiques

- **Progression naturelle** : Chaque module ajoute des fonctionnalités
- **Contexte réel** : Vous comprenez POURQUOI on utilise chaque outil
- **Portfolio** : Une vraie app à montrer en entretien
- **Apprentissage actif** : Vous construisez, pas seulement vous écoutez

🚀 À la fin de la formation

Vous aurez :

- ✅ Une application **complète et fonctionnelle**
- ✅ Un projet **déployable** (Vercel/Netlify)
- ✅ Une **vraie compréhension** de Vue.js
- ✅ Des **compétences recherchées** par les entreprises

Revenir au sommaire

🎨 Aperçu visuel (wireframe)

